

A friendly introduction

Edge Computing

Perry Lea



MANNING

Edge Computing MEAP V03

1. [Copyright 2023 Manning Publications](#)
2. [welcome](#)
3. [1 The Edge, the Cloud, and the hype](#)
4. [2 Digital transformation history](#)
5. [3 Edge systems componentry](#)
6. [4 Software and system frameworks](#)
7. [5 Connecting things \(networking and communications\)](#)
8. [6 Edge protocols: The language of Edge machines](#)
9. [7 Making the Edge work through AI](#)
10. [8 Security at the Edge](#)
11. [9 Edge computing futures and predictions](#)



MEAP Edition Manning Early Access Program Edge Computing A friendly introduction Version 3

Copyright 2023 Manning Publications

©Manning Publications Co. We welcome reader comments about anything in the manuscript - other than typos and other simple mistakes. These will be cleaned up during production of the book by copyeditors and proofreaders. <https://livebook.manning.com/#!/book/edge-computing/discussion>

For more information on this and other Manning titles go to

manning.com

welcome

Thank you for purchasing the MEAP for *Edge Computing*. This book is custom tailored for a wide audience. Edge computing involves a large number of technical domains ranging from hardware design, communications systems, networking, cloud computing, machine learning, embedded systems, and security. This book is perfect for managers, executives, marketing, finance, and of course seasoned engineers and technologists. The book is written in a way to expose the reader to edge computing and how all these domains come together to build a commercially successful product.

There are countless books on machine learning to communications theory and cloud computing. However, there is no comprehensive material tying these technologies together in a manner that anyone can understand how an edge system works. Additionally, there is quite a bit of material on IoT systems that are tailored for hobbyists and maker movements, edge computing extends IoT for commercial enterprise and consumer products.

It is important to consider how real-world edge use cases operate. The content includes countless commercial applications and use cases to develop concrete examples to base your designs and products around. They provide good contrast in the wide variety of edge systems. Additionally, there are use cases in the book that should serve as cautionary examples especially around catastrophic security failures on edge systems. I spend time dissecting these use cases in nearly every chapter of the book.

By the end of the book, you should have a good breath of knowledge of how edge systems work and where they are applicable. You should have the tools and vernacular to understand edge and IoT systems and apply that learning to real-world products and businesses. Please let me know what comments and suggestions you have in the [liveBook Discussion forum](#).

-Perry Lea

In this book

[Copyright 2023 Manning Publications welcome](#) [brief contents](#) [1 The Edge, the Cloud, and the hype](#) [2 Digital transformation history](#) [3 Edge systems componentry](#) [4 Software and system frameworks](#) [5 Connecting things \(networking and communications\)](#) [6 Edge protocols: The language of Edge machines](#) [7 Making the Edge work through AI](#) [8 Security at the Edge](#) [9 Edge computing futures and predictions](#)

1 The Edge, the Cloud, and the hype

This chapter covers

- The prevalence of edge computing.
- A standard definition of edge computing versus other forms of information technology.
- Defining the purpose, usefulness and business case that empowers edge computing.
- Understanding real-world use cases where edge computing creates value and impact.

Welcome to edge computing. This chapter covers the introduction to the edge. More importantly, it talks about the edge reality and edge hype. Today technology is cluttered with buzzwords and hype-circles that prognosticators, media, and analysts pitch routinely. Edge computing is this era's buzz word. Is edge computing a new hot topic of a recently discovered technology that will change the way we live? No, it is far from a new technical revelation, but it may change the way we interact, work, and entertain ourselves in a digitally connected world.

Edge computing has extensions to the Internet of Things (IoT), 5G and cellular communication, machine learning, Industrial IOT (IIOT), Long Range/Low Energy Wireless (LoRaWAN), and Industry 4.0 (all buzzwords to their own extent). You purchased this book for clarity and guidance around this subject and are trying to determine how edge computing may fit into your organization or business. Above all, you want to know: “what is it and what can it do for me”. In short, there is a massive amount of data being generated in the world. Each year 20% more data is generated than the previous year. This makes sense with nearly everything being digital and interconnected. Consider these facts at the time of this writing:

- There are between 3.5 and 4 billion daily users of smartphone devices worldwide.
- Nearly 45 billion IoT connected devices and sensors are online transmitting data.
- In 2019, Netflix video streaming alone was responsible for 61% of all network bandwidth.
- 254,000,000,000,000,000 bytes of information were transmitted across the internet.

Each one of these devices whether it's a sensor, a smartphone camera, or a server is a source of data generation. Much of the data created in the world is not even created by humans, they are autonomous machine creating data that are consumed by other machines. The availability of cheap computing, cheap sensors, nearly unlimited storage, and fast methods of data communication have allowed the internet and modern digital computing to impact nearly every aspect of life.

Edge computing serves a purpose by bringing programmable machines close to where data is generated or data is consumed. Its purpose is to fill a gap between computing in the cloud versus computing exclusively at the source of data (on-prem or on-premise). We will learn in chapter two that this is not a revolutionary concept and has been around for decades.

Edge computing for all intents and purposes is nearly the same from a basic hardware concept as a computer in the cloud. They have processors, memory, storage, and communications. However, the difference is in scale. Typically edge computers will not have the processing performance, storage capacity, and networking fabrics of a machine found in a hyperscale datacenter. They will be smaller, remotely located, and in some sense less capable. Edge machines pick up where the cloud leaves off. Think of an entire computing device providing the services of a large datacenter in the form factor of a small Raspberry Pi single-board computer. That said, a key word here is "typical". Edge computers are not typical, as matter of fact in many cases they are bespoke and designed for the environment and workload they need to do the job. Some edge machines are servers and identical to what you see in a data center (we will explore content delivery as an example later). However, most edge systems are significantly smaller and purposely designed as we shall see.

Now we will dive into a more formal description and the core principles of what edge computing is. This definition will be used throughout the rest of this book.

1.1 How to Read Edge Computing

This book is intended to be focused on the high-level overview of edge computing and all the constituent components that make up edge hardware, software, and services. It is intended to quickly instruct and inform the reader on edge computing use cases, the myriad of technologies forming the edge, and the risks of improperly designed systems.

One does not need a deep technical background to get something out of this book. This work is targeted for both senior technologists and engineers that need to quickly ramp up their understanding to include edge systems. Someone versed in technology will find that edge computing involves a variety of disciplines from communications systems and theory to embedded systems design to security. This book is designed to also be understandable for project managers and senior executives that need to understand what the edge is and how to separate the hype from reality. The material should also be relatable by providing concrete real-world use cases that have been deployed and are commercially successful. After all, if the technology doesn't make business sense, then it is more of a hobby.

1.2 What is the Edge?

We need to apply rigor in defining what this edge is. The edge has lots of definitions and many organizations misuse the term. For now, the six basic goals of edge computing are as follows:

1. Reduce the latency in processing, storing, or transmitting data.
2. Reducing the cost of cloud and telecommunications by filtering data and aggregating data sources.
3. Bridging networks and communications from simple sensors and non-IP capable networks to IP and internet capable networks.
4. To provide resiliency of computing and critical communications to remote areas as well as devices that are moving. We all have witnessed

dropped calls and lost connectivity, which is not acceptable for critical applications.

5. To provide security and data assurance. In some situation data must be anonymized, scrubbed or “denatured” of content before being send to the cloud.
6. Edge devices can programmatically compute, but also are remotely managed/administered. In some cases, they form an extension of the cloud, in other cases they are more autonomous.

These requirements cannot be satisfied with today’s current infrastructure of cloud computing. The hyperscale public and private clouds provide a relatively frictionless computing ability that can automatically scale globally. Cloud infrastructure consists of powerful server blades, high speed petabyte storage clusters, 400-gigabit optical networks, megawatts of power distribution, air and liquid cooling machinery, and the hardened security that surrounds them are few and far between. For example, worldwide Google has 20 datacenters spread between North America, South America, Asia, and Europe. Microsoft managed 58 data centers, and Amazon has 22. These are built in areas where there is geographical need as well as reliable and inexpensive power. The cloud serves a different function than the edge.

Edge devices touch the physical world, whether that’s a patient in a hospital, a fleet of trucks, a surveillance system in New York City, or users watching Netflix videos on a smartphone. These uses cases produce as well as consume massive amounts of data and require real-time control with as little latency as possible. Think of the edge as a “micro” datacenter that exists outside of the confines of the megawatt-capable billion-dollar datacenter.

It’s a fair question to ask: “why aren’t there only edge machines and why require a public cloud?” The answer is that the services provided by the cloud will still need to exist. There is no better method for much of the compute required for business applications and the internet in general. Software-only tasks such as global websites, large database applications, machine learning training, file and data archives, and office productivity/collaboration tools run seamlessly on the cloud. Edge devices have a limited set of resources. Clearly a large datacenter can provide significantly more compute and storage resources than an edge device. However, datacenters are centralized and built to house thousands of servers

close to the cheapest power plant or dam. Edge machines are found everywhere and often are in mobile or moving systems like vehicles. They reside in the harshest of environments and where communication and reliable infrastructure is something to be concerned about. Cloud customers can't afford disruption of infrastructure. The edge is built to robustly handle disruption.

Another factor separating cloud computing from edge is ownership. Cloud technologies became prevalent and are now mainstream due to the digital transformation of on-premise data centers that were owned, managed, and funded by companies and private enterprises. This came at a substantial cost in capital equipment like servers and networking systems. The cloud allowed companies to shed the up-front capital costs that did not amortize as well as using a hyperscale public cloud. The bottom line is that public clouds are financially and technically better than private datacenters (except in some cases of security and specialty systems). Edge devices are a different story. In many cases edge computing is owned by a service provider or the final customer. While some cloud providers are extending their reach into edge computing either through dedicated hardware or software middleware, most edge systems are privately owned. The reason is that edge systems are nuanced. There is no general edge computer you can deploy that will work in farm machinery as well as palliative care home health monitoring. Edge systems are also deeply customized for end solutions. They can be bespoke and custom designed for their mission.

How would current life be different without edge computing? How has it made any difference? Technologies that are dependent on real-time decision making, low latency, and reliability are built on the edge. Take Netflix for example. Originally, Netflix video content was archived and delivered from Amazon AWS cloud centers exclusively. While this can work, three problems arose:

1. The amount of data Netflix consumer on the internet backbone fabric grew year over year.
2. Users demanded near instantaneous play without lengthy buffering.
3. Content size was increasing as users migrated from 1080p HD videos to 4K videos this increased file sizes by 4x – exacerbating the problem.

Without moving streaming systems to closer to the subscribers, the backbone would become saturated and lead to more bottlenecks and poor latency.

Netflix decided to build edge streaming systems that could be deployed at various Internet Service Provider (ISP) datacenters that were closer in many cases to the subscribers. This allowed traffic on the backbone of the internet to be reduced while also eliminating several hops across internet routers to AWS. Netflix calls these edge systems “Open Connect Appliances” and they sit in secured ISP centers across various regions. These machines are essentially datacenter blades with similar memory and processing capabilities. The machines are solely managed by Netflix. As matter of fact, the machines cannot be accessed by the ISP operator and are intended to be self-monitoring and self-reliant.

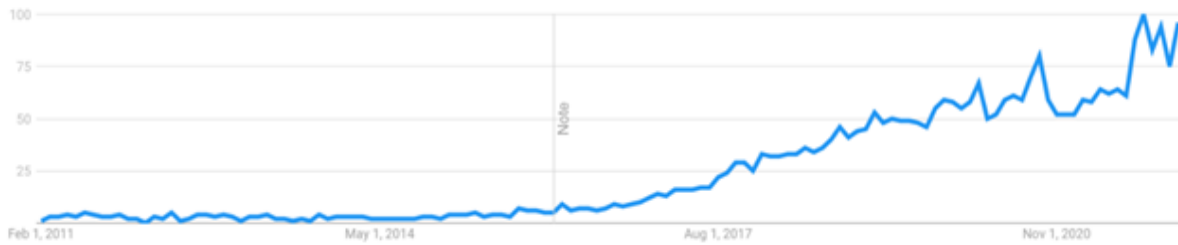
While we should now have a clear definition of edge computing, we will next describe what edge computing isn’t and importantly how it differs from cloud computing and the Internet of Things.

1.3 Edge Hype and What the Edge is Not

While edge systems are certainly part of recent hype and conference buzzwords, it is not a new concept entirely nor is it revolutionary. Internet connected devices have been in existence for over 40 years. One of the first remote devices connected to the Internet was a soda machine in the computer science department of Carnegie Mellon University in the 1980s. It allowed computer science students to buy soda remotely via their workstations. The edge however is different, it complements cloud computing by extending its reach to remote devices and machinery.

However, innovative technologies generate hype. One can look at Google search trends to see the interest in edge computing. In 2015 there was barely a mention of edge computing in marketing nor technical literature. Interest continues to peak month after month well into 2022. Shown in figure 1.1, one can see the steep interest in edge computing.

Figure 1.1 Google Search Trends 2011 to 2022 on Edge Computing



With peaking interest comes hype. Edge computing serves many functions, and we will see how it is essential in different use cases. There are limits and edge computing does not have a place in some environments. This book should help you understand what the edge is and where to use it. Just as importantly, it should help you understand where not use edge technology.

The edge is not a replacement for public cloud services. The cloud and massive hyperscale datacenters will continue to serve a function for a vast majority of information technology. Often is the case, that native cloud applications act as the overall administrator and orchestrator of “worker” edge nodes. The cloud serves this function well as it is centrally managed and scales with application growth.

Edge systems, also, are not standalone embedded systems and microcontrollers. As will be discussed in chapter 2, we have had embedded machines running everything from televisions to toasters for decades. What these embedded machines and computers have lacked was connectivity. Edge systems require communication in multiple directions.

Edge computing is also not another term for the Internet of Things (IoT). While most IoT devices are edge computers since they are connected and intelligent, not all edge computing systems are IoT devices. An example of this is Netflix edge computing to aid in streaming video. These are public cloud servers that are found close to the consumer. IoT devices are smaller and technically simpler than edge computers.

We continue on by diving deeper into the architectural components that define edge computing.

1.4 Edge Architecture

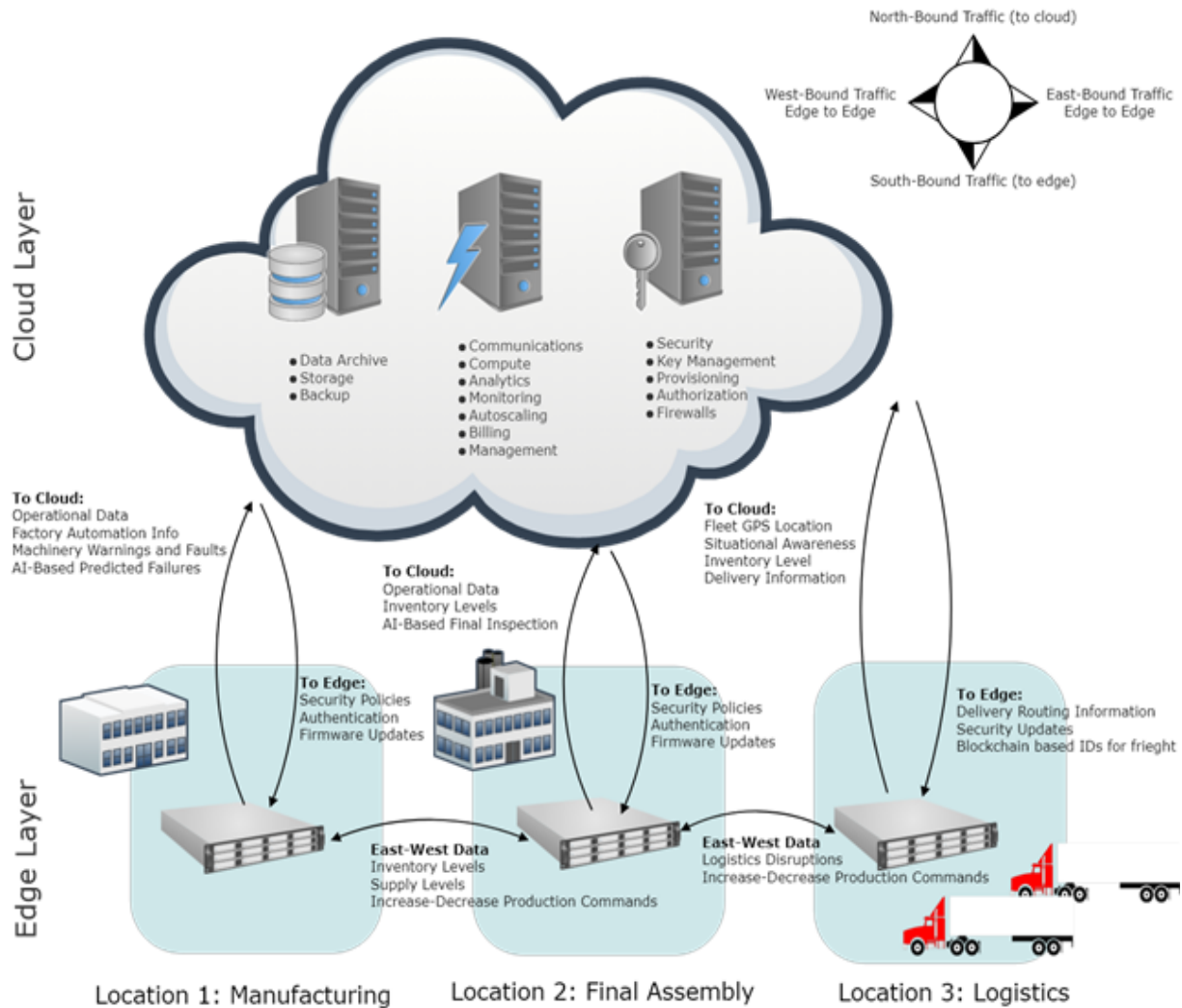
We should set the record early in this book: edge computers and edge computing are simply the same computers and services that would see in a cloud datacenter or in an embedded device. The difference between traditional computing and edge computing is the function they serve and where they live. There are also constraints within the edge that need to be factored. This chapter will supply context and definition as to what the “edge” is and what problems it resolves.

There are several viewpoints that also define edge computing as “any form of computing outside of a datacenter”. This is broad and could include even the smallest embedded systems in a vacuum cleaner or a microwave. Others claim edge computing to be a form of computing that reduces latency. It can be argued that latency is just one problem resolved by edge devices. Furthermore, depending on the distance and number of hops to a traditional datacenter, latency may not be an issue.

Our definition of edge computing is a form of computing services and hardware that provides processing, storage and programmable control close to the sources of data generation or data consumption. Edge computers may be extensions of applications and services hosted in a datacenter, but they do not necessarily need to be. One aspect of all edge systems is that they are connected by networks to relay data and for management. Therefore, simple embedded microcontrollers found in the bulk of machinery are not considered edge computers.

A typical edge topology is shown in following figure 1.2 below. The cloud provides the central authority for services like data warehousing and archiving, control and administration of the collective system, and for overall security and provisioning. We show three edge nodes that in three disparate locations. Typically, edge systems will be associated with a location while the cloud has no physical presence.

Figure 1.2 Cloud to Edge Overall Architecture. This diagram depicts the separation between cloud and edge layers in a system. Here data moves from cloud to edge (South-Bound) to deliver cloud-controlled firmware updates, security policies, and control information. Data moving from the edge to cloud (North-Bound) conveys edge generated data including status information, AI predicted failures, and logistic GPS coordinates. East-West data typically transfers data from edge node to edge node. Here, edge systems quickly communicate information to start or halt production based on real-time logistics issues or inventory level.

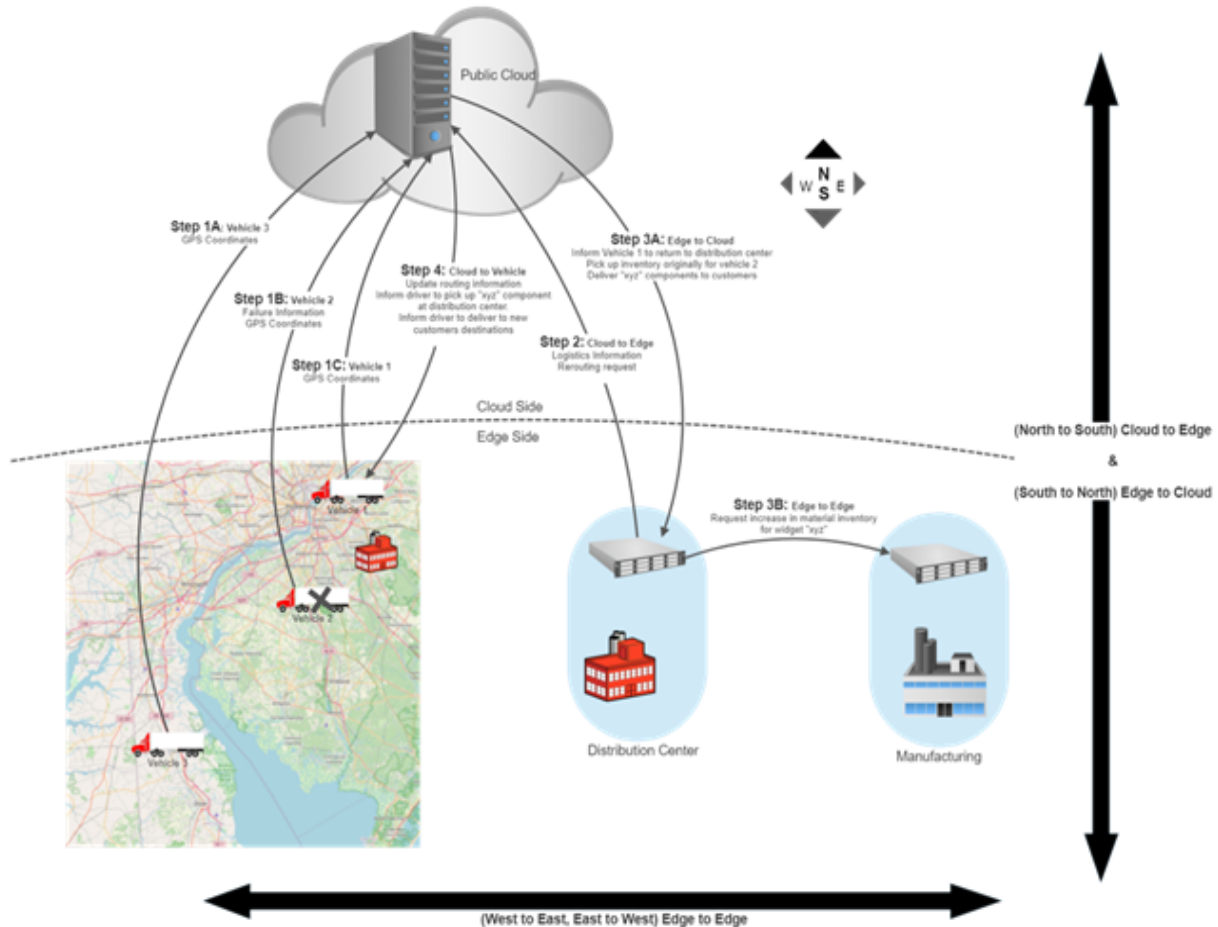


Another key aspect in the figure is the communication between the edge nodes and the cloud. We call this “North-South” traffic. We speak of “North-bound traffic as originating from the edge and propagating to the cloud. “South-bound” traffic is the reverse where traffic originates in the cloud and moves to the edge. Communication between the edge and cloud can be raw data for archival or exchange of security keys. The other form of traffic that is important in edge architectures is “East-West” traffic. This is data exchange between various edge nodes with each other and do not directly involve communication with the cloud. Think of this communication between two edge computers managing an industrial production line. They communicate by providing time-critical data between a machine in final assembly and a machine in manufacturing to alert each other if there is a

disruption in service or if manufacturing should ramp down in case of decrease demand.

Let's expand on the dataflow in the edge example from figure 1.2. In typical edge systems, disparate edge machines utilize the public cloud for as a means to communicate with remote devices. In this case, the fleet of trucks and vehicles need the public cloud to route data to and from the distribution centers. The vehicles may be using cellular 5G communication provided by cellular carriers and will not have direct communication with the distribution nor manufacturing sites. The cloud servers are also logging and storing vehicular GPS coordinates as well as delivery plans. It is making real-time predictive judgements on the best route for the company's fleet. In the example in figure 1.3, we can assume one of the vehicles has an unintentional failure or breakdown.

Figure 1.3 Example dataflow in a typical edge system. Here a fleet of trucks are delivering widgets to customers throughout the Philadelphia metropolitan region. All trucks deliver real-time updates on GPS locations, vehicle status, delivery completions, and inventory levels to cloud. This allows for real-time logistics analysis and the ability to reroute vehicles in the event of an issue.



All vehicles report their status, position, inventory levels, and customer completions to the cloud asynchronously (steps 1A, 1B, and 1C). Vehicle 2 is subject to an unexpected failure. At that time, the cloud software informs the distribution center who makes receives all information, delivery status, and customers impacted and makes the decision to reroute vehicle 1 to assume vehicle 2 deliveries. This decision is based on the deliveries vehicle 2 is scheduled as well as the proximity of the closest vehicle to take over. It is also based on the inventory of “xyz” widgets at the distribution center to cover the supply. The distribution center also uses East-West communication to the factory to schedule production of additional “xyz” widgets to cover the inventory gap.

1.5 Edge Computing Use Case Examples: Problems Resolved by Edge Computing

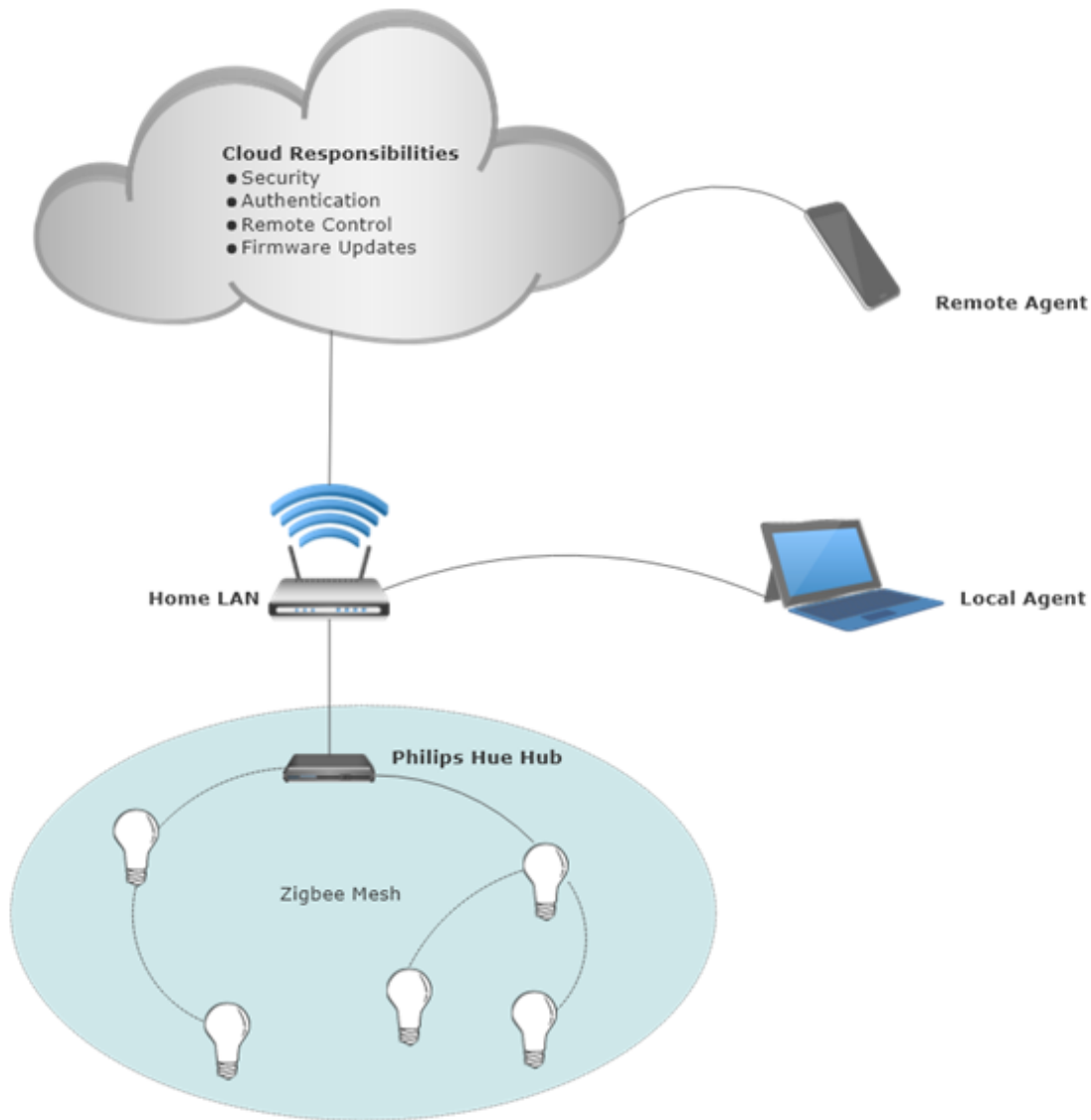
To illustrate how edge devices are being deployed and used we will look at specific examples and analyze what problems they resolve, how they have been implemented, and what results they have obtained.

1.5.1 Use Case: IOT Gateway

This is the most familiar example for consumers as well as enterprises. Here an edge computer is in close proximity to IOT sensors, actuators, and output devices. In a consumer application, a fair analogy of this edge architecture is a smart lighting hub like the Philips Hue system. Philips Hue systems are smart lighting appliances that use energy efficient LED bulbs connected in a mesh network. A mesh is simply an adhoc network where data traverse across different nodes (in this case actual light bulbs) before arrive to a central hub. Traditional networks such as WiFi and Ethernet use a hub-spoke model where a device communicates directly to a router or gateway. Local orchestration of devices in this mesh is administered by a Philips Hue hub device. This hub for all practical reasons fits the bill as an edge computer:

- It communicates and manages all the smart lights in the Philips mesh using a personal area network (PAN) called Zigbee.
- The hub also connects to cloud services across a customer's local area network (LAN) and internet service.
- The cloud supplies remote management through a Smartphone application, provisioning and security, and firmware updates for the hub and lights.
- The hub is a simple microcontroller that could be considered a small embedded system as it is a fixed function device.

Figure 1.4 Edge Use Case of Philips Lighting Gateway



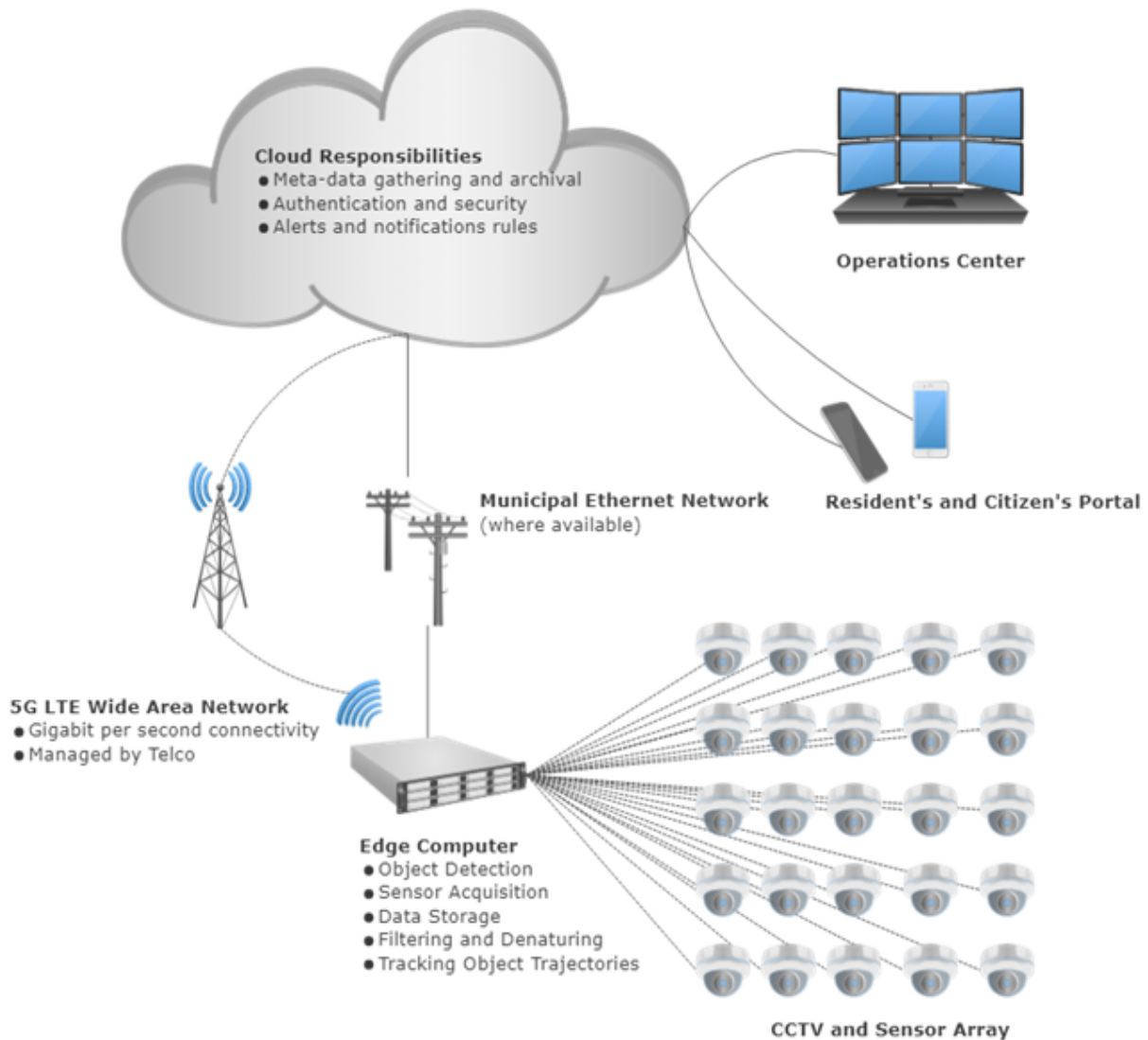
This architecture is most efficient. Without a central edge computer each light would need to have communication to the cloud and internet. This would have added considerable cost and complexity to each bulb that has a strict cost constraint.

There are issues with architectures like this. It relies on a non-Internet Protocol (IP) called Zigbee for the lights to communicate. Since it is not IP based it introduces security risks with non-authorized users try to hijack the mesh network.^[1] We dedicate an entire chapter to edge security later in this book.

1.5.2 Use Case: Smart City Surveillance

A very common use case of edge computing is in the area of Smart City deployments. Often municipalities or even large corporate campuses employ edge computing to add significant computational abilities to otherwise dumb sensors and systems. One such example are surveillance systems that utilize many cameras to monitor the security, safety, and behavior. Smart cities use camera systems to monitor for potential crime events, parking utilization, crowd control, traffic fees and violations, as well as environmental health. The constraint that necessitates edge computing is the sheer amount of volume of data that is generated by camera and other unstructured data sources. Unstructured data refers to data that doesn't follow a regular pattern or separable field representation. A database is a good example of structured data where information is arranged into specific fixed length "slots or fields". Video and audio data are essentially random values of colors and tones and this form of data is random, unordered, and can't be divided into separate fields effectively. Unstructured data also tends to be large. Think about HD 1080p video. Even when we compress the data an HD camera will generate between 8 to 100 megabytes of data per minute using modern compression technology. The data also has real-time deadlines and latency constraints. For example, viewers of streaming video usually will not tolerate frame rates below 30 frames per second (FPS).

Figure 1.5 Smart City Use Case



For a smart city deployment, a recent pilot project in the UK used existing CCTV video cameras paired with air quality and noise sensors to determine traffic and pollution problems in a metro area. Reusing existing CCTV allowed for reduced cost, but a problem immediately became evident. As previously mentioned, each camera is capable of streaming HD images at 60 frames per second. On average, these twenty cameras would generate 160 megabytes of data per minute or 6,912,000 megabytes (6.9 petabytes) of data per month. The intent of the video data was to analyze the content using machine learning algorithms to detect pedestrians, vehicles, bicycles and safety issues. If all that data would need to be transmitted to the cloud, the telecommunication service costs, latency, and cloud storage costs would have made this project prohibitive from the start. Analyzing the data on the

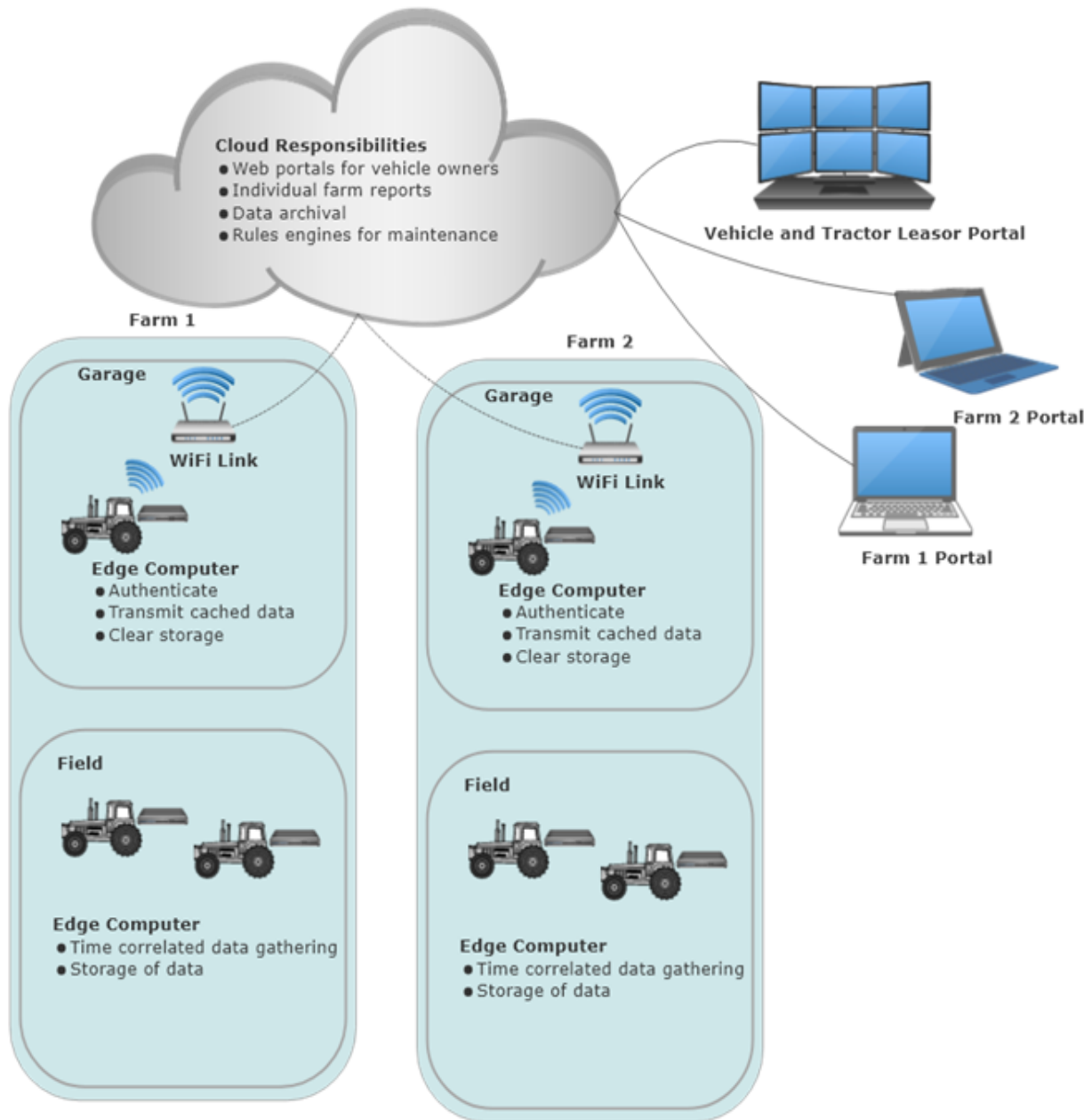
edge is the only method to reduce the network service level agreement (SLA) costs, and to reduce latency in responding to an issue. The architecture of this deployment is illustrated in figure 1.4.

1.5.3 Use Case: Vehicle Telematics

An emerging use case for edge computing is in vehicle telemetry which also includes logistics and vehicular tracking. Whether this is for a fleet of delivery vehicles, municipal snowplows, or first responders in emergency situations; there is a need for edge computing in moving and non-stationary situations.

One implementation is in agriculture. A customer that manages several large farms in Canada needed a system to extract tractor engine health data as well as utilization of equipment. Often agricultural machinery is leased and understanding the usage patterns can help provide predictive maintenance for the owners of the machinery and the farm business. These farms are in remote areas that have no or unpredictable cellular coverage. The customer needed a method to record and log information of the tractor (and associated machinery), while the machine is in use. When the vehicle returned to the farm garage, a standard wireless Wi-Fi signal would be present and the cached data would stream from an edge computer on the tractor to an associated cloud system maintained by the leaser of the tractor.

Figure 1.6 Vehicular Edge Computing



In this example, the edge computer consisted of a small, embedded computer with a large amount of storage capacity. The edge computer had a number of wired interfaces to various sensors and other computers on the tractor. It captures time-series data on engine health, hours used, distance travelled, multiple temperature inputs, humidity conditions, oil and fluid reserves, and so on. This data would be cached in the edge computer storage until the vehicle returned to its garage where it would automatically pair with a WiFi module. At that time the time would be transmitted, and the edge storage

flushed to a clean state. If there were issues or if maintenance was required cloud rules engines would make the recommendation and pass notification to the tractor supply company as well as the end user (the farm owner).

This is a widespread problem in moving and non-stationary IT systems. Often there is no guaranteed or reliable communication channel. A design must be robust enough to manage spotty cellular coverage by utilizing edge computing systems to run autonomously without constant connectivity to the cloud.

Vehicular edge computing has found its way into consumer vehicles as well. Nearly all manufactured vehicles today have some form of connected computing. Whether this is vehicular infotainment systems or automated driving and steering (ADAS) as found in Tesla vehicles, the compute power in an automobile is profound and they are connected to cloud systems (as well as other vehicles) to update maps, receive over the air software upgrades, calculate traffic patterns, and find the nearest electric vehicle charger.

1.5.4 Use Case: Video Streaming Services

A new use case for edge computing is in video streaming services and cloud-based gaming. These types of services are also called content-deliver networks or CDNs. The explosive growth of streaming content on-demand has exposed weaknesses in the traditional cloud to customer communication model. Consider that streaming of video data is again a huge amount of unstructured data. Netflix, YouTube, and other streaming services accounted for 36.2% of the world's total internet traffic in 2019. A typical 4K Netflix movie that is about 1 ½ hours long is about 10 gigabytes in size. An average person also will also want a sustained frame rate of about thirty frames per second or higher with little “stuttering”. Stuttering is when latency drops, and frames arrive late or not at all. Skipped frames are evident to someone watching a video in real-time.

In a typical cloud to client type of service, compressed and ready to serve content is stored in a cloud datacenter some distance from the client. Data will stream from the server across multiple hops in the internet before arriving at the client's device. There can be substantial and unpredictable

network behavior once data leaves the control of a datacenter. Typical internet data may traverse between different high-speed switches and router in the datacenter and then exit on the internet backbone. From there data progresses through narrower communications channels (e.g. local Internet service provider) as it approaches the “edge” or where the client device resides. All these traversals add latency and variability in how fast data can move.

For example, between my computer that I’m writing this book on now (Eagle, Idaho) and Google’s closest datacenter (Mountain View, California) a single byte of data had to cross 9 “hops” to get to me each adding a small amount of overall latency. This is shown in table 1.1.

Table 1.1 Example of Internet Hops and Additive Latency

Hop	Location	Physical Location	Cumulative Time - Latency (milliseconds)
1	Computer to cable modem	Eagle, Idaho	7.3ms
2	Modem to Internet Service Provider (ISP)	Boise, Idaho	14.3ms
3	ISP internal routing	Boise, Idaho	14.4ms
4	ISP internal routing	Boise, Idaho	16.5ms
5	Level 3 Internet IXP	Seattle, WA	22.0ms
6	Level 3 Internet IXP	Seattle, WA	24.2ms

7	Google	Mountain View, CA	26.1ms
8	Google	Mountain View, CA	27.6ms
9	Google Server Destination	Mountain View, CA	28.5ms

This additive latency as network packet migrates through the internet lead to loss of quality in video streaming. Additionally, as more users demand streaming services, datacenters in geographic region become saturated with requests. The subtlety here is that video streaming follows the sun. By that I mean, that there are peak hours and days of demand that follows a work/home/sleep cycle. Highest demand for Netflix streaming is during the hours of 5PM to midnight and that follows respective time zones across the globe. Demand is cyclical and variable.

To combat demand cycles, increasing consumer demand, and latency issues we turn to edge computing. In this case, the computing services are video streaming devices brought closer to the consumers. They are extensions of streaming servers found in traditional datacenters but eliminate the number of network hops a data must propagate to arrive at a customer. Netflix developed an edge architecture called “Open Connect” to address these challenges.

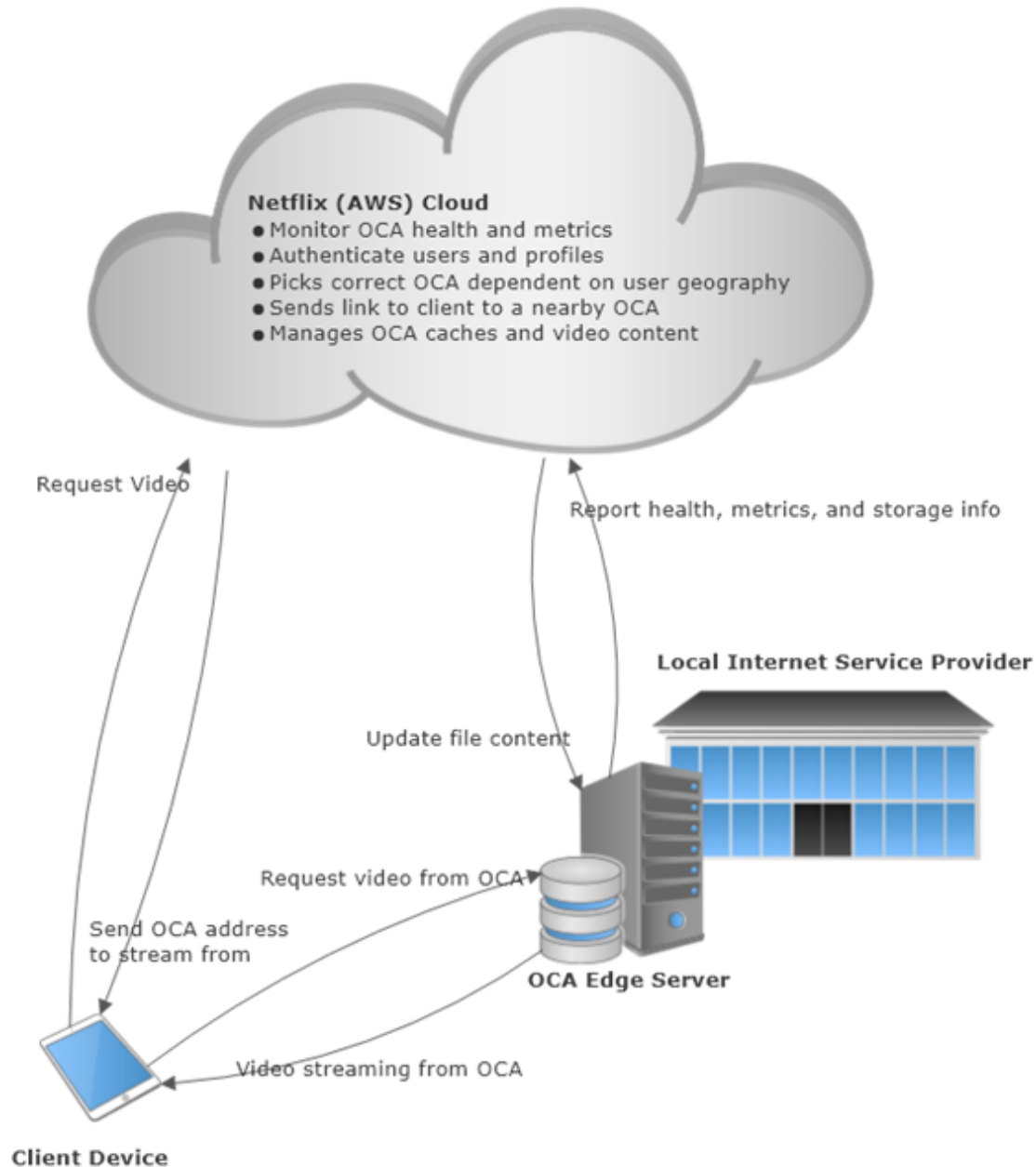
Open Connect is built on datacenter server class hardware but these appliances (OCA for short) are placed in ISP centers across the globe. These devices are video storage caches. The devices can be placed at Internet Exchange Points (IXPs). In the table above, the Layer 3 network in Seattle would be an example of an IXP. Alternatively, the edge server can be collocated (COLO) at a partner ISP datacenter close to the end user.

These are video caching machines intended to reduce latency and balance network demand. No user data such as credit card information, credentials, or personal information is stored within an OCA server. Rather, the Netflix cloud (hosted on Amazon's Cloud AWS) manages all user interactions. OCA edge machines perform these two functions:

- Report status, health, routing information, and video storage information to Netflix cloud.
- Stream content using HTTP/HTTPS to a client device.

Netflix main cloud control manages what we call a “control plane” while the OCA devices manage the “data plane”. The control plane in the cloud will change and modify the videos stored on each OCA based on demand dynamically. This is important because some regions may have video and movie content that has more popularity than other regions or may be language specific. The control plane also identifies the user's location and will match an OCA to the user dependent on their distance, latency, and bandwidth to an appropriate OCA. Below is a graphic illustrating this technology.

Figure 1.7 Example of Edge Computing Used in the Netflix Content Delivery Network



1.6 Summary

- Edge computing is a fast-growing technology and research area that is finding its application in a number of industries.
- We have established that edge computing serves a viable function when there are constraints that traditional cloud-based infrastructure cannot

remediate, nor can an information device exclusively reside without larger cloud connectivity.

- If a system is constrained by too much added **latency** in migrating data to and from the cloud or if the **cost** of that data movement is too high an edge system has a purpose.
- Additionally, relying exclusively on a cloud system can add risk if a system needs to be **resilient**.
- **Bridging** networks and communications systems is also a common trait of edge machinery by providing the ability for non-typical networks like Bluetooth to communicate through the Internet.
- Edge devices also **extend** the cloud to devices that normally have never been connected to the Internet.
- Edge devices can **filter** data the customer does not want to migrate to a public cloud.
- The cloud-layer and edge-layer each serve their unique function and systems are partitioned to run services on each. This was demonstrated in working examples from Netflix content delivery systems to smart-city municipal surveillance systems to vehicular and agricultural telemetry in very remote areas – all while being connected to the Internet.

We will progress in the next chapters to talk about history of IT that progressed and enabled edge computing. We will then study the underlying hardware and software structures that form typical edge machines.

[\[1\]](#) E. Ronen, A. Shamir, A. O. Weingarten and C. O’Flynn, "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, 2017, pp. 195-212.

2 Digital transformation history

This chapter covers

- How Moore's Law and Dennard Scaling has built the foundation of the IT industry and is now being exploited on the edge.
- Why information technology has shifted back and forth between centralized and decentralized computing.
- What embedded systems are and how they differentiate edge machines.
- The differences between the Internet of Things and edge computing.

This chapter will lay out the history and trends in IT and technology that got us to edge computing. Before we dive deep into “edge” and edge use cases, we should pause to align ourselves on technical trends and address some questions of why IT shifted from single-use machines to mainframes, to personal computers, to clouds, and now a hybrid of cloud and personal computing.

Information and computing technology have advanced significantly over the last sixty years. There have been many “digital transformations” over that period as well. This book concentrates on edge computing, but we need to start by understanding how and why edge computing devices came to be. Advances in silicon processing, nanoscale electronics, computer architecture, and software have improved the power of computing devices. In 1956, an IBM 7090 mainframe cost \$2.9 million dollars (in 1956 currency) and could perform 100,000 operations per second. The machine was widely used from controlling the NASA Mercury space flights to managing the first airline reservation and tracking system. Over the progress of 60 years of research and development, a device like the Microsoft Xbox can manage 10,000,000,000,000 operations per second for a few hundred dollars and fit on a shelf. We have seen between a performance increase of 100 million-fold and a 38,000-fold decrease in cost. This has led to different models of computing over the years and has given rise to the Internet of Things (IoT) and edge computing which this book is about.

Today sensors, mobile devices, intelligent systems, AI, cloud computing, and streaming entertainment seems ubiquitous and pervasive. One may consider much of technology in the 2020s to be interoperable and seamless. You can view a Netflix video on your set-top box, pause it, and resume where you left off on your smartphone from a completely different area without losing quality or where you stopped the movie. However, the fact is that there is significant infrastructure and backend services that make that a reality. Edge computing is a critical part of the overall infrastructure that resolves many issues in modern IT. We now examine how the information revolution over the past 50 years has enabled and led the industry to edge computing.

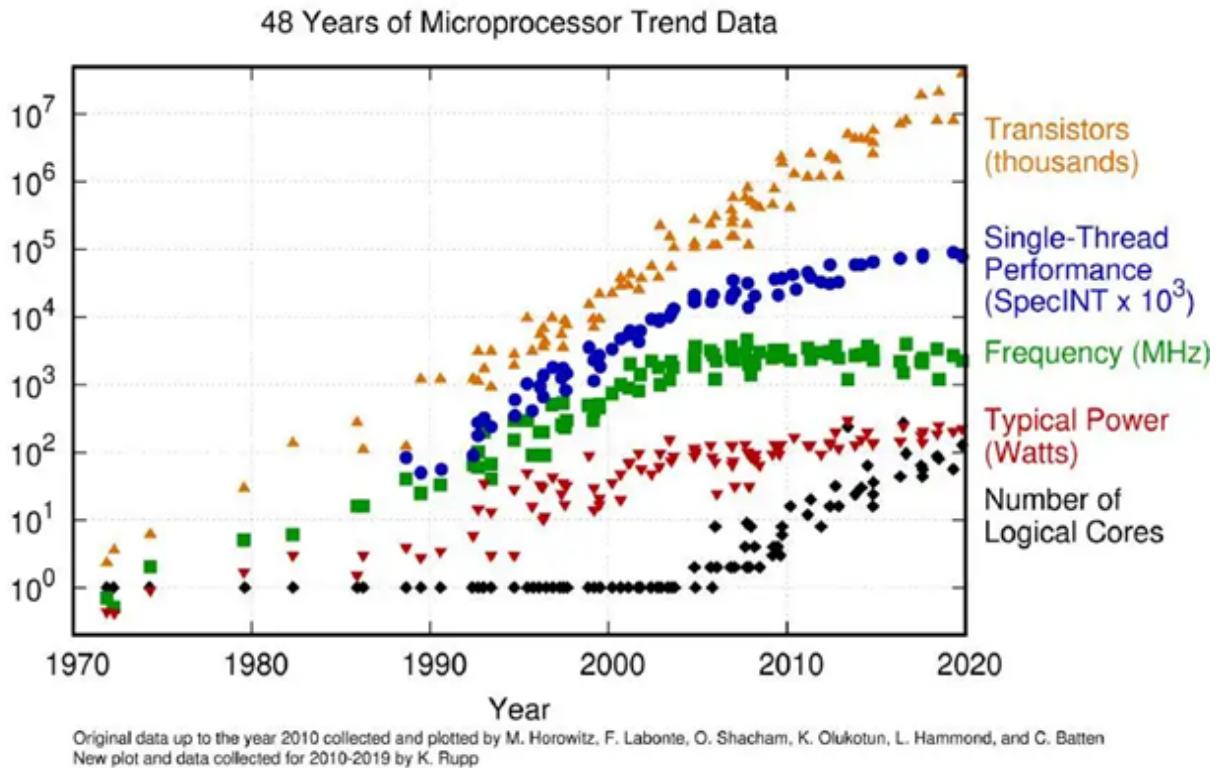
2.1 How We Got Here

We start examining edge computing by describing how information has transformed itself many times within the last sixty years of information revolution. When the IBM 7090 computer was designed, it was during a time when mainframe systems grew to relevance. Computers during that time were significantly expensive, took large square footage of specialized floor space, and were essentially single tasked. “Single task” meaning they performed a fixed function such as managing airline reservations or piloting the trajectory of a Mercury space capsule. As matter of fact, renowned author Arthur C. Clarke in 1974 stated that the computing power of a large mainframe would be reduced to the size of a console and available to every household. Of course, engineers knew this, as they were proceeding to ride the technology and financial curve of Moore’s Law. Moore’s Law stated that the number of transistors in an integrated circuit would double every two years. Indeed, that was true for thirty years since Gordon Moore made that prediction. Mainframes did make way for personal computers. However, devices started becoming much more intelligent as electronics and microprocessors found their way into everything from video game consoles to home printers to automobiles. Microprocessors and software allow for design flexibility. Before microprocessors existed, much automation was through crude mechanical or electronic circuits. By enabling software in systems, devices became much more intelligent and could change their programming on-the-fly.

While Moore's Law was the enabler that allowed computers and information technology to become more powerful and much more economical, there is another law that also impacted the way we use computers. Dennard Scaling (named after the researcher Robert H. Dennard) predicts that as Moore's Law reduced the physical dimensions of a transistor and they grow smaller and smaller, the power consumed stays constant. What this means is that not only transistors shrinking in size, but the power they use to do the same calculations stays constant. Put another way, the performance of computers was increasing by 40% every generation but with a looming issue of power consumption. This is especially prevalent today with a typical datacenter that hosts several thousand high-performance servers consuming megawatts of energy – enough to power small cities. Power consumption remains a large issue even in small systems that are powered by batteries.

The figure below shows the contrast between Moore's Law (increasing the number of transistors and the number of processor cores) versus Dennard Scaling limits. Even though more transistors can be placed on a silicon chip, the power limits how fast the chips can operate which graphically appears as the leveling off of single-threaded performance and frequency. The principal issue is that more transistors on a chip dissipate increased power in the form of heat. This heat has reached a point where the chip will burn itself up and become non-functional.

Figure 2.1 Graphic illustrating Moore's Law and Dennard Scaling limitation in modern IT processors and semiconductors.



While the trend of Moore's Law and Dennard Scaling has had an impact on building some of the most complex microprocessors and silicon devices in high-end servers and computers, the Internet of Things and edge computing devices that may not use the most advanced and fastest high-performance silicon is now riding the wave of innovation and performance. This is one of the reasons why it has become economically practical to connect remote devices through the Internet. Edge computing and IOT devices are exploding in growth and opportunity just as the PC revolution grew in the 1990s and 2000s.

Some termed the 1990s as the PC revolution. Large mainframes ceased to exist, and computers were cheap and powerful on every desk but were interconnected by a myriad of different networks and mixed protocols. In the 1980s, as an example, there were six competing commercial network standards from Apple (Appletalk), IBM (SNA), Novell (IPX/SPX), Microsoft (NetBEUI), and others competing against the open TCP/IP protocol. TCP/IP is ubiquitous today but owes its foundation to the United States Department of Defense. It was developed as a research project funded by the Defense Advanced Research Projects Agency in the late 1960s and

early 1970s through collaborative work with academics. TCP stands for Transmission Control Protocol and IP stands for Internet Protocol. Together they comprise an open standard of the layered network “stack” used by nearly every piece of modern networking and communication equipment. All proprietary protocols became obsolete and are rarely used today.

We should also not lose the fact that moving data from one place to another in the 1980s was costly in terms of performance but also the equipment and infrastructure weren’t ready for the surge of data and users that exploded in the 1990s. What changed in the mid-90s was the advent of improved networking protocols such as HTTP that enabled the web, but also higher speed and less costly methods to move data such as cable broadband and telephony digital subscriber line (DSL).

The 1990s ushered in the digital revolution, or some call it the information revolution. A personal computer was present at nearly every office worker’s desk as well as in the home. PCs were more powerful with the advances by Intel releasing the Pentium processors and data movement was affordable with high-speed ethernet linking each computer. It was also during this time that wireless communication became prevalent, first with voice and audio communication, and then with data as the cellular industry moved from 2G service to 3G service around 2001. Additionally, advances in WiFi (802.11) networking were also enabling personal computers to be mobile.

2.2 The Computing Pendulum

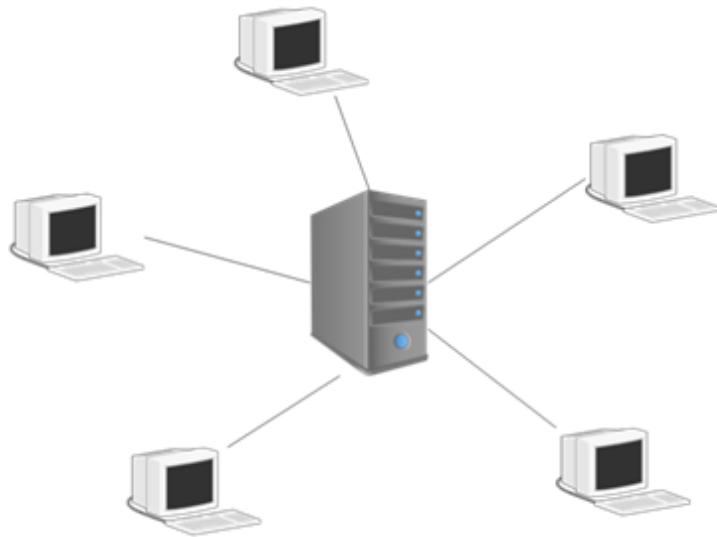
Originally mainframes were practical and, in all actuality, economical. In the 1960s there were few IT or computer companies outside of IBM, Control Data, Data General, and other large mainframe manufacturers. Mainframes were central to transaction processing tasks common to banks, billing departments, and records management. Typically, a set of “dumb” terminals would be connected to a mainframe. These terminals had the appearance of a stereotypical personal computer with a monitor but were nothing more than a screen and keyboard. The IT industry has moved from hub and spoke systems with central mainframes to distributed computing using PCs, and now the cloud has migrated back to large, centralized computing coordinating with smaller computing devices. This is the computing

pendulum. We often see such shifts in the IT industry as new use cases and computing trends evolve.

With the advent of minicomputers in the 1970s and PCs in the 1980s computing tasks shifted from central control in a mainframe to distributed and personal control on a PC. This was much more economical and led to a myriad of software packages that competed with one another. As connectivity and network standards evolved, these machines became interconnected. Often, they would be accompanied by an office server which was a shared file system and didn't provide much compute horsepower.

As networking technologies became faster and more prevalent, personal computers became satellites of larger and more powerful servers. Servers now started performing duties such as maintaining email and file systems. These servers became larger on-premise datacenters that IT organizations managed. Some companies at the time created "hub and spoke" architectures with a central computing node and peripheral smaller computers/workstations that surround the central node as shown in figure 2.2. Technical advances progressed throughout the 1990s until the internet provided a revolution in connectivity. No longer were "dumb" terminals needed to connect to a dedicated mainframe. No longer were PCs satellites of a datacenter server. Now computing could exist anywhere and so could servers.

Figure 2.2 Typical Hub and Spoke Connectivity with a Central Server and Peripheral PCs, Workstations and Dumb Terminals



It was in the 2000s through early 2010s that cloud computing became pervasive. At first these were servers managed by cloud providers that could be leased by corporations. Rather than spend capital for on-premise datacenters, these servers, and their respective costs could be expensed by another company like Microsoft or Amazon. It should be noted that the costs of managing a datacenter not only include the capital expenses of the computers, networking equipment, racks, buildings, and cooling systems but also include recurring costs of power consumption which we know now to be a very significant expense. This model of leasing computing equipment hosted elsewhere is what is termed a Platform as a Service (PaaS) in modern vernacular. This required a customer to manage, deploy, and administer the computer and software remotely. Later, datacenters started growing the number of services offered. Rather than simply leasing bare-bone hardware and having a customer manage all the software, cloud providers deployed services such as online databases and cloud storage systems for end users. This type of cloud computing is sometimes called Software as a Service (SaaS) and is prevalent in today's high-performance servers seen in typical hyperscale cloud datacenters.

These new cloud technologies reduced operating costs and capital expenditures but caused a rift for IT management. How would a centralized IT department manage security and operations with a public cloud? We will address this issue throughout this book as it's a central theme for successful edge computing.

Today we have many services and systems that are managed, hosted, and deployed in large cloud datacenters and mobile devices feeding data into and out of such cloud systems. For example, most web sites are cloud hosted and content is delivered to a tablet or smartphone through standard internet protocols and software frameworks. The cloud provides unmatched power and ease in developing and deploying software at a global level.

Operational costs for maintaining servers cannot be underappreciated. The costs in hardware, cooling systems, floor space, and support personnel on-prem were contributing factors in “outsourcing IT” and the move to public clouds. This model worked very well (to this date) for applications like hosting websites (www.manning.com) , massive data storage (www.wikipedia.com), and online productivity (www.office365.com) but fails to address the unique needs of edge computing.

We have seen how computing has migrated from large central mainframes to personal devices to a hybrid mix of cloud and mobile computing but where does the edge fall into this topography? We will now examine why edge computing became prevalent.

2.3 Everything Connected IT

Mainframes gave way to personal computers; personal computers gave way to mobile computing. While this is true, we are overlooking a major source of all computing which are embedded systems. What is an embedded system? Essentially any device that incorporates a programmable computer for a fixed purpose. Everything from personal computers to large datacenter servers is powerful and can run a variety of software packages, and operating systems, and perform countless tasks. Embedded systems typically have a processor, memory, and storage, not unlike a traditional PC or server, but will typically be a part of another machine in a non-traditional space. For example, a printer or copier uses an embedded controller to receive print jobs and control the imaging mechanisms. Automobiles have dozens of embedded systems for controlling infotainment systems, comfort and climate control, and of course autonomous driving. What is astonishing is that 98% of all microprocessors used in modern computing are not sold for use in personal computers, smartphones or even servers; they are used

exclusively in embedded systems. This is not a new phenomenon. Embedded controllers have existed for decades. Some of the earliest embedded computers include the Apollo Guidance Computer. Before that analog control systems were used by energy producers and the electric grid. During the 1960's the energy sector replaced these analog machines with small digital computers which became known as Programmable Logic Controllers (PLCs). These were the very first devices that could be considered embedded systems.

Embedded devices continued to flourish while the PC and IT revolution continued. They were designed, built, and purchased for different reasons. Embedded machines such as a printer or a factory automation machine would be designed for the users that used those systems: office workers and factory personnel. Personal computing devices evolved to be managed by an IT department. This disparity is one of the reasons that the Internet of Things has been difficult to grow.

The effect in enterprises set into motion some of the constraints we face in edge computing today. It was in the 1990's that the acronym "IT" became mainstream. This led to organizations funding IT departments. Their role was to manage the information assets of a company including PC's, mainframe, data centers, and communication systems. The IT management function was constrained to the physical PC's and networks within a company's brick-and-mortar establishment. Typically, an IT Department will have a charter similar to the following:

- Support and helpdesk functions for corporate users.
- Managing the health, reliability, and security of internal and external networks.
- Establishing approved hardware and software for corporate usage.
- Defining the processes and rules for information security.
- Managing who and what may reside on a corporate network.
- Data collection to improve and secure company assets and networks.

As mobile devices starting with laptops became more prevalent, IT management had to consider new tools and policies for a mobile workforce. No longer did all the IT managed assets (PCs, networks, and software) reside within the confines of a company's walled building, but a mobile user could

be anywhere on the globe using layers of networks that are not managed by the IT department. PCs today are remotely managed and secured. IT departments can push software updates, remove, or grant access to corporate networks, and monitor the health of IT systems. These policies make sense but have been problematic as computing devices have exploded in popularity.

2.4 Clouds, Mobility, IOT and The Edge

While local computing and cloud computing have a rich history and combine 60 years of technology development, there are gaps and issues. Computing devices such as PCs, smartphone, and tablets are prevalent for end users and yet they are fed by cloud enabled applications like video streams, databases, file storage, and web backends. For the most part this works well. When it doesn't work well is when there are issues with performance, reliability, or cost in moving data to and from a cloud datacenter and a mobile device.

Additionally, with Moore's law allowing hardware to scale exponentially in performance as well as decrease cost generation over generation we have seen the proliferation of electronics and embedded systems bringing intelligence, programmability, and connectivity to devices that simply had never been "connected" or "intelligent before". For example, in decade of the 1990s, devices such as smart thermostats, intelligent automobiles, and machine learning in factory automation were science fiction. Today we call this the "Internet of Things". We have the ability and need to connect everyday objects to the internet. We do this not just for the convenience or "wow" factor, but for safety, security, and to improve productivity. With billions of devices now connected to the internet and with embedded computing abilities we have a different infrastructure issue than with clouds and personal computing devices. IOT devices have power, size and space constraints that may prevent them from effectively connecting to a cloud directly. Furthermore, with the volume of IOT devices and the aggregate amount of data produced it becomes costly and impractical to move that data to and from the public cloud.

The Internet of Things is a twenty-first century buzz phrase, but the concept of remote monitoring and management dates to the first half of the twentieth century. In the 1930's and 1940's as energy generation was becoming mainstream, there rose the need to monitor and control remote power stations and sub-stations to an ever-expanding grid across the United States. To enable this process, control systems made use of "pilot wires". We might call this sideband communication today. They were emergency presentative circuits using a communication wire between substation relays that was used to detect faults in AC power. As the electric grid expanded and demand increased, so did the need for remote operation and dynamic control of power. In the 1950's power companies looking at methods to improve OPEX (operational expenses) and improve the reliability of generators installed analog computers as Energy Management Systems. These systems dynamically scheduled power generation on-demand and allowed different power companies to exchange energy. Additionally, the EMS systems allowed substations to be monitored and managed remotely without requiring an electrician to be resident 24/7. Later in the 1960's these analog machines were replaced with digital mainframes as they became commercially available.

The first Internet connected device (the first "thing" in Internet of Things) can be attributed to a graduate student, David Nichols, at Carnegie Mellon University in the early 1980s when less than 300 computers were connected to the early Internet. He was studying computer science at the renowned university and spent countless hours in his office some distance away from the vending machines on campus. He and two other students then added sensors to a remote soda machine and added logic to a network gateway connected to the University network. A larger mainframe called a PDP-5 communicated with the remote gateway and would indicate to the computer science students if the soda machine was stocked and even if the sodas were cold. While a novelty, this essentially exhibits all the traits of IOT devices and edge computing.

Edge computing has evolved with IOT technology. As IOT devices have become more complex and prevalent, edge system have evolved to provide more compute power, bridging services, and cloud extensions to devices. One can think of an IOT device as an edge computer, but not all edge computing devices should be thought of as IOT.

- **IOT device:** A physical object (heart rate monitor, smoke alarm, environmental sensor) that happens to be connected to the Internet (directly or indirectly) that may also include physical sensors and actuators. These devices can receive and transmit data without human interaction. IOT devices may have no display or visible feedback. IOT devices are typically massively deployed. Think about very low-cost intelligent sensors with limited compute capabilities. They require low-power and small form factor designs.
- **Edge device:** A direct Internet-connected computer with more compute power, storage, and networking facilities than a simple IOT devices. IT may or may not have physical sensors and systems, but not necessarily. Edge systems reside near the location data is consumed or gathered such as a factory floor or livestock feed lot. Edge systems may have a full display, touch screen, or a variety of human input devices. Finally edge systems typically have a deeper connection and coordination with a cloud system. Edge systems also sit between IOT devices and the cloud.

2.5 Summary

- Information technology has taken a winding course in architecture and structure over the last fifty years.
- This trend has followed the constraints of computing and new use cases over the years
- Moore's Law and Dennard Scaling have been instrumental in the explosive growth of information technology for the last 50 years, but we also are witness to the physical limitations of power that constrains the most advanced silicon designs.
- IOT and edge computing are now experiencing explosive growth since these industries do not generally rely on the fastest and most powerful processors and hardware, but are enjoying all the technology and advances the PC industry witnessed in the 1990's and 2000's
- IT industry has migrated back and forth between centralized and decentralized computing between mainframes to PCs to cloud computing.
- The Internet was a catalyst enabling the connectivity of everything. The Internet of Things came into existence in the early 1980s first as an

academic “hack” and now with billions of devices connected using standard protocols and networking concepts.

- IOT devices and edge computers share many of the same concepts, yet IOT devices are more closely aligned with remote “things” using simpler less powerful hardware and physical sensors. Edge computers are associated with more powerful computing than IOT but are physically close to where data is consumed or gathered.

Over the next two chapters we will explore the components of edge computing starting with edge hardware and then software.

3 Edge systems componentry

This chapter covers

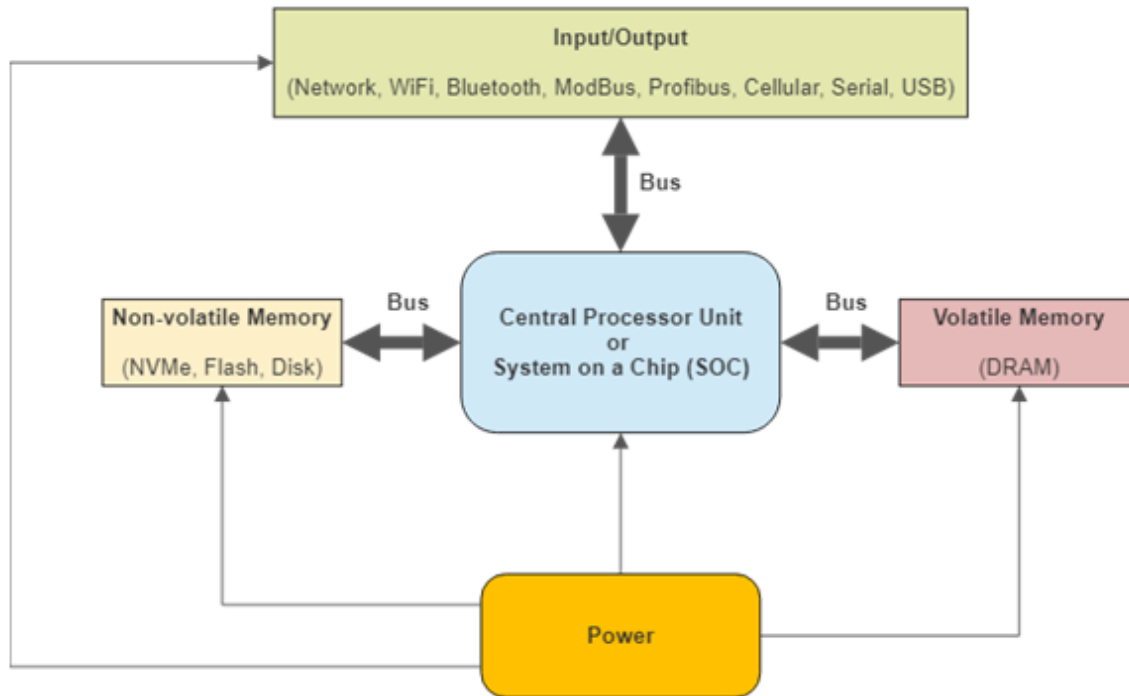
- Basic concepts of traditional computing systems and hardware.
- Examining high-performance edge hardware: HPE Edgeline computers.
- Examining constrained performance systems: Eurotech Reliagate hardware.
- Differentiating communication hardware for the edge.
- Understanding of the role of embedded systems.

Edge computing starts with edge hardware. Hardware components include everything from computing devices, sensors attached to the device, and communication systems between the device and the cloud. But hardware alone doesn't define the edge. Software is also unique for edge applications. As we will learn, edge applications require unique hardware designs as well as bespoke software and applications. We will look at the system architecture of edge machines and edge hardware. By "system" we mean how everything comes together from the edge to the cloud including communications and networking.

3.1 Hardware Architecture

We could complete this book without touching on any hardware discussion, but that would be disingenuous in the understanding of what the edge is and what it can do. After all, each type of edge device is unique. Any computing device consists of three main components: a processor that executes code and performs arithmetic and logical functions, memory to store code and data, and a bus that communicates a processor to various memories. Figure 3.1 captures the relevant blocks of any computing system.

Figure 3.1 Block Diagram of Modern Computing Hardware



In addition to the processor-bus-memory model, there also exists input and output devices such as USB ports and video ports. This type of architecture is typically termed a “Von Neumann” architecture after John Von Neumann, a prominent and influential physicist and mathematician during the first half of the 20th century. Power delivery is also needed and often overlooked but is a particularly onerous issue when we connect powerful edge computers in vehicles and remote areas of the globe. Whether a system is an edge device managing a self-driving car or even your cellphone, all modern IT hardware can be described in an abstract manner using this diagram.

Going a bit deeper into computer architecture, a central processor such as an Intel chip or perhaps an ARM processor is the core of the system. There may be a single chip with a single Central Processor Unit (CPU), or a single chip with multiple processors that can run in parallel. There may be additional classes of computing like a graphics processor unit (GPU) which is used for specific workloads in graphics, machine learning, or signal processing. A CPU is connected to other components via a bus. A bus makes up wiring to transmit and receive data as well as connect signals between components. Buses are important because they are responsible for all the data movement in these machines. The other side of a bus may include additional processors, storage systems and input/output devices. Storage systems take two forms:

volatile storage (which includes DRAM) is not persistent when power is removed from the system and non-volatile storage includes flash and disk which will retain data when power is removed. The final block of interest is input-output (IO) which includes everything from wide area communications like cellular radios and Ethernet LAN to slower near-range communication like Bluetooth or a legacy serial bus.

3.2 Different Classes of Edge Hardware

What we just described is essentially the computing structure of a Smartphone, a PC, or even video game console. Yet this is where general computing devices and edge computing devices start to differ. An edge system may require additional hardware to support different use cases. An edge server for Netflix may look very much like a server in a data center, but an edge computer monitoring effluence in a sewer system will need device hardening to allow it to survive the elements. Edge hardware may have custom IO that would never be used in your home. An example of custom buses and interfaces is ModBus, which would be used in heavy industry and factory machinery to connect robots and automation systems to computers. What needs to be reinforced is that there is not a single edge computer you can buy that satisfies all the requirements in the edge industry. Some need to act like server, some need to perform in a vehicle, and others may find their way into the remotest locations on earth. Here are two examples of edge computers and hardware on the market. Contrast the differences:

First, we examine The Hewlett Packard Enterprise (HPE) Edgeline EL8000t. This is a server class computer and sits in a sheet metal chassis. It consumes a substantial amount of power at 1500 watts (whereas a typical Smartphone uses about 10 watts). This machine also utilizes a robust “out-of-band” management communication system for mission critical computing. This allows an operator to monitor and control the server even if all other connectivity and internet access is lost. The computing power in this edge computer is substantial. Each processor can have up to 40 separate cores and access up to 6 terabytes of DRAM memory. Storage is through six ultra-high speed NVMe (non-volatile memory) modules. Additionally, an Nvidia high-performance Tesla T4 graphics processor can be installed for high-speed machine-learning and video creation. This entire blade can physically weigh

35 pounds or more and is intended to be housed in a rack and isolated from weather and environmental effects. While this sounds impressive, the machine has been designed to run edge computing applications. Essentially, this is a datacenter server and can cost anywhere between \$2000 to \$5000.

While the HPE edge solution is intended to be an extension of the datacenter using familiar server hardware at the edge, there are many (if not more) use-cases of smaller and less costly systems serving functions. For example, the Eurotech ReliaGATE 20-25. Eurotech is a manufacturer of smaller embedded systems devices, IOT devices, and gateways. The ReliaGATE is a standalone small hardened box that can withstand environmental elements. It uses a single Texas Instruments system on chip (SOC). The SOC has a single Intel Atom microprocessor running between 1.75 to 1.9 GHz. By comparison the Eurotech has about 1/1000 of the computing capability and speed as the HPE machine. IO, however, is extensive. It uses relatively slow 1Gbit Ethernet interfaces but includes a wide variety of extensible IO ports: CAN bus for industrial automation or automotive control, a vast array of digital IO for interfacing to switches and sensors, serial communication, a GPS for mobile positioning, and a cellular radio for mobile communication. The device can run industrial automation protocols (which we will cover later in this book) such as ModBus, MQTT, and S7, which are widely used in manufacturing and factory control systems. This device will cost about \$348 through various distributors.

A good way to contrast these two different mass produced edge computers is by referencing the following table.

Table 3.1 Contrast of Two Edge Computers (HPE Edgeline EL8000t and Eurotech Reliagate 20-25)

	HP Edgeline EL8000t	Eurotech Reliagate 20-25
Cost	\$2000 to \$5000 depending on configuration	About \$348

Processor	Intel Xeon 2.1GHz to 3.6GHz configurations 8 to 40 CPU cores.	Intel ATOM 1.75 to 1.9 GHz 2 or 4 cores
Other Processors (Graphics Processor Unit)	Optional Nvidia Tesla T4 GPU	None
Memory	Maximum of 6TB of DRAM	Up to 4GB of DRAM
Storage	High speed NVMe solid state storage 256GB to 40TB configurations	8 GB of eMMC storage
Input/Output (IO)	10-gigabit Ethernet, USB	1-gigabit Ethernet, USB, serial, CAN bus, ModBus, temperature sensors, accelerometer, audio in and out, LTE cellular, GPS
Power Supply	1500 Watt Power Supply	About 100 Watts
Reliability and Robustness	iLO (Integrated Lights Out) second control channel to check the	Eurotech Everywhere Software Framework for remote access.

	health of the edge computer.	
Security	Trusted Platform Module (TPM) and data encryption hardware.	None
Size and Mounting	3.5" x 19" x 16.91" Rack Mountable	2" x 7.5" x 5.6"
Environmental	-5 to 55 degree Celsius operating temperature No environmental, atmospheric or weather protection	-40 to 65 degree Celsius operating temperature IP30 environmentally hardened aluminum case

Clearly the Eurotech device has much less compute power than the HPE Edgeline server. So, what good is it? The HPE device is typically used to extend the data center closer to users and businesses that need extreme compute and storage close to where data is generated or consumed. This type of use case is also called the “converged edge”. One customer uses this type of system to run machine learning data analytic packages in a factory. Data streams into the device where machine learning workloads are trained to predict downtime in the manufacturing process. Customers may have security concerns and may not want data to leave the factory floor as the data generated is highly confidential. For example, edge computers monitoring the electric grid across the US require the data to be secure. These customers still want edge computers integrated to cloud backend services like billing the ERP (enterprise resource planning) software but need to securely steer where data is stored.

The Eurotech device does not have the CPU resources, memory, storage, or compute facilities to run any machine learning training very well. It would be too slow and/or simply not work. An edge machine such as this would have enough computational resources to run machine learning inference. Inference is the execution of a trained model and requires significantly less processing power. This device has been used in other factory automation control systems since it has a vast array of IO that integrates with legacy (also called brownfield) machinery through the ModBus protocol. A customer using it also needed it to be placed directly on the shop floor in a dusty and exposed environment where there was no infrastructure built to house and cool a server. The customer using the ReliGATE also had no networking service on the shop floor, therefore the cellular radio became imperative to provide instant Internet access in an area without any modern network infrastructure. The Eurotech can also be integrated easily in vehicles and is designed to use the common CAN bus communication interface prevalent in automobiles. Therefore, the Eurotech is the computer of choice when one is solving mobile, vehicular, or bespoke industrial settings.

This illustrates how differentiating and unique edge hardware can be. In summary, the HPE Edgeline system is roughly 1000 times as powerful as the Eurotech device and about 10x more costly. They both serve a function and target requirements for different edge computing users. If you need an edge machine that will store videos and serve them to customers in various cities, you will want an HPE Edgeline system in every municipality. If you are intending to manage and measure the flow of effluent in a sewer tunnels, the Eurotech is a much better choice. The point of this is that edge computing hardware is unique per application.

3.3 Everything Connected Through Embedded Systems

We keep referring to “embedded systems” but what are they and how are they different than a traditional computer? They are critical to all of IT. 99% of all computing devices that exist are embedded systems versus traditional personal computers, servers and even smartphones. Billions of embedded systems exist in every automobile, TVs, appliances, and light poles.

Whereas mainframes gave way to personal computers, personal computers gave way to mobile computing. While this is true, we are overlooking a major source of all computing which are embedded systems. Essentially any device that incorporates a programmable computer for a fixed purpose. Embedded systems are targeted to a specific task. Often, they are constrained by power limitations, size constraints, and environmental hardness. Embedded systems like Eurotech's ReliGATE 20-25 are crucial to the edge computing market.

3.4 Communication Hardware

While edge computers share similar architectural traits with their server, mobile, and PC equivalents, there are some fundamental differences. An important aspect of edge systems is the breath of different input and output communications systems the edge requires. While we will study networking and communications later in this book, the hardware aspect requires some attention.

Edge computers find themselves in many different places and configurations. For example, some edge systems will manage vehicle telemetry such as monitoring a fleet of trucks or municipal waste vehicles. Other edge computers will find themselves managing a factory floor with industrial equipment. The way these edge machines communicate and control the equipment they attach to is unique and what makes edge systems powerful but also complex to design.

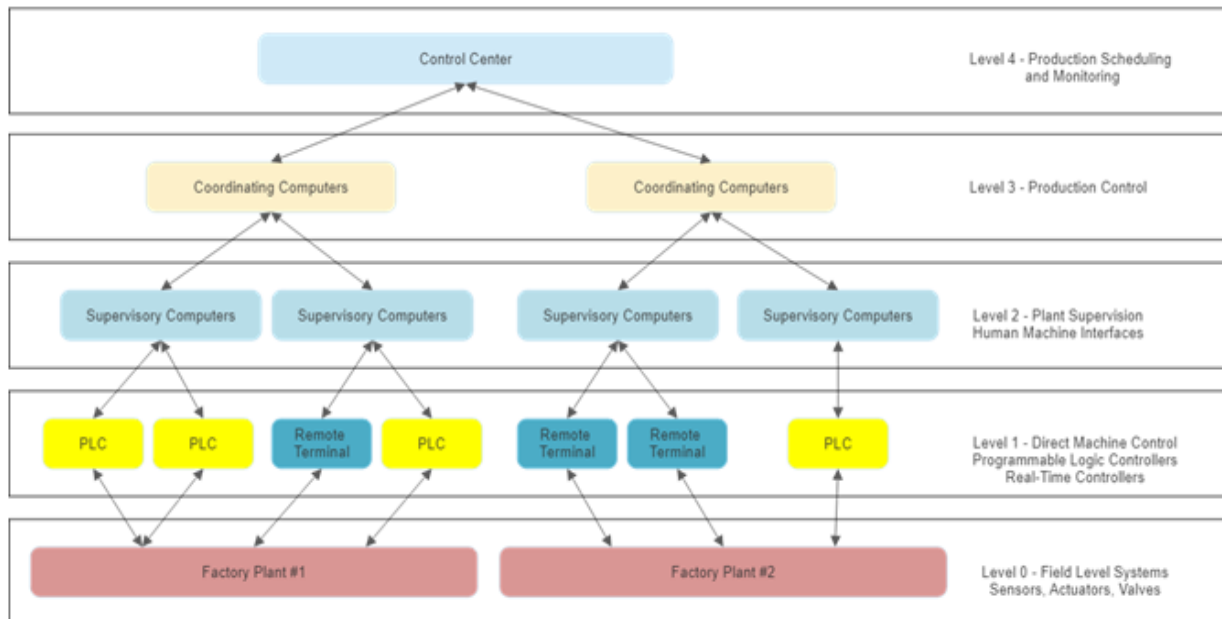
3.4.1 SCADA

A typical use of edge system is the monitoring and control of industrial equipment. Industrial machinery control systems have existed for quite some time. We previously talked about PLC controllers as an example of factory automation. The types of communication systems (IO) in manufacturing are also called **Supervisory Control and Data Acquisition** (or SCADA for short).

SCADA systems allows operators to issue commands and monitor the status of connected factory machines in real-time. Real-time implies that a system

or machine needs response or input in a guaranteed set period. If a system fails to deliver a response within that period failure or disaster can occur. This is especially true in factory automation that needs adjustments to machine speeds, power, or temperature in millisecond time periods. Since an edge machine can communicate directly with this local machinery it performs much better at this task than a cloud connected system. SCADA systems connect local sensors, actuators, and other computers together. Additionally, it may connect to remote terminals for monitoring, supervisory computers, and alarm systems. Figure 3.2 shows a SCADA hierarchy.

Figure 3.2 SCADA Hierarchy



The interfaces shown as arrows in the block diagram are the IO and communication pathways that require novel edge hardware. These interfaces include protocols like ModBus and Profibus which are not equivalent to the communication standards of consumer and business IT. Whereas a typical enterprise will easily adopt WiFi for wireless communication and Ethernet for wired networking, industrial automation is a unique example requiring significantly different hardware that is not based on TCP/IP protocols.

ModBus was first developed in 1979 by Schneider Electric as one of the first communication and bus protocols to connect machinery and PLCs. ModBus

is an open and free of license protocol that is widely adopted today and is managed by the Modbus Organization. It is a serial communication protocol meaning that data is delivered one bit at a time rather than in parallel. Modbus supports communication bidirectionally between multiple devices simultaneously connected to the same cable. For example, there can be a vibration sensor and temperature sensor connected to the same bus, each communicating measurements to the same edge computer. Additionally, it integrates seamlessly with TCP/IP to connect to the Internet.

An alternative protocol is called Profibus. While Modbus is an open standard, Profibus is proprietary and licensed through Profibus International. First designed in 1989 by BMBF of Germany and then licensed by Siemens of Germany, the protocol is still used today in factory automation. Profibus, like Modbus, is intended to connect factory sensors to controllers. The Field Message Specification (FMS) was the original protocol, but industry found it far too complex for simple connectivity to sensors. Today, the Profibus DP (or decentralized protocol) and Profibus PA (or process automation) is used. Profibus DP allow for several edge hosts to communicate using “tokens” to control who may talk at any one time. This is called a “token network”. Profibus PA is used to directly communicate with end sensors in a safety critical manner. Profibus PA supplies not only communication through a twisted pair of wires, but also supplies power to the sensor. Safety is critical in factory automation, thus Profibus PA was created. An edge computer does not communicate directly with an edge sensor, rather it uses bridge to isolate the communication, again for safety critical aspects. Think of Profibus DP as a faster communication with multiple edge computers monitoring a plant and Profibus PA used to connect to end point sensors in a safe manner.

There are several other protocols both open and proprietary in the industrial automation segment. Many are proprietary and developed by industrial manufacturing companies like Allen Bradley, Mitsubishi, Honeywell, and General Electric. There are also bridges that translate protocols from one standard to another. SCADA systems have been around for decades and control machinery that can be decades old. Some term this a “brownfield” industry since industrial machinery will amortize on a factory floor for many years. The take-away is that industrial automation requires specialty hardware communication interfaces built into edge computers to be a citizen on their network.

3.4.2 Networking Topologies

We have talked about industrial protocols such as ModBus in the previous section. Modbus is a wired protocol meaning that it utilizes a physical wire to transmit data. This was described as a serial protocol where one bit is transmitted at a time versus parallel communication which can transmit multiple bits of data simultaneously.

There are other communication systems worth mentioning. The most important, of course, is Ethernet. Nearly every piece of modern IT equipment utilizes Ethernet. Ethernet was developed in the 1970's at the legendary Xerox PARC research site. It became commercially available in 1980. Ethernet is based on a standard called the IEEE802.3^[2] protocol. Modern Ethernet transmits data serially or in parallel using multiple twisted pairs of wires or optical fibers. Transmit rates range from 1Mbit per second to extreme 400Gbit per second in data centers. Typically, the length of these cables is dependent on the speed of transmission but can extend hundreds of feet. Ethernet also allows for bridges and switches to expand the network. This allows for point-to-point networking between two computers but also a high fanout configuration that may connect thousands of machines.

Besides Ethernet, other forms of wired networking and communication also exist. Take for example the communication between various computers and sensors in a typical vehicle. A modern car has upwards of 20 to 100 different computers managing everything from power windows to the engine diagnostics. In this environment a wired protocol called CAN (Controller Area Network) was developed by Bosch in the late 1980s. It is a simple but robust design built for the transportation industry.

These types of networks utilize physical wiring or optical fibers for communication. Later in this book we will go deep into wireless networking which is foundational for many edge systems. For now, the types of wired networks fall into three categories:

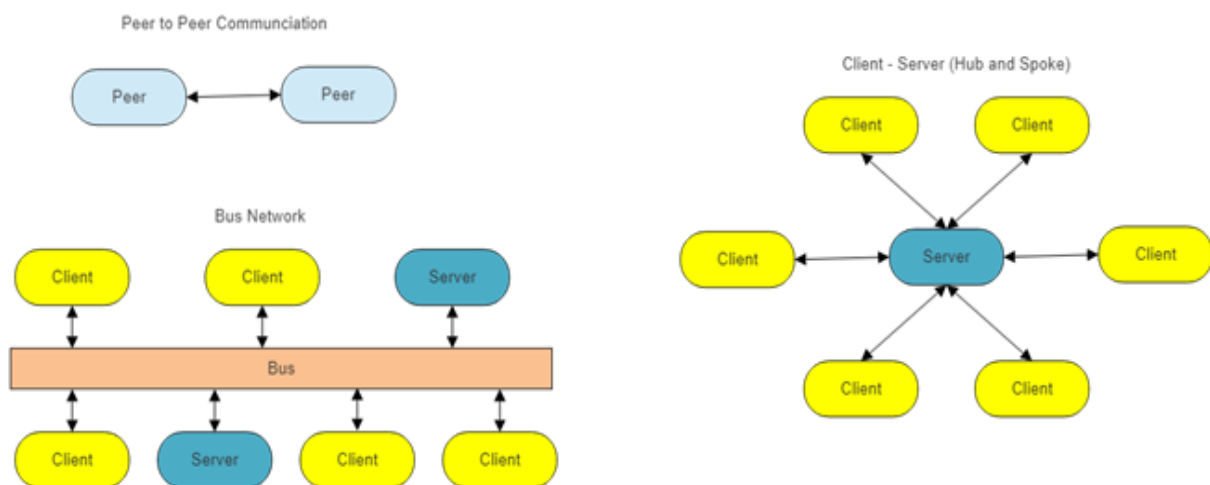
1. **Peer to Peer Network:** This is typically a dedicated communication path between two peer devices. This is especially relevant in serial communication protocols like RS232. This type of communication is

less of a network and more of a dedicated circuit. This networking is prevalent in a type of IOT device call Machine to Machine.

2. **Hub and Spoke Network:** This form of network relies on a central system (a server for illustration) to be the communication aggregator. No client communicates directly with another client, rather the central server acts as a proxy to route communication. This type of network is typical of certain wireless communication systems like Bluetooth and is analogous to a myriad of edge devices managed through a central cloud server.
3. **Bus Network:** This type of network architecture uses a common fabric (called a bus) as the central channel for all communication. Client computers, servers, and devices all share a common fabric for communication. In this model, the bus can be divided into separable “lanes” to allow for simultaneous communication. Alternatively, a token can be handed to each node on the bus to indicate when a device is clear to transmit data without interference. Or in the case of Ethernet, transmission can occur simultaneously and if two or more devices collide transmitting data, the algorithm randomly “pauses” a device until the channel clears.

Figure 3.3 contrasts these three types of networks:

Figure 3.3 Typical Network Configurations



While Ethernet is the most ubiquitous networking standard, other hard-wired systems exist. For example, the Reliagate 20-25 mentions CAN bus. CAN is another wired interface developed by Bosch in Germany in the 1980's and supported by the Society of Automotive Engineers. It is used heavily in automotive systems, agriculture machinery, elevators, smart buildings, lighting systems, and factory automation. It is the standard protocol in consumer vehicles that controls all the computers, safety systems, and infotainment systems in cars. It is the basis of the automotive On-Board Diagnostics protocol sometimes called (OBD)-II.

3.4.3 Personal Area Networks

Personal area networks (or PAN) are near range communication systems measure in inches or feet in reliable distances. PAN systems include such widely used wireless protocols such as Bluetooth, USB, and Thunderbolt. As such, PAN systems can be wired or wireless. What defines a PAN is short distance communication. Short being relative, but any communication under 100 feet fits within the PAN definition. While short range communication has its constraints, the benefit of PAN systems is that they are generally very low power. This is an important characteristic since edge devices and certainly IOT systems may be battery based.

A good example of a PAN network is Zigbee. Zigbee is a wireless communication system that is typically used for low cost and low power communication to sensors and home automation systems. Products like Phillips Hue smart lightbulbs operate using Zigbee. Zigbee is based on a network model called a mesh. Unlike the other forms of networks illustrated in figure 3.3, a mesh is a reconfigurable network where each node acts as its own router. We will talk more about PAN and mesh systems later in this book, for now what needs to be understood is that PAN networks are common in edge computing systems and require a separate radio transceiver than other forms of communication.

3.4.4 Local Area Networks

Local Area Networks (LANs) extends the range further than PAN systems. LAN systems are your typical ethernet and 802.11 Wi-Fi networking

common in consumer and enterprise IT equipment. It provides low cost and reliable communication with ranges up to 1 mile in certain conditions. It is hard to find a modern notebook, smartphone, or computer that does not support Wi-Fi today, these communication standards are ubiquitous and proven. Most edge systems will incorporate Wi-Fi and Ethernet transceivers in hardware for this reason.

3.4.5 Wide Area Networks

Wide Area Networking encompasses communication 100 feet or longer (miles and kilometers in many cases). These are typically wireless protocols like 4G-LTE and 5G (cellular). These networks typically consume more energy than PAN systems but have a much greater range. Edge systems may incorporate cellular radios as the primary backhaul to the Internet. Other times, cellular may be used as a failover system. Failover networking implies that the cellular radio is a backup communication if the primary network fails. In chapter 2 we briefly mentioned that the HP Edge system included an out-of-band networking feature. That would be an example of failover networking for highly reliable and resilient edge systems.

In this case the primary may be a subscribed DSL broadband service and a 5G radio as the backup. This is useful since cellular systems typically require a service level agreement (SLA). This refers to the cellular subscription plan. If the use case requires a substantial amount of data to be transmitted via WAN, the cellular service cost could grow significantly. In these cases, cellular is a backup system and the primary Internet communication will be broadband or some other low-cost transport. In other cases, a 5G cellular system may be the primary means of communication (e.g. a smart vehicle).

While cellular radios typically consume a large amount of power, WAN systems can also be exceptionally low power such as LoRa. LoRa (derived from long range), is a proprietary radio technology developed by the French company Cycleo and acquired by Semtech. LoRa has significant range (up to 10 miles or more) and uses extremely low power radios. Some even call this technology Low Power WAN (or LPWAN) because it consumes lower power than cellular and has the same range. This make LoRa suitable for edge applications like battery-based tracking systems, livestock monitoring

systems, municipal electric and gas meters, and city-wide smart parking meters. The downside of LoRa is that it does not support high speed communication. As matter of fact, while cellular can achieve speeds faster than one 1 gigabyte per second using 5G, LoRa is limited between 0.3kilobits per second to 50 kilobits per second. This means cellular is up to 20,000 times faster than LoRa, but comes at a cost of power, battery life, and SLA subscriptions.

When building an edge system, it is imperative to consider the communication systems, service charges, reliability, and power consumption. A useful way to categorize these diverse types of networks and radio systems is shown in the following table:

Table 3.2 Differences Between PAN, LAN, and WAN Systems

	PAN	LAN	WAN
Network Examples	Zigbee, Bluetooth, UWB (Ultra Wide Band)	802.11 WiFi	LoRa, 5G,NB-IOT, M1, 4GLTE, GSM. Starlink Satellite
Speed (Bandwidth)	<2 Megabits per second	>1Gigabit per second	10 to 1000 Megabits per second
Range	<100 feet	Up to 1 mile	10’s of miles
Power	Very Low	Medium	Low (LoRa) to High (5G)

Cost (implementation as well as data transfer costs)	Low	Medium	High
--	-----	--------	------

3.5 Summary

- Different segments of edge computing require different approaches to hardware design. There is no one platform to solve them all. This adds to the complexity of edge systems, but also enables us to bring compute closer to the machines and industries that have never been capable of using such information technology.
- We covered the basic operational description of all modern computing architecture. This includes the processor, storage, bus, and IO configurations. All computing devices at their most fundamental level can be described with this model.
- We provided a cursory exploration into two edge computer systems (HPE Edgeline EL800t and the Eurotech Reliagate 20-25). While the HPE system is more akin to a datacenter server and has nearly 1000 times more processing power, storage, and speed, the Eurotech is much better suited for industrial, low power, and vehicular edge computing due to its weight, IO flexibility and cost. Unlike the more generic PC and Smartphone industry, no two edge segments will necessarily have the same hardware.
- Communication hardware is another critical aspect that makes edge computing unique. Since the edge will communicate to brownfield industrial machinery to vehicles to sensors, using the correct IO is imperative.
- Networking and communication is the heart of edge computing. The three models of traditional network are: peer to peer, hub and spoke, and bus type networks. Each model has their purpose in edge computing from OBD vehicular networks to edge to cloud communication.
- Personal area networks (PAN) and wide area networks (WAN) based on wireless technologies are prevalent in a myriad of edge use cases from commercial Zigbee and Bluetooth communication in smart homes to

5G communication systems in remote edge computing. PAN systems typically are used for very close range and low-power communication with limited bandwidth while WAN systems typically have extended range and more capacity.

- Embedded systems are used heavily in edge computing. This is because various industries and use cases demand specialized hardware, form factors, and power systems to run in disparate and remote areas.

Since we now understand the nuances of edge hardware, we next will explore the litany of software that can run on edge computers.

[\[2\]](#) IEEE refers to the Institute of Electronics and Electrical Engineers professional society.

4 Software and system frameworks

This chapter covers

- What functions do edge systems provide. While edge devices serve diverse markets and use cases, there are some common elements within edge software that are typical and found on nearly every edge or IOT solution
- Developing an understanding of common edge frameworks and their high-level overview
- Digital Twin technology
- Definition and architecture of fog computing and mist computing
- An overview of centralized and decentralized computing for the edge

The previous chapter provided groundwork on edge hardware and cursory fundamentals of networking. We learned that edge systems are not ubiquitous and standard. Each edge application typically comes with bespoke hardware, IO, communication interfaces, and performance requirements. The same holds true for the software running on different edge machines. The architecture of edge systems also needs to consider the software environment running on the machine. This chapter supplies a high-level view of edge software. This includes device environments called frameworks.

Later chapters will explore specific edge applications such as machine learning and federated computing. This chapter is constructed to explore edge frameworks and the typical features all edge devices share in software.

4.1 Typical Edge Functions and Services

An edge machine provides computational power in remote or mobile environments. Edge devices in many cases are also extensions of the cloud. Developers will want to have services and features typically found in managed cloud environments to also be present on edge devices. Some services like security should be minimum ante for any edge devices, others

may not be necessary or required. Here we will detail some of the typical services found on edge devices.

4.1.1 Security and Hardening

We start by examining what should be considered the most important aspect of edge systems: security. Edge devices will not necessarily have the convenience of a security guard or a locked door protecting assets on a corporate campus. Furthermore, like most IT systems today, edge systems are connected via the Internet or some other medium. Edge systems also host as much software as servers running in a cloud. To an attacker, all these pathways to compromising a machine is called the *surface* area of an attack. Edge and IOT systems have been easy to compromise. The reason edge systems are attractive to compromise has some measure to do with the maturity of edge computing versus traditional PC and servers. Edge computing is in a relative infancy compared to the decades of maturity in PC software and hardware. Remember, often edge hardware is an embedded system, as we explained in earlier chapters. These systems are constrained by cost and size and do not have traditional PC level hardware and operating systems. While the maturity of hardware and software continues to improve, it is best to build security in as early as possible.

So what is at risk? Edge devices contain business logic and are essentially extensions of the public cloud. Confidential source code and data running on the device needs to be protected. Additionally, edge devices contain various credentials for the cloud. Finally, edge devices also have credentials to securely access and communicate with IOT device, which again, an attacker could use the IOT device as a threat. We will see such real examples later in this book.

Edge and IOT systems should address these fundamental security threats and attack surfaces:

- **Physical Attacks:** Edge systems may be in an environment in public venues, remote areas, and certainly in locations that are not physically secure. This provides the opportunity that the device may be tampered with. Hardening involves both hardware and software. The system needs to know if the system has been rebooted, powered off, opened,

disconnected from the network, or if devices have been installed (like USB thumb drives). While this book concentrates on electronic theft and countermeasures, we cannot exclude the fact that edge machines can be vulnerable to physical theft and vandalism. Edge system design does not stop at hardware and software, one must also consider physical attacks.

- **BotNets:** BotNets are compromised systems that have rogue software installed. This software acts as a proxy for a malicious actor to control the device for nefarious reasons. This also allows the attacker to use the device as a host to infect other edge systems. One of the most nefarious and dangerous cybersecurity attack was called the Marai virus in 2016. That attack spread to 600,000 edge systems including smart cameras and consumer wireless routers. The end effect was these “bots” launched a coordinated attack to various web sites, organizations, and crippled major portions of Internet traffic. We will explore this type of attack later in this book.
- **Theft:** While less widespread than other types of attacks, theft of electronic material stored or collected on edge machines is a risk. A great example is an attack on Home Depot where point of sales (POS) terminals were infected with malware. These POS terminals were fairly unsecure embedded systems making it vulnerable to classic types of hacker methods. After the malware was installed on the terminals, the software would gather customer credit card information. Over 56 million credit cards were stolen.
- **DDoS Threats:** DDoS stands for “Denial of Service Attack”. This is a type of attack that floods a server with internet traffic to prevent users from accessing connected online services and sites. Sometimes called a DNS attack (DNS is a domain naming system used for Internet navigation), this software works if many devices are simultaneously affected. Essentially, these attacks are analogous to trying to have a conversation with a friend in a stadium hosting a major league sporting event. With thousands of people yelling, it is difficult or impossible to have a conversation. The Marai virus describe above, besides being a botnet, also had a DDoS attack built in. That is how it disrupted a major slide of the Internet. BotNets typically are used to launch DDoS attacks.
- **Ransomware:** Ransomware is a prevalent problem in IT. It is a type of attack where the malicious actor installs software that encrypts valuable data on the device. At that point the key to decrypt it is held by the

attacker. He or she then informs the owner of the device of the attack and attempts to extort money (typically in crypto coins). If the legitimate owner of the system refuses to pay the ransom, the information may stay encrypted and useless forever or be deleted.

The services defining security for edge machines include features like encryption and decryption of all data incoming through communication services as well as **encrypting** of all customer and sensitive data stored on the machine. The device should also include services to securely boot the machine and determine if all the running software is trusted. The system should include **authentication** services to determine if the privilege of various user accounts. Edge machines also provide **attestation** services. These are software services to ensure that other machines or the overlying cloud can safely identify and ensure that a particular edge system is trusted and is who it's saying it is.

Many edge frameworks provide security services and software to address these issues which we will explore later in this chapter.

4.1.2 Remote Management and Monitoring:

Edge systems do not have the convenience of being in a nearby datacenter or server room. They could be located anywhere on this planet (and perhaps others). Therefore, the system must be able to be remotely managed, controlled, and serviced. These devices should appear as an extension of familiar IT tools and infrastructure. Additionally, the health of a remote computer should also be considered. Things go wrong with hardware and if the edge computer is in a remote oil field in Alberta and it should be able to be managed as if it were in a corporate campus in Seattle.

What does edge management entail?

- **Logging:** Capturing information of the health of the edge hardware and software is critical to diagnosing failures and monitoring for security breaches. Logging information includes such things as access and usage identities, processor performance and thermal characteristics, power outages and reboots, network and communication downtime, and health of systems managed by the edge computer. Edge systems should

capture local logs of events and errors. Additionally, edge systems should have runtime filters to determine when to send an alert that a critical issue has been discovered.

- **Account Management:** A modern computing environment will allow for multiple systems accounts with various privileges. New accounts on the edge device may need to be created. For example, an edge system we have developed for factory automation required each operator to have an account on the edge machine. In this case, the edge device provided the interface to login, but the accounts were verified through an Identity and Access Management (IAM) service in the framework.
- **System Settings and Control:** A modern edge system and operating system will have thousands of different settings. Certain qualified accounts should be set up to control the edge box. Settings and controls also imply setting quotas, quality of service levels (QoS), and device limitations. For example, many edge systems may want to disable hardware, ports, and types of unused IO for security reasons. Other times, different users will have different experiences such as faster or slower networking abilities especially in situations where the edge device is network bandwidth restricted like in LoRaWAN systems we will explore later.
- **Remote Updates:** It is imperative to have a secure, robust, and reliable method to patch and update software in the field on remote devices. A remote edge device will need software updates throughout its lifetime to repair bugs, add features, and most importantly, update security holes. Since edge devices are typically remote, updates must be managed by the cloud and deployed in a controlled manner. If there are millions of edge devices needing an update, one could saturate the cloud by updating all devices simultaneously. Therefore, rolling the update by region or by some other controlled manner is needed. You may also want to roll updates globally during off-hours or when the systems are not in use. Updates also must be robust. If something fails during the software patch, the edge system should automatically roll back to the last known good and verified version to keep the system running and update the logs indicating the update had failed.

Management of edge systems should be enabled remotely and securely. It should not be expected to be in physical proximity to control any of these features.

4.1.3 Interconnectivity and Networking:

What separates edge computing from unconnected computers is networking, particularly Internet-based communication. This provides the ability for an edge device to pass data and control messages to the cloud or other devices. Standard models of communication are used in edge networking like TCP/IP stacks. Additionally, the industry has adopted other standards like MQTT and CoAP protocols, which are very efficient and robust for edge and IOT communication.

The edge device may also act as a gateway between the internet and near range communication protocols to wireless devices and sensors. For example, Bluetooth, Zigbee, and mesh protocols (covered in the next chapter) allow the edge device to host many wireless devices in a hub-and-spoke type model. Networking also includes several layers of security services, encryption, and authentication. The networking and communication layers of system software are complex and large. They are also standards based and it makes sense to reuse well tested and proven software stacks rather than design them from scratch.

4.1.4 Software Provisioning and Upgradability

If there is one message that should resonate with the reader of this book or other IT-centric books: **ensure you can patch and upgrade software remotely!** Software is malleable and there will be bug fixes and security patches that must be rolled out to field edge computers. To do this in the most robust fashion system software must manage, validate, authenticate, provision, and install patches. Software may be installed immediately through manual deployment or automatically scheduled during maintenance downtimes. The system should also be able to recover in the event of an error during software installation. The trending modern method of pushing software to remote devices is through software container management. We will visit containers later in this book; however, containers logically wrap all the surrounding software and libraries that typically comprise a large software system into a neat package that is easily deployable and decoupled from the environment (OS) where the software runs.

4.1.5 Reliability and Robustness

Since these systems are remote and often the case managing critical infrastructure, the systems need to be reliable, robust, and fool proof. We will see that many failsafe and fallback solutions are a must. Edge computing systems often employ redundant components in the event of critical failure. For example, software must be able to determine if the main wide area network providing internet communication is lost and be able to divert data to a redundant system. An example of this is dynamically detecting a wired network is down and to move all communication to a cellular fallback network. Then detect when the main network is back up to divert data back to the lower cost WAN.

Out-of-band (OOB) communication should also be considered for managing an edge device outside. OOB relies on a separate network interface outside of the main Internet access point and is primarily used for reliable IT management and servicing even if the main network connection fails. This adds a level of redundancy to access an edge system. This is also called *fault-tolerant* or *failsafe*. Essentially, if all else fails, this channel should be accessible to control a remote edge machine. The Eurotech device discussed in Chapter 3 has OOB communication ability using wired serial interfaces or even LTE cellular communication. If the main network fails, the system can automatically switch to a less efficient backup communication system. It will keep the device running, and an operator can still remotely debug the machine.

4.1.6 Operating System

Most edge systems will use a modern operating system such as a Linux (e.g., Yocto Linux, OpenWRT, Ubuntu/Ubuntu Core, RaspberryPi (Debian), RedHat Linux, or even Google Android). Others may use a Microsoft Windows commercial OS like Windows IOT. The reason to use one OS variant over the other depends again on the use case. If data is migrating to Microsoft Azure, using Windows IOT with extensions to manage the edge device and marshal data to and from Azure servers is an easy choice. In other cases, a Linux derivative offers more flexibility in design choices and can accommodate nuanced hardware.

Another choice point to consider is if the edge system must respond to real-time events. A real-time event is one that must be handled within a strict time limit, or some other failure may occur. For example, if the edge system is managing some low-priority task, it may need to be preempted to manage a high-priority critical task. Not all operating systems handle real-time, and the designer may need to elect for a true real-time OS like FreeRTOS from Amazon, LynxOS from Lynx Software, or ThreadX from Microsoft Xpress Logic.

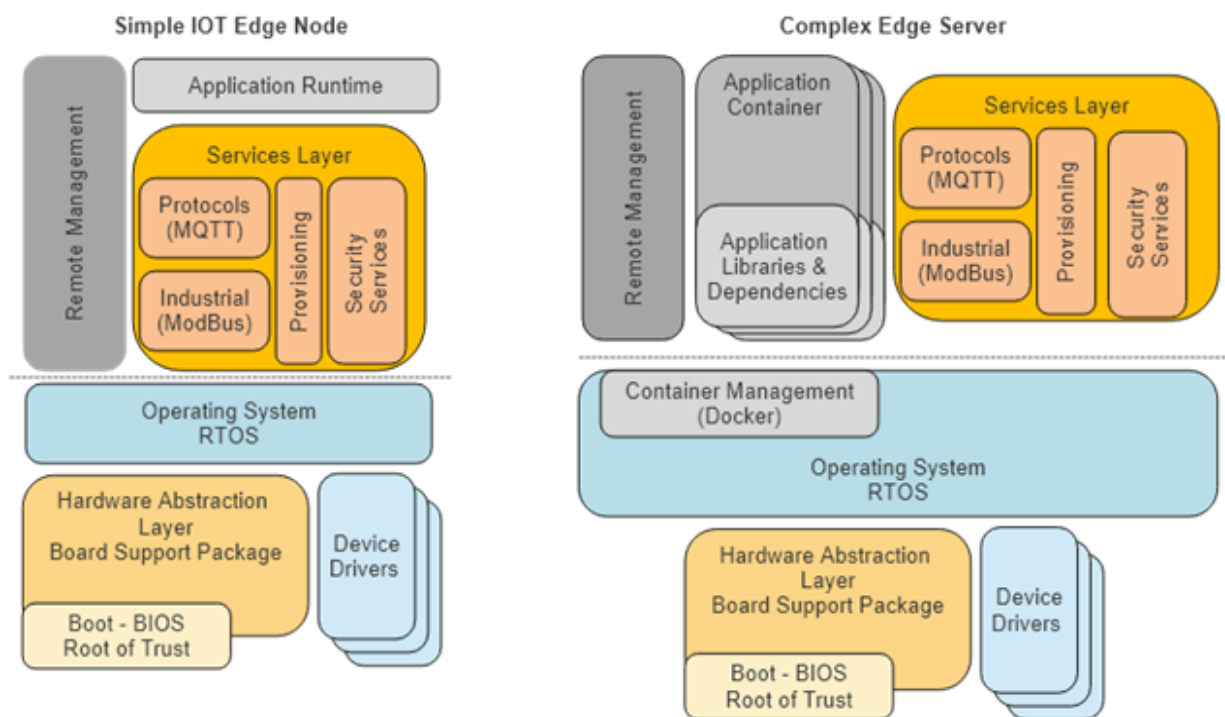
So, what is a real-time event versus a non-real-time event? A use case of real-time processing is the control system for autonomous driving. It can be argued that a self-driving vehicle is an edge system. It is constructed of extremely powerful vehicular computers, graphics processors, cameras, and image processing algorithms along with extremely sophisticated trained machine learning algorithms capable of “seeing” and responding to driving events as they occur. An autonomous vehicle also communicates with the cloud for navigation assistance, weather information, traffic status, and reporting vehicle health. What separates the vehicular use case from the cloud portion is real-time processing. In one second a vehicle moving 60 miles per hour will travel 88 feet in one second. Within 1 second, the computers onboard must take multiple images, LiDAR samples, and radar snapshots of the entire surrounding area. This includes lanes, oncoming and nearby vehicle information, pedestrian-predicted paths, road signage, speed limit signs, and also understanding the instantaneous motion and direction of the vehicle. This is an enormous amount of processing let alone an enormous amount of data collected each second. The edge computers here must make life-critical decisions hundreds of times each second to avoid disaster. This would not be possible if all that data must traverse to the cloud, be processed in the cloud, and then send vehicular driving controls back.

If the edge system is more of a remote server for content delivery, it probably will not run a lightweight RTOS or operating system targeting an IOT device. It will most likely require a true virtualized environment and hypervisor typical of hyperscale datacenters.

4.2 Software Architecture

When we consider the entire software architecture, we like to draw out the applications and services in a stack diagram. We start at the bottom layer closest to the hardware and move up to the exposed services and applications that comprise the system. As we move down the stack closer to the hardware software becomes more restrictive and privileged for security and protection. This layering approach is convenient for abstraction as well since modern software isn't necessarily constructed as a monolithic block of code. A typical hierarchy of software and services can take different form depending on the use case, but there are commonalities (figure 4.1).

Figure 4.1 Simple and Complex Software Stacks. Edge Systems can be as simple as an single function embedded computer to a complex server hosting multiple users and applications.



Software architecture should start with security as a focus and the system boot should maintain a root-of-trust. This is a hardened and immutable hardware/software block that guarantees the system integrity from power on to a full running system. The hardware abstraction layer (HAL) and device drivers are also architecturally common between low-end and high-end edge systems. This area of software provides the interfacing and abstraction from upper layer applications and the hardware. The operating system, which we

have already talked about, resides above the HAL and provides all services for real-time control, software security, memory management, and storage systems. Above the dotted line is the separation between application code and system code. System code generally needs more protection for software security and protection of hardware. While an application can crash for a variety of reasons, a single application shouldn't bring down the entire system. The OS can't crash without affecting all other services and software running upon it.

Application space can vary between simple and complex edge use cases. In a simple edge node such as an IOT edge gateway like a Samsung SmartThings Home Hub, the application may be fixed and more of an embedded system used to manage home appliances and IOT devices like smart-doorbells, smart-locks, and water leak sensors around a home. It will have a set of services like MQTT for edge to cloud communication as well as provisioning and security services to ensure the device is tamper resistant and field upgradable (MQTT and edge protocols will be covered later in this book).

A more complex example would be a device that requires many disparate applications to run simultaneously and provide utilities such as machine learning and predictive analytics on real-time data, a local user-interface, device manifesting and registry, and local data management. This architecture lends itself well to multi-user environment such as the edge servers used by CDN providers like Netflix that may have dozens of users all streaming from a single server. Container-based design can also be used, which requires some additional overhead for a container management service to run, like Docker or Moby. This architecture will have more software demands and require more aggressive and powerful hardware to support it. IF your edge system is a very small purpose-built embedded system, it may not have all the processing and storage resources for containers. Think of it this way, a container encapsulates your application, including all dependencies on other software and operating systems, into a tight package that can be deployed easily. This alleviates a lot of the issues of ensuring the software works on the systems you deployed. If there is a myriad of different hardware you are dealing with, a container-based system is the right choice.

4.3 Frameworks

If you want to build an edge machine and service, you don't have to design all the software yourself. Just like operating systems provide a foundation for other software on PCs and servers, frameworks provide a myriad of features and services to get edge system running quickly. There are many "frameworks" available from organizations like the Linux Foundation and companies like Microsoft and Amazon to get you started. The framework provides the operating system, the remote management, security, and cloud communication we talked about earlier in this chapter.

4.3.1 EdgeX Foundry

A useful framework for edge machines is EdgeX Foundry (or just EdgeX). This is an open-source community support edge platform supported by the non-profit Linux Foundation. One can download and start using EdgeX Foundry simply by downloading the framework from: <https://www.edgexfoundry.org>. The intent of EdgeX is to provide a vendor and hardware agnostic architecture to rapidly develop edge and IOT software upon. The first release of the EdgeX framework was in October of 2017 and supported by organizations such as ARM, IBM, ATT, Dell, Qualcomm, Red Hat, HP, and Samsung. The purpose of EdgeX can be summarized as:

- **Ensure Security:** It is understood security is the foundation for all IT. EdgeX Foundry provides typical services, firewalls, and encryption features built in. Edge X also allows designers to selectively determine what data is processed locally and what propagates through the Internet to the cloud (this filtering also helps with reducing transmission and cloud costs).
- **Ease of Connectivity:** EdgeX provides a litany of network and communication protocols widely used by connected devices. Rather than invest in designing such protocols like ModBus or integrating other third-party libraries, EdgeX provides out-of-the-box connectivity.
- **Interoperability:** An architecture that support a plug-and-play philosophy with a variety of hardware, processors, and software.

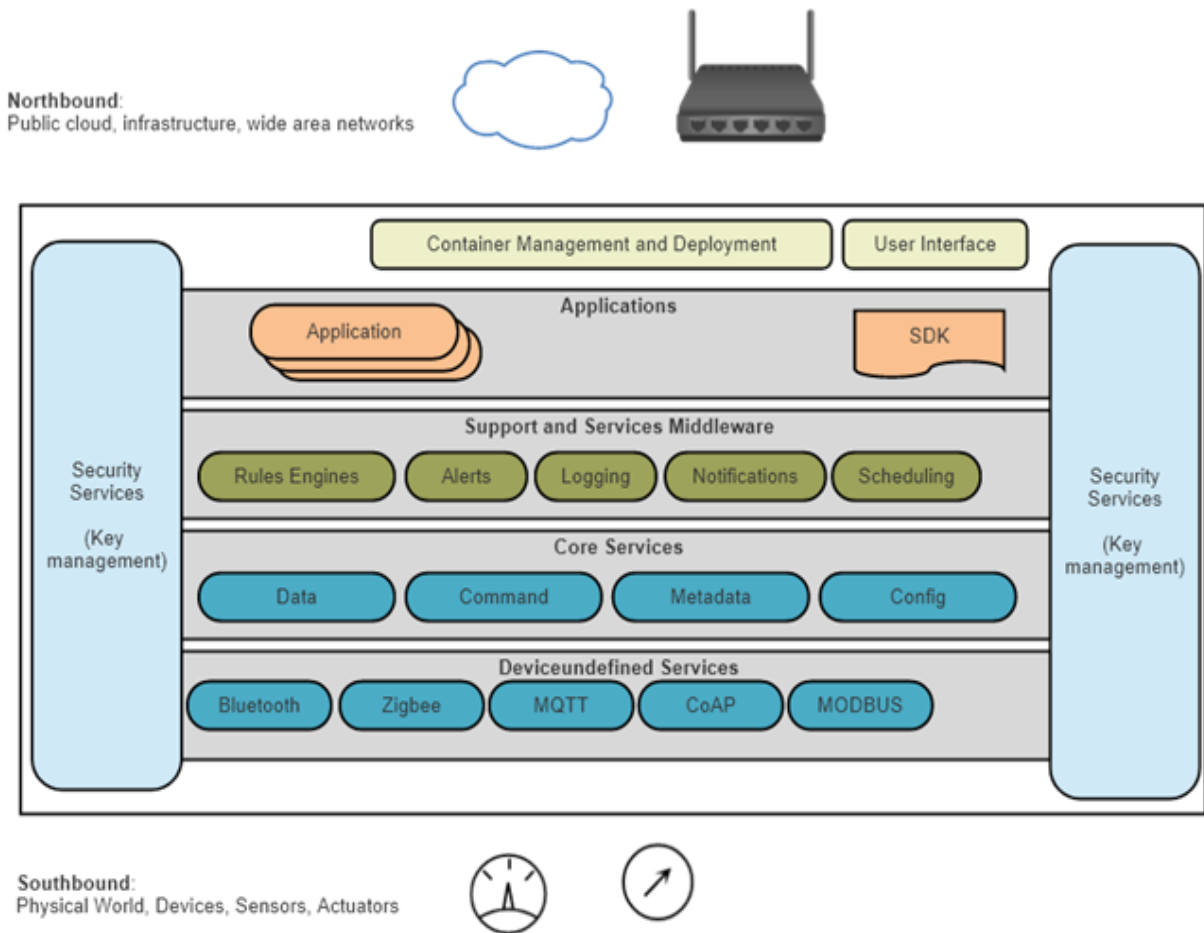
- **Open platform:** Opensource software under Apache license terms. Community developed and community supported.
- **Low Latency Operation:** To satisfy real-time applications that must process data at millisecond intervals, Edge X is designed around real-time constraints.
- **Positive business value:** Accelerate time-to-market by allow users to add and develop their business logic on a standard platform.
- **Reduce risk:** EdgeX components will be certified through the Linux Foundation. EdgeX also is used extensively in IOT and edge applications both commercial and in research.
- **Robustness:** Provides services to operate an edge device when connectivity is interrupted. Even if all networking is lost, the edge device using EdgeX Foundry will continue to operate, gather data, and re-establish communication at the next opportunity.

As you see, many of the requirements that we used to define the purpose of edge computing are captured in this framework: latency, networking and bridging, security, resiliency and robustness, and cost control.

In this architecture and many others, we will refer to data directions in compass terms. For example, “Northbound” and southbound data refer to data movement from a device to the next level in the hierarchy of devices. An edge device might connect to “Southbound” systems like IoT devices and sensors, process or store data locally, and then send pertinent data northbound to the public cloud. “East-West” directionality refers to devices in the same hierarchical tier but data communicates to peer systems. For example, one edge system talking to another for failover/resiliency reasons or perhaps as a model for federated machine learning (which we will explore in later chapters).

The EdgeX architecture is built upon microservices. The term *microservices* simply means the architecture is composed of a collection of loosely coupled services. Rather than create a large monolithic software package, the system is broken down into manageable components that can communicate with each other by light weight software interfaces. These services are also easy to deploy and developer friendly. A new service can be installed while a system is running. The diagram in figure 4.2 shows the architectural hierarchy:

Figure 4.2 EdgeX Foundry Framework Hierarchy. The bottom of the diagram depicts the physical sensors and instruments used to connect the analog world to the digital.



4.3.2 Microsoft Azure IOT Edge

While EdgeX Foundry is an open-source solution to provide a great starting framework to enable an edge device, there are commercial frameworks we should also consider and contrast. Microsoft provides a framework called Azure IoT Edge. The framework has many similar features as EdgeX Foundry, but with notable differences.

IoT Edge is composed of a runtime system that manages the edge machine and the rest of the framework. The other components are IoT Edge modules which are lightweight containers that run numerous services and is also where your software would be installed. The final component of the

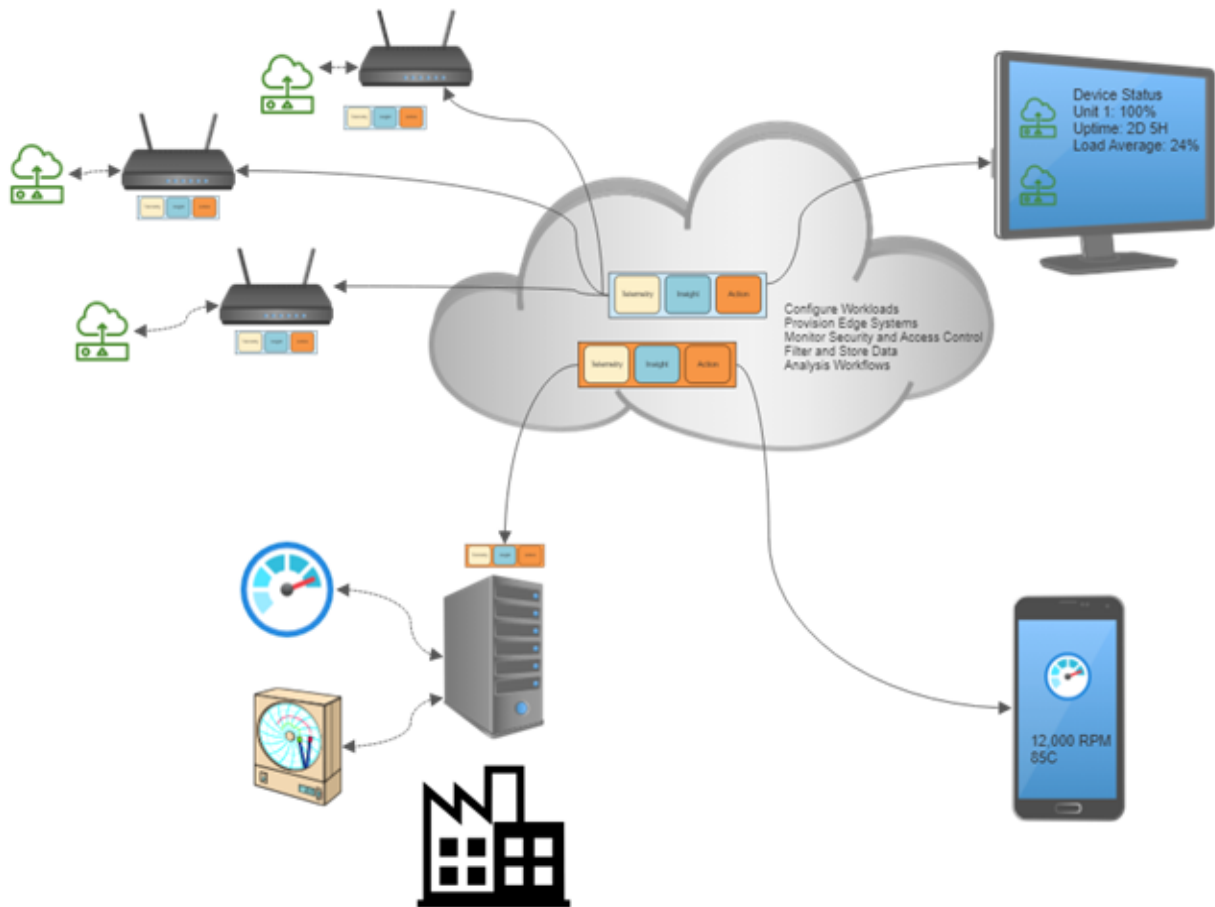
framework is the cloud interface which allows you to remotely monitor and control any edge device through Azure's public cloud portal.

Like EdgeX, modules are built on container management services (e.g., Moby). Several independent modules can be deployed at any time. Microsoft also provides modules to aid with machine learning and artificial intelligence on the edge device. We will go into detail of such AI features later in this book, but for now, these AI modules help with complex event processing, image recognition, audio and speech recognition, and time-series analysis. Microsoft Azure has historically supplied a rich set of services for data analysis (Azure Stream Analytics), these services can also be deployed as modules on the edge device to process and filter data as close to the source as possible.

Azure IOT Edge is not an operating system, rather it sits upon various edge operating systems including Linux. Therefore, any machine capable of running Linux and Docker can run the Azure IOT Edge framework. It is also programming language agnostic. You are free to design edge and cloud software using common programming languages like C, Javascript (and Node.js frameworks), Python and so on.

The framework allows for millions of edge devices to be manageable, secure, and reliable. The cloud connectivity allows for module containers to run both on edge nodes where it makes sense and within Azure for heavier computational processing. As mentioned, all software is deployed as containers and delivered to edge nodes in a controlled manner.

Figure 4.3 Azure IoT Hub conceptual diagram. Two different Azure IOT Hub instances manage a remote wireless IoT system and the other manages factory automation. The runtime provides services such as telemetry and communication services, insight for data analytics, and actions such as alerts and notifications.



The runtime module performs the heavy lifting for management and communications of the edge device. It is responsible for installing and managing software updates, enforcing all the security standards, and monitors the health of the edge machine and connected devices.

There are many other frameworks you should consider. A good source of various frameworks is located here: <https://github.com/qijianpeng/awesome-edge-computing#frameworks>.

4.4 Digital Twins

Azure IOT Hub is a fitting example of a cloud to edge framework. One enabling feature of Azure IOT Hub is the ability to have edge and server modules to create a synthetic clone of the physical device being monitored. This is called a *digital twin*. First termed in 2002 by academics at the University of Michigan, a digital twin is a virtual representation or

simulation of a real physical system. For example, a factory automation system may have several sensors and monitors gathering data from the machines on the factory floor. This data is relayed to the edge node and later delivered to the cloud. Within the cloud is a model or simulation of the factory machinery (sometimes called an avatar).

Digital twins have various categories:

- **Component Twins:** This is a model of a particular component of a device such as a drive train in automation or maybe the current speed of a fleet of delivery vehicles.
- **Asset Twins:** This involves the modelling of two or more component twins. This allows you to study the behavior and interaction between objects. For example, if you monitor the speed of a vehicle and the engine RPM, you can derive correlations between performance and engine efficiency.
- **System Twins:** This is the next higher level of digital twins. Here we take different asset twins and combine them into larger systems.
- **Process Twins:** This is the final “all-encompassing” digital twin level. It may examine an entire manufacturing line and determine the efficiency of a whole factory.

Digital twins have been a buzzword for some time in Industrial IOT and what is sometimes called the Industry 4.0 (short for Fourth Industrial Revolution). Having digital twins supplies substantial benefits to various industries besides manufacturing. Power and energy sectors, railways, shipping, logistics, smart-city projects, construction, and engineering all have benefited from using digital twins.

We should talk more about actual use cases of digital twins and how edge systems enable them. Let's examine a case study in energy production. In this example, a large European energy producer contracted with the computing consultancy *KPMG* with the problem of optimizing the output of an installation of 200 wind turbines. This required real-time feedback on the efficiency of each turbine and the angles of the blades to optimize energy production. This is a delicate process. A wind turbine can and has been catastrophically damaged in high winds, blade angles, and the direction of the turbine to the wind direction. Turbines are also directly affected by the

weather as well as fluctuating energy pricing. Therefore, this energy customer needed a method to combine real-time data from each turbine with weather models and forecasts originating in the cloud. Machine learning models were created in the cloud to predict optimal blade direction based on weather and wind forecasts. The models allow the system to simulate and optimize each turbine's production efficiency while ensuring the turbine would not be damaged in the digital model using weather data, positions of each turbine, commodity energy prices, and angles of each turbine blade. The model also used physical equations to predict if damage was to occur if the blades were spinning beyond manufacturing parameters. When exhaustive cloud-based testing was completed, the AI model and the control systems were allowed to manage the real physical wind farm. Doing this without a digital twin could have resulted in possible damage to one or many of the wind turbines at a cost of \$2.6 to \$4M US dollars per turbine. It was far better and less costly to model this using computational models, physical equations, and machine learning than to risk damage to physical property. In the end, the efficiency improved each turbine's efficiency by approximately 3%. This resulted in the 200 turbines producing an extra 32,000 megawatts of energy and increasing revenue by 2.4M US dollars.

The benefit of digital twin technology is that it allows for better R&D through real-time insights as well as continuous improvements to existing products. Digital twins also allow for improved data visualization of a larger complex system. Functions like predictive maintenance become a reality with digital twins. A system can monitor and predict when machines or systems are going to fail or stop. Additionally, data can be accessed from anywhere allowing for world-wide team collaboration of remote physical systems. This all leads to better efficiency of business and better financial planning.

4.5 The Fog and The Mist

Edge computing satisfies the problem of moving connected intelligence close to users and the sources of data generation (e.g., sensors). In all the use cases we have examined, the separation between the cloud and the edge is well establish and defined. An edge computer has unique workloads that separate it from the cloud. The cloud, as we have studied, performs macro-

operations such as ingesting data, training machine learning algorithms, and provisioning a vast array of edge nodes. Software that exists on the cloud must take orthogonal routes to communicate and perform an action on the edge. There are strict APIs and communication protocols like MQTT that perform those services.

However, what happens when we simply want edge nodes to be transparent to the cloud? Could a software engineer design a service in the public cloud simply push that code to the edge? Could we have software that is abstract and nebulous and where it executes is hidden from the user? The advent of *fog computing* completes the edge transition.

Fog computing is defined as edge nodes that share a common framework, API, and communications protocol with the public cloud. [\[3\]](#) Since the fog is an extension of the cloud, each fog node can be an extension of each other. Essentially work can transpire in the cloud or be pushed down to the edge. It is important to understand that the relationship of the cloud to the fog is considered to be the “north-south” direction. Other fog nodes are considered to be “east-west” in direction from each other. Fog nodes can communicate with sibling fog nodes in a similar fashion as communicating with the cloud. Being able to address other fog nodes allows for load balancing between nodes.

Another concept related to fog computing is called “the mist” or “cloudlets”. *Mist computing* refers edge computers at the very far end of the network topology and as close as possible to the source of data (such as a sensor). These devices have significantly less computing power and are embedded systems. They do share some constructs with fog nodes but have significantly fewer resources that can be used.

There have been initiatives to develop a common framework for fog computing. In 2015, six industry leaders and academics from Microsoft, Dell, Cisco, Intel, ARM, and Princeton University form the OpenFog Consortium. This group grew to 57 members and a reference architecture was drafted. Unfortunately, this effort hasn’t had much traction in the industry and in January of 2019, the work was merged into the larger Industrial Internet Consortium. That said, there are commercial frameworks

that can be purchased. Most are provided by hyperscale cloud companies and provide some of the features inclusive of fog computing.

4.6 System Architecture – One Architecture to Rule Them All, Probably Not.

After describing the rules and requirements for edge computers, we now must design a complete architecture as a system. As we saw in chapter two, many use cases have multiple edge nodes and a central controller most likely situated in the public cloud. This is called the hub-and-spoke model of architecture.

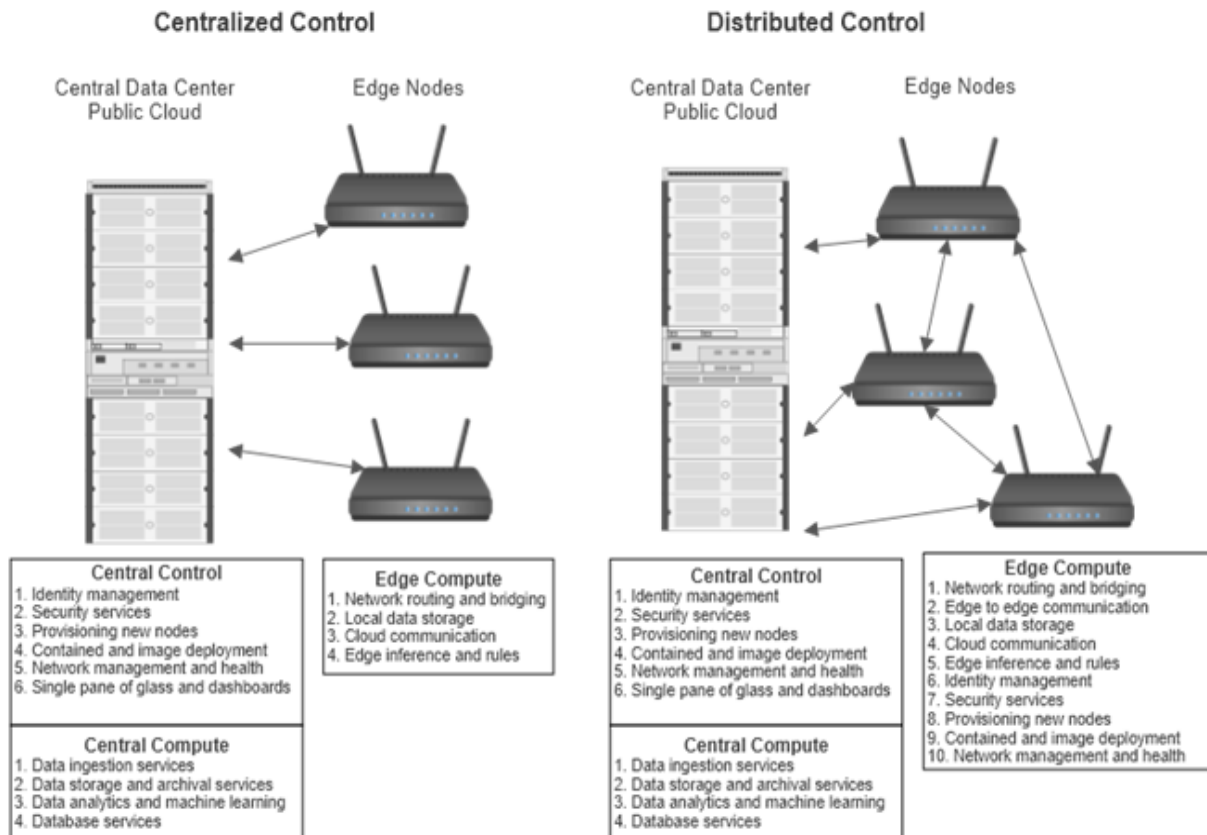
Typically, an edge machine separates the data this communicates with the cloud into two pieces: a control plane and a data plane. The control plane manages things described here such as provisioning software to edge nodes, identity management, and so on. The data plane represents the raw data generated, processed, or stored that is particular to the use case. The Netflix content delivery network may manage a huge multi-petabyte amount of data in the form of movies and television shows streaming around the globe, while an industrial automation system may only manage kilobytes of data generated daily by some factory machinery.

The architect of an edge system has two choices in building the control plane. Either the system is centrally managed or a distributed control plane. In a centrally managed system, all services managing and controlling edge compute nodes reside in a public cloud or datacenter. Each edge node has a heavy reliance on the central cloud or datacenter to manage itself. If the node loses communication, there is a potential that the device may become non-functional. In an extreme case, not only are control services centrally managed, but all compute functions could also be performed in the cloud with the edge simply serving to move data from the edge source to the cloud. Typically, an architecture is not robust if all data is sent blindly to the cloud without some degree of preprocessing or filtering.

A centrally managed architecture is a good choice for systems that may not need to be completely resilient to network outages. Alternatively, if an enterprise is scaling from a traditional datacenter central model to now

include edge nodes, this architecture may be the best choice for rapid deployment (figure 4.3).

Figure 4.4 Difference in centralized and decentralized edge computing. Note how decentralized computing typically has more interdependencies and communications between edge nodes.



Unlike a centrally managed system, distributed architectures place more demands on edge nodes to be self-reliant and robust in the event of failure. The edge subsumes many of the roles the central service provided. The datacenter is still a central management controller responsible for overall network and edge health. The edge nodes must now orchestrate and sync with the central servers to ensure the entire edge system has consistent policies and software updates. This architecture is much more resilient to failure as the edge nodes must be self-sufficient in the event of communication loss. This architecture also allows edges nodes to communicate and service each other. For example, if an edge node loses

communication with a datacenter, it may shuttle data to a neighbor edge node that still maintains a healthy connection to the cloud.

Managing a distributed architecture does require more software and services to be designed to work autonomously on the edge compute nodes.

Orchestrating and keeping the edge nodes in sync requires design discipline as well. The central server can operate as a federated data model. In this design, each node manages its systems independently, but the collection of all edge nodes appears as a single uniform database of nodes and metadata. All this occurs through network communication. The alternative is that each node sync with the central server (and potentially other edge nodes) on a periodic basis attempting to keep all data and control in sync.

4.7 Summary

- The software stacks of edge machines can be as complicated or more complicated than many PC and standard IT systems. This is further complicated by the fact that these machines are often used for life-critical applications and must be robust, fool proof and secure all while operating 24/7 remotely.
- To help with the large software lift, one should consider common frameworks that provide a litany of the services and features for edge machine.
- The most important aspect of software services that should be designed and architected first is security services. This includes all software to encrypt data in flight and resting on the device, authenticating accounts and users, and attesting the device is trusted.
- Other common features of edge machines include:
 - Remote management and control to allow for automated alerting and system control of various settings.
 - Software provisioning and upgrades which is critical to ensure the device operates securely and reliably.
 - Networking services which include complex communication software stacks.
 - Reliability services which handle failures automatically and ensure the uptime of systems. These services also include out-of-band communication as a failsafe method to control remote machines.

- Operating Systems which can be very simple and power efficient real-time OS or a full-blown Linux server.
- Edge software architectures are typically hierarchical and modular.
- Several edge specific frameworks are available in public domain or licensed. These supply much of the heavy lifting of all the typical services of edge devices.
- Frameworks with edge to cloud connectivity allow for digital twins to be used to analyze systems worldwide in real-time. This not only provides insights to the health of a system, but also enables predictive analytics to determine the state of the system in the future.
- Fog systems are essentially extensions of the cloud. Many of the frameworks are provided by public cloud companies and fog frameworks allow software on edge devices to be developed harmoniously with cloud software.
- Centralized control architectures are a hub and spoke design with a public cloud used to manage edge devices.
- Distributed control architectures pass more responsibility on edge nodes to manage themselves. This may be useful in high-uptime and areas requiring high-reliability.

[3] <https://www.ibm.com/blogs/cloud-computing/2014/08/25/fog-computing/>

5 Connecting things (networking and communications)

This chapter covers

- The prevalence of wireless and radio-based networking for IoT and Edge systems
- Typical use cases of connected devices.
- Exploring different types of near range communication including Bluetooth and Zigbee.
- Understanding how NFC and RFID Personal Area Networks are used in industrial, financial and commercial applications.
- How long-range communication systems including 5G cellular, satellite, and LoRa provide the connection to the cloud.

If you feel like you are learning more about how the edge connects to things, you are correct. IoT and edge systems are now able to connect previously unconnected and unintelligent devices, in part, because the cost for data networking and communication have dramatically decreased over the last thirty years. Take consumer broadband as an example. In 1998, the average cost of a megabit per second service was close to \$1000 per month using lease lines and old telco ISDN technology. Today, the cost per megabit of service is less than \$1 per month (I currently pay \$120/month for gigabit service for my home). That is a 35% year-over-year decrease in cost all while increasing the available bandwidth 35% as well. Today we have a myriad of choices of connecting nearly any spot on the Earth (or in orbit), enabling the IOT and edge computing. This ease of connectivity is something to which we are accustomed. Your smartphone works seamlessly in your home using Wi-Fi or while riding a commuter bus using cellular communication. But how well does it work somewhere such as the middle of Yellowstone Park (where there is no cellular coverage) or ten miles out to sea? Not very well at all, and that's a problem that edge can solve. Without considering networking and communications there would be no edge computing. Radio based communication is an integral part of the definition

of edge. This is the power for edge and IoT computing à radio communication now gives us the ability to connect what had previously been unconnectable.

5.1 A Typical Edge System in Your Home

Take a common appliance many people use: a smart doorbell. Smart doorbells are connected devices that alert the homeowner to movement and record video for security reasons. This simple doorbell includes significant computing abilities within the doorbell but relies on services in the cloud to store videos, analyze data, and alert the homeowner of issues.

When you unbox a smart doorbell, the first thing you must do is provision it. Provisioning is a common theme for edge and IOT devices where the new owner configures and personalizes the Ring doorbell and securely grants it access to our home network. But there are no buttons and no display on the doorbell. This is what we call a “headless” machine. To configure the device, we need to first attach it to our Smartphone via an app. Once the app is installed and running, it attempts to connect to the doorbell through Bluetooth. Bluetooth is a low power short range communication system that allows you to communicate and provision the doorbell through your Smartphone (it is the keyboard and display for the doorbell). Once configured, the doorbell will reboot. To reach the cloud, a Ring doorbell must have access to a homeowner’s Wi-Fi, so if the customer has no Wi-Fi access point, then a smart doorbell is not an option for them. This is a typical edge system: edge device à connectivity à cloud. Usually installing a Ring doorbell is seamless because the engineering and technology make it “just work.”

So, what have we learned? An edge device like a consumer smart doorbell includes two communication protocols. If the homeowner does not have a Bluetooth capable smartphone or a home with Wi-Fi access, the doorbell simply won’t work. Many edge devices run in the same manner. There is a reliance on having networking and communication infrastructure.

5.2 Communication Systems Differences

There are many types of communication systems available for IOT and edge systems. Choosing the correct communication service is critical to the success of an edge project. One must determine the speed and bandwidth required for networking, the range, reliability, service costs, and power consumption. Before we begin exploring the various communication standards, we need to become conversant in the types of networking.

First, there are wireless and wired types of communication standards. Wired communication involves traditional Ethernet cabling, serial communication, fiber-optics and industrial protocols like ModBus. These are copper-based or optical wired networks. They serve a purpose as they are inexpensive, standardized, and provide exceptional bandwidth at low power. The main issue with wired networking is the constraint of the wire. Either the cabling exists and can be reused, or the cabling becomes an added cost to the system.

Alternative, there is wireless communication and networking. These are classified by their respective range:

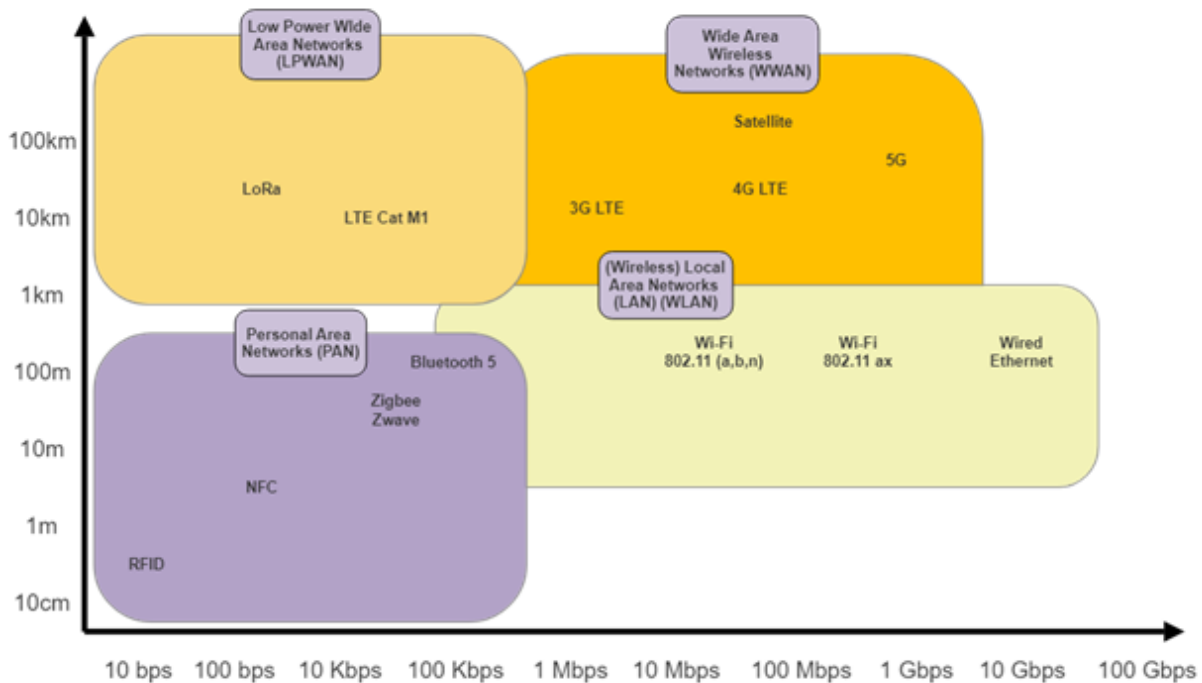
- **Personal Area Network (PAN):** These are extremely near-range types of communication systems. Typically, these are within a personal workspace ranging from centimeters to about 100 meters in extreme cases. PAN systems include popular protocols like Bluetooth and are popular in wearable IOT applications like heartrate monitors. PAN protocols are low power and can allow for battery-based sensors to be located anywhere within range. They also include extremely close-range protocols like Near Field Communication (NFC) or RFID. These ultra-low range devices can operate without any power and receive their energy through radio waves. Most PAN communication systems are not TCP/IP compatible and use an efficient but proprietary protocol for communication. PAN systems also allow for adhoc mesh networks to be established.
- **Local Area Network (LAN):** Ethernet is a widely used LAN technology and has been in existence in some form for over 40 years. Other wired LAN protocols do exist like token-ring, but Ethernet is a defacto-standard to date. LAN in this context involves wire-based networking either through copper cabling or fiber-optic lines. LAN systems interconnect computers. Wire-based LAN networks do not have the issues of wireless communication where signals can be lost

(e.g., a cellular user driving through a tunnel) or bandwidth drops in a contentious area. They are constrained by the length and cost of wiring. Either the infrastructure already exists with Ethernet cabling, or it must be designed into the project.

- **Wireless Local Area Network (WLAN):** The most common form of WLAN is Wi-Fi or 802.11. Nearly every laptop, smartphone, and mobile device support Wi-Fi. WWAN shares quite a bit with LAN technology. While one is wired and WWAN uses radios, both are TCP/IP compatible. WWAN systems typically have bandwidth of about 100 Mbps to more than 1Gbps with new protocols like 802.11ax (Wi-Fi7). However, the range is limited to about 100 meters.
- **Low Power Wide Area Network (LPWAN):** LPWAN provides two important features for edge and IoT systems: very long range and very little power. The tradeoff is bandwidth and speed of transmissions. Most LPWAN protocols are proprietary and must be licensed (both the hardware and software may require a service agreement). The most common forms of LPWAN are LoRa and LTE Cat M1. LoRa is a very long-range system capable of ranges up to 50 km and LTE is a cellular service for IoT systems. Both have limited effective bandwidth but can run on coin cell batteries. LTE is capable of transporting TCP/IP traffic, but the data units of Lora and Sigfox prohibit the size of TCP/IP data (packets). Therefore, Lora and Sigfox require edge bridging to move data to the Internet.
- **Wireless Wide Area Network (WWAN):** Cellular service and satellite communication are example of WWAN. This includes services like 5G and Starlink. WWAN requires cellular radios and cellular infrastructure such as cell towers. This can be managed by a service provider like ATT or T-Mobile, but new standards called CBRS (Citizens Broadband Radio Service) allows you to build a private cellular network and avoid the service level agreement with a telco. Satellite communication is the new variety of WWAN and allows for communication at slightly lower bandwidth than cellular but can be used where no cellular service is available. Beside the cost-of-service agreements, there is also a cost for the necessary hardware. Cellular radios typically are more costly and require more power than LPWAN. However, of all the communication standards, satellite communication currently is the most expensive form.

Figure 5.1 illustrates the speeds and ranges of these various communication technologies.

Figure 5.1 Types of communication systems arranged by their typical speed and maximum range



5.3 Radio Spectrum

If an edge system is not capable of being hardwired (e.g., ethernet cables, ModBus, serial) then the communication to various sensors (southbound) and to the Internet (northbound) will need to rely on wireless technology. This is where some understanding of radio frequencies and spectrum have a significant impact. Radio frequencies are used for everything from Wi-Fi to cellular signals to television broadcast to satellite communication to military radar. This spread of frequencies and who may use a particular slice is managed by governmental bodies like the United States FCC ([Radio Spectrum Allocation | Federal Communications Commission \(fcc.gov\)](https://www.fcc.gov/)) and the International Telecommunication Union (ITU).

Radio frequencies are measured in a unit called Hertz or abbreviated Hz. Hertz represents the number of events or cycles a radio signal per unit time. A 1 Hz signal means that the radio pulses data of 1 event of wave in 1 second. A 100 Hz signal transmits 100 cycles per second. The range of frequencies that radio communication can use is between 0 Hz to 3000 Hz. Different areas of this frequency range are allocated by the ITU to different purposes. Part of the spectrum is sold, licensed, or leased for commercial purposes such as for cellular companies. Other areas are restricted for military, maritime, or aircraft navigation. Finally, some are completely open to unlicensed usage by anyone. These portions of the spectrum that are “open” are called ISM bands which stands for Industrial, Scientific, and Medical. The ISM bands are where Wi-Fi, Bluetooth and many of the communications are allowed to operate by governmental restrictions. While this has enabled standards like Wi-Fi to be commonplace and ubiquitous in the IT industry, it also leads to congestion in these bands. Typically, only a single transmitter and a single receiver can listen to the same channel or frequency and have reliable communication. In a very crowded setting with multiple radios colliding in the same frequency space, the result is degradation in performance.

Typically a higher frequency provides more bandwidth with limited range, while lower frequencies provide less bandwidth but more range.

To understand who may use frequencies in the overall spectrum, the following table will help map some of the protocols we will explore. Highlighted are the ISM bands of interest.

Frequency Range	Usage
0 Hz to 535 KHz	Maritime and aeronautical use
535 KHz to 1.6 MHz	Broadcast AM radio
1.6M Hz to 54 MHz	Mix of maritime, aeronautical, mobile communication
54 MHz to 108 MHz	Broadcast TV and FM Radio
108 MHz to 512 MHz	Mix of aeronautical, mobile, satellite, and astronomy
512 MHz to 698 MHz	Broadcast TV
598 MHz to 928 MHz	Mobile
902 MHz to 928 MHz	ISM Band (Cordless phones, garage door openers, IOT devices)
928 MHz to 2.4 GHz	Mix of mobile, scientific applications, amateur radio, radioastronomy
2.4 GHz to 2.5 GHz	ISM (Wi-Fi, Bluetooth, Zigbee, IOT devices)
2.5 GHz to 5.725 GHz	Mix of maritime, aeronautical, mobile, radio location
5.725 GHz to 5.875 GHz	ISM (Wi-Fi)
5.875 GHz to 24 GHz	Mix of maritime, mobile, and satellite communication
24 GHz to 24.25 GHz	ISM (Amateur mmWave)
24.25 GHz to 61 GHz	Mix of 5G mobile, satellite, radiolocation, and navigation
61 GHz to 61.5 GHz	ISM
61.5 GHz to 122 GHz	Mix of 5G mobile, astronomy, satellite, and radio location
122 GHz to 123 GHz	ISM
123 GHz to 244 GHz	Mix of radio astronomy and space research
244 GHz to 246 GHz	ISM
246 GHz to 300 GHz	Mix of satellite, mobile, and radio location

In addition to restricting where one may freely or openly use a radio, the amount of power one can use for a particular radio is also limited. One may correctly assume that to increase the range of transmission you can simply increase the power to a radio. While somewhat true, the result of unbounded power can result in significant damage of radios and antennas. Antennas will transfer some of this power into heat energy and potentially cause a fire. Federal regulations also limit power within ISM bands to ensure that a single

transmitter will not overwhelm a large area. Wi-Fi (802.11) is limited to 100 milliwatts in the 2.4 GHz frequency space. This allows Wi-Fi to have a realistic range of 100 meters. Contrast this to AM broadcasting in the 535 KHz range. This licensed area is not freely available and therefore there is less of a chance of radio collisions. AM broadcasting can use up to 10,000 Watts of power (100,000 times more powerful than Wi-Fi) and can reach over 100 kilometers.

5.4 Near Range Communication (PAN)

Communication systems range from near range (within 100 meters or about 328 feet) to extremely long range (kilometers/miles in range). We start by describing near-range communication via personal area networks (PAN). These are communication systems intended to service close range (0.1' to 300') range. Think of these as communication systems for sensors, personal heart rate monitors, audio streaming devices, beacons, lighting systems, security systems, and proximity sensors. These types of networks do not connect directly to the internet as would Wi-Fi or Ethernet, relying instead on much lower power. Often devices that use near-range radios often rely on a small coin-cell battery.

Protocols that encompass near-range communication include the very popular BLE (Bluetooth Low Energy), Zigbee, Z-Wave, Thread, and custom 900MHz networks. As mentioned, these protocols **do not connect to the internet directly**, thus they do not use TCP/IP as the standard networking stack. These are termed *non-IP protocols*. Each uses a custom stack, and some like Z-Wave are proprietary. These all require special hubs or bridges to convert their proprietary protocol to an internet-capable protocol. These hubs or gateways are effectively edge computers.

5.4.1 Bluetooth

Bluetooth (named after the Norse king Blatand) was first developed by Ericsson around 1990 as a wireless method to replace cables for computer peripherals. It evolved over the years and is officially managed by the Bluetooth Special Interest Group (SIG) (www.bluetooth.org). The SIG manages the specifications, release, and logo certification for devices

wishing to use Bluetooth. Currently, over 30,000 member companies belong to the SIG and they have shipped over 2.5 billion devices using Bluetooth. The products include audio headsets, computer mice, Smartphones, and most importantly for edge computing: sensors, beacons, and actuators.

The latest Bluetooth 5.0 standard supports bandwidths of 1 to 2 Mbps but can be driven down to 125 Kbps if you would want to improve communication resilience and range. This is a good example of a technology choice point in edge architectures where we try to balance network speed with network range. You can adopt slower speeds that are more reliable and have a longer range, or higher speed when you can ensure proximity. Bluetooth operates in the 2.4 GHz spectrum and within the ISM band. Since this space is also used by Wi-Fi, Zigbee and other devices, collisions and potential degradation of performance will occur. Bluetooth attempts to manage this by a technology called Adaptive Frequency Hopping (AFH). This technology (co-invented by the actress Hedy Lemarr during World War 2 in 1942) jumps between set frequencies in a pseudo-random behavior that is synchronized between the transmitter and receiver. Bluetooth divides the 2.4 GHz spectrum into 37 channels (79 for classic Bluetooth). Each transmission uses a new frequency which helps mitigate collisions in this ISM band.

Bluetooth has many modes of operation. Since its initial concept design, Bluetooth has served as a means of file transport between computers using Bluetooth Classic, it is most widely used wireless audio technology enabling generations of wireless headsets and ear-buds, and in Bluetooth 4.0 introduced Bluetooth Low Energy (BLE). Bluetooth Classic, the older protocol, consumes substantially more power than BLE. Thus, it is not suitable for many edge sensor networks and personal area networks that rely on coin-cell batteries. Unlike many other forms of hardware and communication accessories, Bluetooth devices are plug and play. Essentially, they do not need any intervention by the end user or specific software drivers that needs to be loaded. Manufacturers can design different Bluetooth hardware but are assured that they will always work now and in future versions. Additionally, Bluetooth is hot-pluggable. That implies a device can power down, reset, or become inactive without affecting the host system. When power is restored, the device will maintain a *registry* that allows it to automatically pair with the host automatically.

To help contrast the difference, reference the following table:

Parameter	Bluetooth Classic	Bluetooth Low Energy
Frequency	2.4 GHz	2.4 GHz
Range	<100 meters	>100 meters
Maximum Speed	1 to 3 Mbps	Up to 2 Mbps
Latency	100 ms	6 ms
Typical Power	1 Watt	0.01 Watt to 0.5 Watt

Concentrating on BLE as it is the most widely used version of the Bluetooth protocol for IOT and edge systems, it is a master -slave communication. This means there is a strict one to one relationship between a master Bluetooth device and a slave Bluetooth device. BLE uses three advertising channels (out of the 40 total channels) to broadcast identities and allow a master and a client to “find” each other. The process is called *pairing* and is a multiple step process to establish a communication link between the master and slave device. Once established, data communication between the two nodes may commence. One aspect of BLE and the pairing process is the BLE profile. The BLE profile is a set of parameters (or a contract) that specifies the fields, addresses, and data values for a particular device to communicate. In other words, a device like a BLE temperature sensor communicating an edge master computer encodes temperature characteristics at specific addresses in the BLE communication. There are many profiles Bluetooth has standardized upon such as heart-rate monitors, glucose monitors, telephony, and so on. Refer to the Bluetooth specification located here ([Specifications –](#)

[Bluetooth® Technology Website](#)). One may also adopt a proprietary profile for their implementation.

Secure communications are also managed by Bluetooth with five basic security features. It provides authentication services through verification and identification of communicating devices. All data in transit can be encrypted using common encryption standards. One can also limit authorization of devices by whitelisting devices that are allowed. All data is also verified for integrity to ensure the data in transit has not been altered. Finally, during the pairing process secret keys are exchanged. When building a Bluetooth application, the designer can choose from five different security levels:

- **Level 0: No security.** This level is used to discover Bluetooth services between master and slave devices.
- **Level 1: No Security.** This non-secure mode does not authenticate devices nor encrypt data.
- **Level 2: Unauthenticated Key.** Allows for semi-secure authentication on initial pairing. Additional security can be provided later.
- **Level 3: Authenticated Key.** In this mode, security is established before the communication link is established. This ensures even device authentication is secured. This mode is intended to stop man-in-the-middle (MITM) attacks. MITM will be discussed in the security chapter of this book.
- **Level 4: Authenticated Key and Secure Connection.** The highest level of security. This mode requires encryption of data for all transactions. This can stop MITM attacks and is also FIPS approved. FIPS stands for Federal Information Processing Standard and is a set of minimum security requirements managed by the National Institute of Standards and Technology, IEEE and the International Organization of Standardization (ISO).

Bluetooth and specifically the BLE portion of Bluetooth has evolved significantly and provides a rich mixture of features and use cases that are especially relevant for edge computing and IOT systems. Some of these features include:

Feature	Bluetooth Version	Description and Use cases
Angle of Arrival and Angle of Departure	5.2	Used for 3-diremsional spatial location and tracking of objects. This allows for an edge device to find, track and locate objects and their position with centimeter level accuracy.
LE-Audio	5.2	This feature is the new upcoming standard for Bluetooth audio. It is significantly lower power and provides audiophile level fidelity than traditional Classic Bluetooth audio. It allows for use cases such as massive stadium level broadcasting of audio data. For example, it can be used by edge system in amusement parks, airports, and stadium to broadcast audio in sync with thousands of listeners. It will be used for modern hearing aid and assistive devices.
Beacons	5.1	Bluetooth can use its advertising channels to broadcast information from and edge device to listening devices like customer mobile phones. This allows for proximity-based information to users. For example, retail customers approaching a sale item in a store can be notified with a listening application. Museums also use this to inform patrons on exhibits when they approach them.

Bluetooth can also act as a mesh, although that is still uncommon but may gain traction in the near future as Bluetooth 5.0 has enabled significant advances in mesh topologies. In Bluetooth vernacular, a master device could very well be an edge computer acting as a gateway or router. Bluetooth slave

devices can have multiple masters as long as they pair and are secured in communication.

Bluetooth is a pervasive communication standard that is used in consumer, retail, enterprise, and industrial IOT devices and is a good example to follow of a well-designed protocol for PAN communication standard when evaluating other PAN systems.

5.4.2 NFC and RFID

Some use cases in industry require extremely low cost and low to no battery power supply. These types of use cases include asset tracking systems for products on shelves or logistics, low-cost disposable identification systems for tracking patrons at an amusement park, track progress of components as they move through an assembly line, or livestock management for a large herd of animals. In all these cases, some unique identification and information carrying device is on the product or person. These ultra-low-power devices require little to no supplied battery, very low costs to the point of being a disposable appliance, and wireless communication to an edge device that can communicate with it. Radio Frequency Identification (RFID) and Near Field Communication (NFC) serve these use cases. Both protocols are not intended for high-speed communication of any kind.

RFID uses radio-frequency communication with the device called a tag and the device reading or writing to the tag is called an interrogator or reader. Tags can be passive (without any power source or battery) or active/semi-passive (with some power source or battery). Both active and passive tags operate in ISM bands of the radio spectrum.

A passive tag is the lowest cost method of using a digital tag of any IOT device since there is no added cost of a battery and power electronics circuits. A passive device derives its energy when it is in the presence of electromagnetic radio waves generated by an external “reader”. A passive tag will be comprised of an antenna and the tag with some memory source attached to it. When a reader generates a signal of sufficient strength and within the range limits of the tag, the device will couple the energy and awaken. Coupling is the process of transferring energy from one medium to another. For an RFID, this can be inductively coupled or backscattering.

Inductively coupling is best for very short-range communication where the interrogator modulates its radio frequency (called the carrier frequency), and the RFID tag's antenna uses a coil that will convert the magnetic energy to electricity to power the tag.

Backscattering communication allows for slightly longer ranges of about 10 meters. Backscattering can be most easily explained by using a flashlight and mirror analogy. Imagine if our interrogator were a flashlight and the tag was a mirror. You could shine a light at the mirror of a certain color and the mirror would reflect some of the light back. Additionally, the mirror could modulate the light in a series of ones and zeros to communicate data back to the interrogator. The same occurs in backscatter RFID. Here the electromagnetic radio waves produced by the reader will power the RFID tag, and the same signal will be reflected back to the reader with data. The range between the RFID reader and tag is called the *interrogation zone* of RFID and is typically less than 15 meters.

An active RFID tag supplies its own power source (typically a battery). Active tags will continuously generate their own signal. These function as beacons and provide much longer distance to communicate with an interrogator. Since an active tag incorporates a power source rich and complex algorithms and functionality can be embedded in the device. An active tag can also operate as a beacon and constantly be transmitting a signal for location sensing or alerting. Another powerful feature of active tags is that they can communicate not only with a read device but to other tags. Thus, active tags are used heavily in real-time location systems (RTLS) in shipyards and large warehouses. Semi-passive tags incorporate a battery but use that power only to supply energy for the microprocessor or other logic on the tag. Semi-passive devices require backscattering or inductive coupling to communicate with a reader. This allows the semi-passive tag to keep running and monitoring what it is attached to even if a reader isn't present to supply power. This is useful for environmental or agricultural sensors that monitor temperature, humidity, shock, PH levels and so on and then report the data when a reader is present.

NFC is the other near range communication system. Unlike passive inductive coupled RFID that can communicate with an interrogator within 1 meter range, NFC operates in four centimeters or less. Like RFID passive

tags, NFCC uses inductive coupling to provide wireless energy to low-cost NFC tags. NFC operates in the 13.56 MHz radio frequency and at very limited data rates of 106 kbits/second to 424 kbits/second. Where RFID used the terms “interrogator/reader”, and “tag” NFC uses the terms initiator to describe the reader and target for the NFC tag.

NFC tags are used heavily in industries like identity tokens at amusement parks and in financial transactions. New smartphones using Android and Apple iOS devices have NFC initiators built into the phone for transactions at millions of retailers and vending machines. Simply place the phone next to the Apple Pay, Google Pay, or Samsung Pay icon on the cash register of soda machine, and the transaction will occur automatically. This technology has effectively removed the need for cash or credit cards in personal transactions.

The following table contrasts the differences in RFID and NFC tags.

	RFID Active Tag	RFID Passive Tag	NFC Tag
Range between tag and interrogator	150 meters	10 meters for backscatter tags 1 meter for inductively coupled tags.	4 centimeters
Typical Operational Radio Frequencies	435 MHz and 2.4 GHz	860 to 960 MHz	13.56 MHz
Cost of tag	About \$20	Less than \$0.50	Less than \$0.10

Battery	Yes	No	No
Size of tag	About the size of a small smartphone	Smaller than a credit card. Can be flexible as on a label.	Smaller than a quarter
Indoor/Outdoor	Outdoor	Indoor/Outdoor	Indoor/Outdoor
Logic Complexity	Can perform complex calculations.	Simple tag and ID information.	Extremely simple

5.4.3 Meshes (Zigbee PAN)

A mesh is another form of networking, but unlike a peer-to-peer model (like wired Ethernet), a mesh allows nodes to build their own network in an ad-hoc manner. This is good for cases where no infrastructure for networking exists and one must build a network themselves. Mesh networking consists of *nodes*. These nodes may in fact be edge computers or IOT devices, but they communicate to the cloud by communicating with each other.

A good example is building parking sensors in a public parking facility. At an airport I frequently visit, there is digital signage on the parking lot entrance indicating how many open spaces are on each floor of the 5-story structure. This allows patrons to quickly go to a floor that has open spots and avoid driving aimlessly around the facility looking for a spot. Each parking spot has an electromagnetic sensor to determine if there is a large chunk of metal directly above (e.g. a car or truck). If the spot is occupied, the sensor communicates to a central computer via a mesh network. Data is sent from the sensor to the next sensor and eventually to a mesh gateway to connect to the Internet. Each sensor, therefore, is a component of the network infrastructure. So why do this? The reason is that in many situations like a

parking structure, the infrastructure was built decades ago, and laying fiber or cable to each parking spot was not a financially viable option. Additionally, using WiFi hotspots could leave shadows of radio communication and some parking spots may have no good connectivity.

There are many mesh networks: Bluetooth and WiFi both have mesh models that can be used. Thread is another example of a low-cost and low-energy mesh network. We will examine Zigbee as it is ubiquitous and very popular for IOT mesh networking and covers many of the aspects of how a mesh works.

Zigbee is another short-range personal area intended to conserve battery power for wireless sensors and IOT devices. Like Bluetooth, this is a non-TCP/IP protocol, meaning that it can't connect directly to the internet and requires an edge gateway. Zigbee was launched in 2002 and the specification for the first draft was released in 2004. Today, it has found its way into products like smart lightbulbs and lighting systems, home and enterprise security systems, shipping and logistics, livestock management, and food processing (especially in cold storage and supermarket freezers). At its core, Zigbee acts as a "mesh network," a common deployment in edge computing where you can't run wires or provide the infrastructure to connect a large number of devices.

Let's examine a popular consumer device: smart lightbulbs, light strips, and even light switches that a consumer can install in their home. The bulbs simply screw in a can light or a lamp like a normal bulb. The lights, however, are all connected and through a smartphone application; one can control turning the lights on or off, change the intensity of the light, change the color to one of over 16 million different colors, and even set up complicated scripts to do interesting things with the lights. For example, I run a script that changes the color of a lamp to reflect how the stock market is performing. I programmed the script to get real-time stock information through an open software interface. If the stocks I own are trending up over a given time the bulb will increase in intensity and glow blue. If the stocks are trending down, I program the bulbs to increase intensity in red. Although this is a personal hobbyist demonstration for my own entertainment, it illustrates that a simple lightbulb can interface to the stock market.

So how does a light no bigger than a standard incandescent bulb connect to the internet and the cloud? Moreso, how can it do this it does this without relying on Wi-Fi or a cellular connection. The answer is mesh networking. At its core these smart lights have a small computer and Zigbee radio embedded within it. The range of Zigbee is fairly short and similar to Bluetooth at about 100 feet for typical reception. When using Zigbee you don't address other nodes with an IP address like 192.168.0.1 or a URL name like www.mit.edu. Rather, you use a special form of addressing to communicate from one Zigbee device to another called a mesh.

There are three forms of Zigbee devices (called nodes):

- **Zigbee Coordinator:** This is the master device and only type of node that can form a Zigbee network from start. It is also required to add additional nodes to the mesh or remove a node. The Coordinator is the *root* of the network.
- **Zigbee End-Device:** These are designed for very low power (e.g., battery powered) communication. End devices will sleep when no transmission is required. Since an end device may be in a sleep state for extremely long periods of time, when it does wake it will *poll* its parent to retrieve any data being transmitted to it. Therefore, it is not suitable for real-time applications.
- **Zigbee Router:** These devices can extend the range of a Zigbee network. Remember, this operates in the 2.4 GHz band where communication is limited to less than 100 meters for reliable transmission.

To address various nodes in a Zigbee mesh each device will have a unique Media Access Control (MAC) address. This is similar to how Ethernet and WiFi work, a MAC address is unique to the device and is 64 bits in length. This ensures that there can be over 4 billion zigbee devices covering each square meter of Earth's surface and still have capacity for more devices. This should ensure there will be enough unique addresses for a long time. To obtain and register a Zigbee MAC address one must work with IEEE (www.ieee.org) and there is an associated \$1600 fee to register the address. The MAC address allows a zigbee device to actually reside on two separate and secured Zigbee mesh networks simultaneously.

In addition to the MAC address, Zigbee uses a shortened address called the node address or *NwkAddr*. The *NwkAddr* is a smaller 16-bit number that identified a particular node on a single ZigBee network. The Zigbee Coordinator is always *NwkAddr* 0x000. There is no relationship between the MAC address and *NwkAddress*. The *NwkAddress* is assigned when a node joins a Zigbee network while the MAC address is built into the hardware and stored in non-volatile storage on the device, so it remains static.

There are 4 ways to send a message on Zigbee:

1. **Table Routing:** This is typical of mesh networks. In this form, an end-dive will discover the path to a target node by building a routing table of the network to determine how to get to the destination node. The source will send data to the first node in the route which will subsequently forward the message to the next node and ultimately arrive at the destination. If any node fails in this chain, an error is sent back to the originator and a new routing table will be created. This allows for ad hoc networks to exist and come and go at will.
2. **Broadcast Routing:** In this form of communication, a node will send data to every node in the network. This should only be used for specific messages and issues like system errors. Broadcasting can overload a Zigbee network.
3. **Multicast Routing:** This form of communication allows a one-to many type interchange of messages. The difference between multicast routing and broadcast is multicast uses preformed *groups* of Zigbee devices that are associated with one another. For example, in lighting use cases, a group may consist of a single Zigbee lightswitch and 4 Zigbee light fixtures. Only those members of a group will receive the message from a switch.
4. **Many-To-One & Source Routing:** Both Coordinator forms of communication originate from the central Zigbee Coordinator and are sent to Zigbee Router nodes. Essentially these are utility messages and are used to help build the Zigbee mesh network routes.

A mesh doesn't rely on any infrastructure such as a Wi-Fi access point or an internet service provider. Instead, it forms a network in an ad hoc fashion, allowing the system to morph and change with nodes coming online and going offline all the time. This is a perfect use for lightbulbs! Think of it this

way – to intelligently connect a dozen light bulbs and lamps in your house you have two choices: either use Wi-Fi communication and burden the cost of each light bulb with a Wi-Fi transceiver, or use a mesh protocol like Zigbee. The cost of a Wi-Fi radio is typically 10x more expensive than a Zigbee or Bluetooth radio. It also requires each light bulb to send a signal to your Wi-Fi router, which may be difficult if the lights are outside your home or in your garden.

The figure below illustrates a typical Zigbee network. Notice that each NwkAddr is unique and the Zigbee Coordinator always starts with address 0x000. Additionally, note that there are multiple paths for some of the nodes. End-Device #7 has two routes in the network. Either it can deliver data through Router B or directly to the Zigbee Coordinator. Additionally, Router A and Router B are attached to the Coordinator but also can communicate with each other. This is what enables Zigbee mesh to provide resiliency and redundancy in communication. It achieves to always find a path to communicate.

Figure 5.2 Typical Zigbee Mesh



A Zigbee mesh requires no Wi-Fi access point or router, and the network is built one node at a time. Lights (especially overhead can lights) are evenly spaced providing a perfect framework for Zigbee lights to communicate with each other. When you add a new smart light to your home, it attempts to join the current mesh by a process called *association*. The problem with a mesh is that it isn't very fast with respect to bandwidth. If you are at one end of your home and wish to control the color of a light at the other end, the packets of data you send cross each node in the mesh until it eventually reaches the correct device. Bluetooth, too, can form a mesh. A mesh network is great for edge computing use cases like parking meters in Smart-Cities. Have you ever entered a parking garage which shows how many spaces are open? Each parking space has an edge sensor to detect parked cars that communicates via a mesh network. However, you generally will never want

to use a mesh network where you need very high performance, such as the smart doorbell which shuttles large amounts of video.

So far, these types of communication systems are low energy and have fairly short range. Bluetooth and Zigbee only have a reliable range of about 100 to 200 feet. These short-range communication systems require a hub or special hotspot to proxy data to and from the internet. Bluetooth and Zigbee are not meant to directly attach to the internet as you do with a Wi-Fi system. There is typically an appliance called a bridge that convert Bluetooth or Zigbee to protocols usable on the internet. Sometimes these hubs or bridges have significant intelligence built in and they too can be considered edge computers.

5.5 Near Range Communication Use Cases

We've talked about Wi-Fi, Bluetooth, and mesh networks, and while these technologies are ubiquitous and cheap, they aren't always suitable for edge computing. Often, edge computers may need to be in a very remote area well outside of any infrastructure like cable internet. Wi-Fi access points and Zigbee bridges all connect to the internet in some manner – typically through your home cable modem or router. However, what do we do for remote edge computing? What if we need to connect a herd of cattle to the internet?

Let's back up a bit – why would you even want to connect a herd of cattle to anything? According to the acclaimed conference, IOT World Congress, smart agriculture is slated to be a \$25 billion business by 2025. The use cases are numerous. Large scale ranching and farming are heavily automated and computerized. Farm plows are now self-driving, robotic, and autonomous. Each head of cattle on a 1000-acre ranch are connected to the internet and monitor to understand their health, location, and feed. How about smart cities that intend to connect every light pole to monitor traffic and identify safety problems. Local municipalities and the federal government intend to transform the 164,000 miles of the United States interstate systems into smart-connected roads. These problems require a different communication system than the short-range communications of Bluetooth and Zigbee.

5.6 Long Range Communication

If one needs long range communication and networking then cellular technology is the de facto standard communication medium. Besides providing service for smartphones, cellular infrastructure is pervasive. According to Ericsson, there are over 8 billion cellular subscriptions as of this writing in 2021. Think about that – there are only 7.8 billion humans on earth. A staggering number of cellular subscriptions are in effect and roughly 3.5 billion of these users will transition to 5G service by 2026. But mobile phones aren't the only devices using cellular communication. IoT and edge devices also utilize this service when communication is necessary that requires high bandwidth, low latency, and good coverage.

Edge systems utilizing cellular service include nearly every first responder vehicle in the US, including ambulances, police, and fire trucks providing internet connectivity and global positioning services. Waste management systems that provide plastic and recyclable waste bins in different cities around the United States use cellular edge computing. These intelligent “garbage cans” inform the collection trucks which bins are full or need service saving time and money. This is convenient for this type of computing because the waste bins are portable and can be moved. There is no reliance on a wired internet connection nor wired power. Both these use cases share one thing: they have no pervasive internet connectivity such as your home or office may have.

5.6.1 5G and Cellular

The concept of wireless telephony had its roots at Bell Labs in 1946 where Douglas Ring and Rae Young created that a cellular array of towers that could form a mobile network. In 1979, Motorola came to market with the first cell phone developed by Martin Cooper in order to brag to a rival that they produced the first mobile phone and phone call. Back then, the cellular devices were extremely large, power hungry, and analog. Modern communication is all digital and needs high speed and low power. What makes cellular systems different than other forms of wireless communication is that the device can be mobile and not at a fixed location. Whereas Wi-Fi may have a reach of a few hundred feet, cellular signals can extend reliable

transmission for many miles. This is enabled by the very fact that the radio towers providing communication act as “cells” and allow a moving mobile phone to “hand-off” from one tower to the next, hence the name cellular.

One of the main governing bodies over cellular standard is the Third Generation Partnership Project (3GPP). The 3GPP is organized by seven telco organizations worldwide and is responsible for defining the underlying technology to meet the standards set by the International Telecommunications Union (ITU) The ITU defines cellular architectures and roadmaps and governs the standards of mobile communication. This governance body actually came to existence in 1865, well before wired telephone systems became a mainstay. They are the body that ensure a cellular device that works in North America can also work in Asia-Pacific regions. They also set the standard as to the labelling of 3G, 4G, and now, 5G technology.

5G (also called 5G NR for “New Radio”) is designed to replace the current 4G LTE (Long Term Evolution) infrastructure. Some parts of 5G are essentially identical as 4G LTE as it should be since it’s backward compatible. 5G will fall back to 4G if the user is not within a new cell site. Additionally, a user with a legacy 4G LTE phone may wander into an area with 5G cells, but everything must work and be backward compatible with older mobile devices.

There are three significant benefits for 5G. This is what makes this technology so exciting and important:

- **Enhanced Mobile Broadband (eMBB):** This part of 5G is all about faster, bigger, and better. When most people think of 5G they are thinking about eMBB as it provides the ultra-fast communication. The specification states that 5G can provide 1Gbps to 10Gbps in the field. This new radio technology should also support 10x to 100x the number of mobile devices per tower than 4G. 4G LTE typically could serve 100 active users on one cell tower, 5G would take that to over 1000. Finally, the specification states that this class of service should be operational on vehicles moving at 500 km/hour.
- **Ultra-Reliable and Low-Latency Communications (URLLC):** This portion of the 5G specification states that devices can operate with an

end-to-end latency of less than 1 millisecond. Typically, 4G LTE suffers from latency of tens of milliseconds to hundreds of milliseconds. For video streaming, online gaming, and real-time IOT this is an issue and makes these types of things impossible on a generic smartphone. Additionally, URLLC by its very name ensures 99.999% uptime of service.

- **Massive Machine-Type Communications (mMTC):** This feature sets the goal of support for one million edge devices per square kilometer. This is 1000x the more coverage than 4G LTE can provide. Additionally, the radios should support a 10-year battery life and consume just 10% of the power of 4G LTE radios.

These specifications should be read as what the ITU determined for a system to be called 5G. However, your mileage may vary. For example, we may not see 5G systems support 10Gbps service in a train moving at 250 km/hour for 1000 passengers aboard that train.

How 5G obtains these speeds and density improvements is no surprise. Part of the technology lies in the expanded radio spectrum that has opened up for new communications. Radio signals move at the speed of light through the air. These signals essentially vibrate at different frequencies or Hz. Radio signals range from 1Hz to 300 GigaHertz. The combination of all the different frequencies is called the radio spectrum. 4G LTE was carved into narrow slices for various carriers like AT&T, T-Mobile, and so on. These carriers used spectrum in the 700 MHz to 900 MHz range and 1.9 GHz to 2.6 GHz range. 5G reuses those operational frequencies range, but also uses the 24 GHz to 28 GHz range, the 39 GHz range, and even frequencies as high as the 60 GHz to 80 GHz range. Any frequency above 30 GHz is a millimeter wave (mmWave). Lower frequencies could transmit data in the hundreds of megabit speed, while the higher frequencies can reach **gigabit** speeds. 5G could be faster than your home internet. This sounds great until we realize that these higher frequencies have significant and profound range issues. The faster the signal, the shorter the range. The typical cellular range of 1km to a nearby cell tower will result in 22x to 28x loss in quality. When a wireless signal degrades in quality you lose information, like in a noisy phone call and missing every other word in a conversation.

So how does it really work with such a poor-quality connection? Essentially, these mmWave signals will require many more towers than what we currently see across the country. Where a 4G antenna would be spaced based on user density as well as coverage distance of about 1km apart, 5G mmWave systems will require thousands of small-cell (also called femtocell) micro towers. These may be propped up on each city intersection, streetlights, and leased space on the sides of building, and could be as close as 100 meters apart in urban environments. It's also worth noting that each of these small cells are essentially edge computing devices. 5G enables edge computing by using edge computing!

The other issue with these high frequencies is their notorious inability to penetrate things like glass or even fog. A typical masonry brick wall between a femtocell at 39 GHz and a 5G Smartphone can result in a 40x loss of quality and may not operate at all. This implies that not only will femtocells be required outside, but also within multiple locations in a building or home. This may not be acceptable for everyone, nor affordable as these femtocell edge systems are essentially mini cell towers and come with some cost and power demands. Alternatively, though, 5G can fall back to 4G frequencies at a reduced data rate. This is the conundrum of 5G technology and must be considered when deploying a well architected edge solution.

An interesting twist in 5G communications that has seen a growing traction for some edge deployments is Citizens Band Radio Service (CBRS) and Multefire, a derivative of cellular technology that uses unlicensed radio spectrum. Therefore, anyone can use this space for communication, not just telco providers. This has a major impact in the service charges and business model since it is based on the same underlying cellular technology and standard cellular radios in client devices can be used. You just need to bring your own infrastructure. This may have tremendous impact on adoption of cellular technology and a disruptive technology for Wi-Fi. Imagine a shipping port that is monitoring logistics and freight of thousands of containers. A technology like CBRS can reach across the entire shipyard and monitor each container's location and status in real-time using a standard cellular radio appliance. It can do this without requiring a service plan for each radio from a telco service provider.

Unlike previous cell models, 5G relies less on the hardware and radio and more on the software control, routing, and software-based security. Prior cellular systems like 4GLTE used custom hardware from companies like Ericsson and Huawei to control all the communication from cell towers to the central offices. 5G is much more generic, utilizing software defined services that are more flexible and less costly. Large carriers like ATT, Sprint, and Verizon have introduced Multi-Access Edge Computing (MEC) that allows cellular carriers to offer edge computing services on their radio network – very close to the cellular customers. Since most of 5G software runs on commodity hardware, this is relatively easy, and the benefits can be significant. By providing edge servers close to the customers services and devices can reduce latency and also reduce the carrier's load on the network. This technology allows content delivery services like Netflix video-streaming and streaming video game services making use of this service.

5.6.2 LoRaWAN

While cellular is a pervasive communication technology, it comes at a cost in service plans, and as we have seen, 5G will require many more small cell systems to be deployed to get good coverage. These data plans depend on the category of service. A typical 5G consumer cellular plan may cost \$90 per month. But what do you do if you are building an edge system to monitor cattle in a remote region of Montana? What if your solution is a true autonomous edge system that doesn't need the bandwidth capability of 5G nor can you afford the cost when you have thousands of edge systems?

Low Power Wide Area Networking (LPWAN), and specifically Long Range (LoRa), technologies offer one potential solution. Managed by the LoRa Alliance which formed in 2015, LPWAN was originally developed by the French company Cycleo and subsequently acquired by another company called Semtech. The technology involves proprietary hardware developed by Semtech that operates in the 800 and 900 MHz region of radio space, an unlicensed spectrum that uses a different encoding scheme than cellular radios. Essentially this entire spectrum transmits a robust signal that withstands interference and can penetrate concrete walls. It is a simple protocol, using a technology called ALOHA. If there is airwave contention, ALOHA simply tries sending the data again until it is successful. In contrast to cellular or Wi-Fi that have very sophisticated carrier sensing, time

division multiplexing, and advanced modulation techniques to ensure little contention or error rates, LoRa can have error rates of 50% and still be functional. An entire LoRa system can be purchased, including the edge gateway, antennas, and a mixture of environmental sensors, for under \$500 and provide a radius of connectivity of several miles. The entire cost of building a LoRa system can be 3x to 10x less than the cost of any cellular system including the purchase of a LoRa gateway. LoRa represents the physical and modulation techniques of this form of LPWAN.

LoRaWAN, however, is the communication protocol used by LoRa that sits above the physical hardware layers. It is possible to have a LoRa radio that does not use LoRaWAN.

The downside to LoRa is that the bandwidth is capped to 0.5 Kbps to 5 Kbps. Latency is also poor with typical latencies of 1 to 1.5 seconds (remember 5G at under 1ms). Therefore, real-time use cases like gaming or video aren't suitable for LoRa. However, this technology is perfectly suitable for IOT devices that trickle data to the cloud and perform more functions on the edge. Use cases include livestock management in remote areas of Montana or monitoring HVAC and smart building infrastructure.

5.7 Satellite Communication

Finally, low earth orbit (i.e. LEO) satellites are also a promising place for edge connectivity. Satellite based communication has been around since the 1960's and satellite-based consumer networking has been available from various companies ever since. These legacy satellite systems (called Echostartm satellites) find their way into connectivity for aircraft as well as 1.3 million American consumers. These types of satellites reside in geostationary orbit at 22,236 miles above fixed locations like North America. Low earth satellites are not fixed and orbit at about 340 miles above the surface of the earth. Not being in geostationary orbit means that these satellites are constantly moving with respect to a position on earth. Many satellites need to be launched at this orbit to ensure that there is always one communicating to users in a narrower area.

In January of 2020, Starlink™ had 50 satellites in orbit. In December of 2020 that grew to 955 with plans approved by the FCC for 12,000. Starlink™ has also sanctioned the ITU with plans for 30,000 total satellites that will provide coverage for the entire globe. While there are substantially more satellites than legacy geostationary systems, they are much smaller (about the size of a pizza box and weighing 260 kg). This congregation of satellites is termed a “constellation”. Each satellite is equipped with lasers used to communicate with each other. As the satellites stream across the sky, a user’s internet service is maintained as service is handed over from one satellite to another. This is analogous to the handover scheme in cellular technology. The distance between the earthbound transceiver and the low orbit satellite is much closer than geostationary systems, reducing latency tremendously. Starlink™ devices show latency in the 30ms range, while geostationary systems average between 600 to 700ms. This makes certain use cases in edge computing and real-time content streaming untenable.

Other low earth satellite ventures have also manifested in lieu of Starlink™ systems including Telesat™ with a 117-satellite constellation and Richard Branson’s OneWeb™ with 700 satellites. Initially, these systems will provide internet connectivity to nearly every point on the globe. If you can see the sky, you will have connectivity. Eventually, this will enable significant competition for connectivity against earth based 5G cellular and traditional broadband companies. Thus, low earth satellites will be an important component for edge-based computing in the most remote areas of our planet.

An interesting hybrid of LEO and LoRa is also being explored by various startups: Fleet www.fleetspace.com, Swarm www.swarm.space and Lacuna www.lacuna.space. This combines the global and worldwide reach of satellite communication with the low-power and low-cost of LoRa devices on earth’s surface. The intent is to have worldwide coverage and communications for IoT sensors and low-bandwidth devices. In this scenario, a LoRa host is not needed as the satellites provide that service. These small satellites fly in constellation patterns ensure total global coverage and persistence. They fly at approximately 500 km and store messages received by sensors, When the satellite passes of a ground-based relay station, the data is dumped from the satellites and the broadcast to their destination or Internet site.

5.8 Summary

- All edge computing devices are based on providing communications to the cloud and to devices. The choice of the correct protocols and communication medium is critical to a safe, robust, and commercially viable edge solution.
- Edge systems may require multiple forms of communication for authentication purposes, initial provisioning or even out-of-band failsafe scenarios.
- Communication to the far edge which includes sensors and devices requires Personal Area Networks. These provide near-range communication with low energy but are not intended for high bandwidth applications. PAN protocols include:
 - Bluetooth, which is a widely adopted near range protocol that can be used for everything from file transfers to audio transmission to low energy IOT to location tracking. It is a robust protocol that can deliver data reliably 100 meters or longer with modest performance.
 - NFC and RFID tags which allow for very low-cost methods for communication especially in industries of logistics, location tracking, ID cards, shipping, factory automation, and has changed how modern electronic payment systems work, effectively eliminating the need for cash and credit cards.
- Often times edge systems require an adhoc communication solution where networking infrastructure doesn't exist. For these applications we use mesh networking which allows for developing a dynamic and scalable network. Mesh systems include Zigbee and Bluetooth 5.0 which are used extensively in residential and commercial lighting systems.
- To reach back to the cloud we use long-range communication systems that includes services like 5G/cellular, satellite and LPWAN.
 - LPWAN systems that are mostly proprietary but also provide extremely long range as measured in kilometers and also very low power.
 - WWAN which allows for long range wide area networking via cellular systems and satellite. These require more power but also have considerably more throughput and bandwidth than LPWAN.

6 Edge protocols: The language of Edge machines

This chapter covers

- Fundamentals of TCP/IP networking and the OSI model.
- Models of network communication: message-orientated, streaming, and RESTful.
- Industry 4.0 and the protocols in industrial and factory automation.
- Cloud to edge communication using MQTT
- Alternative protocols such as CoAP and AMPQ

We have talked about edge machines, hardware and the types of technologies that are used to communicate with an edge device, however, we have not explored the language used to *talk* to an edge computer. Edge devices *speak* certain languages when communicating to the cloud or other edge machines. Think of it this way, when we make a phone call or communicate by email, we use a language, grammar, and sentence structure to have an effective conversation. Edge computers also needs a *language* or protocol. This chapter will visit the important protocols that edge nodes must consider dependent on the use case. We will talk about the ModBus and other industrial protocols used in factory automation as well as protocols like MQTT used in a variety of consumer electronic devices.

6.1 Network Layering and the Basics of the OSI Model

A helpful way to understand networking and protocols is the Open Systems Interconnect Model. It exemplifies how modern networking software and hardware is built upon *layers* of abstraction and functionality. The bottom of the OSI model is the physical medium such as a WiFi, Ethernet or a cellular radio. This layer involved the actual hardware used in communication. Above the physical medium is the data link layer responsible for controlling

the underlying hardware. It treats all data as a series of packets transmitted synchronously. Level 3 is the network layer. The network layer and the transport layer are usually what we refer to as TCP/IP. It is responsible for generating an internet address that can be routed and delivered across many different networks. This chapter will look at top layers of the OSI stack: 5, 6, and 7. This is where software will describe particular languages common to the edge.

Figure 6.1 The OSI model and progressive layering of networking stacks.

OSI (Open Source Interconnection) Model		
Layers	Purpose / Function	Examples Protocols
7. Application	User Application Layer: web browser, file transfer	SMTP FTP HTML
6. Presentation	Syntax Layer: encrypt, compress (optional)	JPEG, ASCII, ROT13
5. Session Layer	Synchronization & Logical Port Routing	RPC, NFS, NetBIOS
4. Transport Layer	TCP: Host to Host & Flow Control	TCP / UDP
3. Network Layer	Packets: IP Address	IP
2. Data Link Layer	Data Frames: MAC address, packet	PPP/SLIP
1. Physical Layer	Physical Device: Cables, fibers, RF spectrum	Coax, fiber, wireless

The concept with a network stack is to provide a layering of abstractions that serve different purposes in a network. This allows protocols like TCP/IP to have clean separable layers and strict interfaces between layers so one can change or improve part of a network without redesigning a large monolithic piece of hardware and software. Layering also allows TCP/IP to be used on

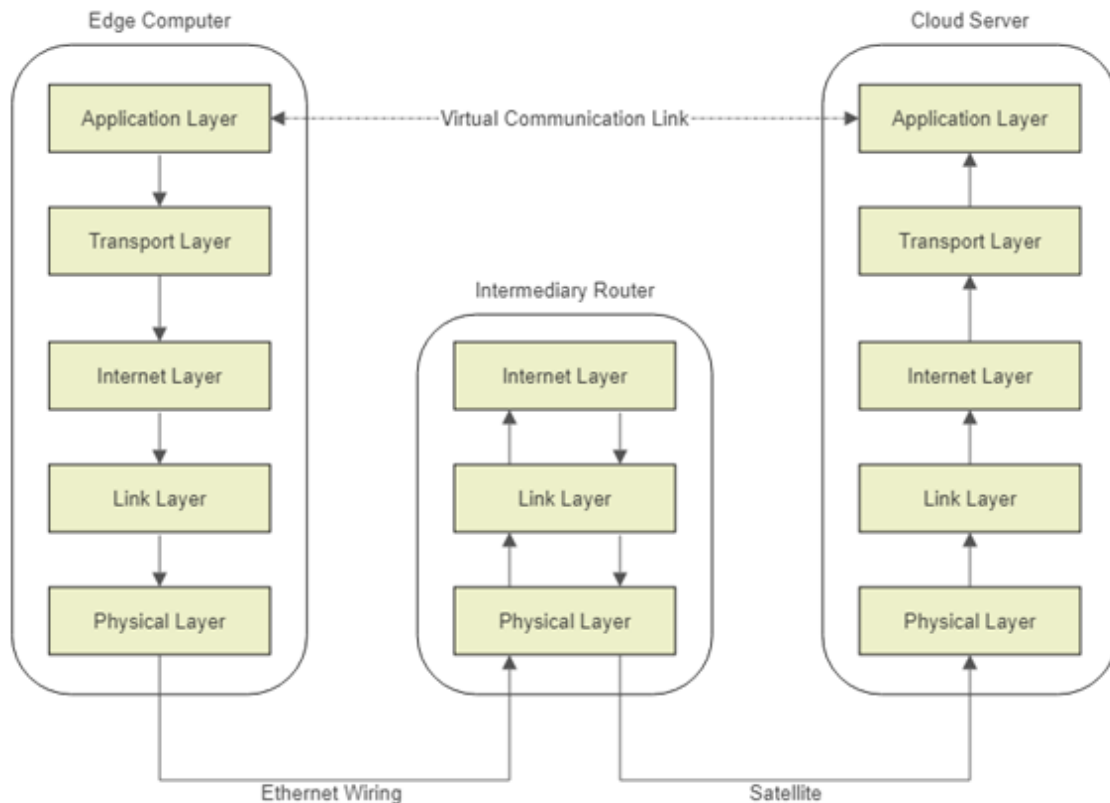
different physical networks like Wi-Fi, copper wires, cellular systems, and optical interconnects by using a different link and physical layer in the stack.

6.2 Diving Deep Into TCP/IP Networking

The defacto standard of networking and wide area networking is TCP/IP (Transmission Control Protocol and Internet Protocol). Like the academic OSI model of networking, TCP/IP follows a similar layering concept. TCP/IP is used pervasively and runs the entire IT world and internet. Think about a website you visit. To connect to a remote website, you will use TCP/IP to connect and communicate to the site, but the data representative of the web page may include text, font formats, images, video, and audio. That description is part of the HTML format and transmitted using HTTP (hypertext transport protocol) that describes web page content.

TCP/IP is a protocol to marshal packets of data to destination addresses that are embedded in the protocol. This allows you to browse the web and transmit data to and from google.com, as an example.

Figure 6.2 The TCP/IP stack layering. This illustrates the hierarchy of different layers as data moves from application to physical medium. Intermediate routers on the Internet provide the flexibility to move data across different network types (e.g., Ethernet to satellite communication). The receiving side then runs the network models in reverse to unwind the packet.



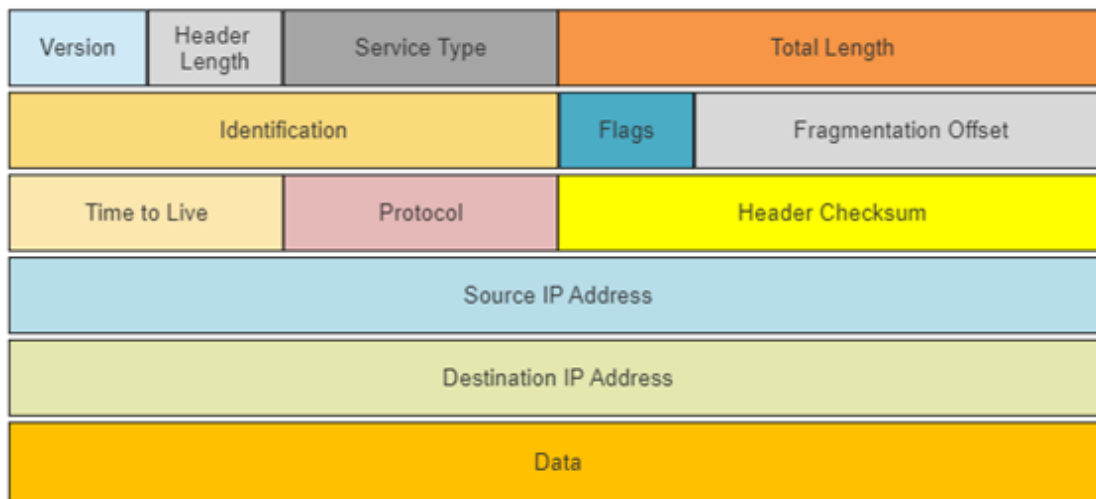
The TCP/IP stack is built from bottom to top starting with the physical layer. The physical layer is responsible for driving underlying hardware. This may be radios in the case of Wi-Fi or Ethernet transceivers in the case of wired connections. This layer simply shuttles data and converts it into proper radio signals or pulses on copper/optical cables.

The next layer is called the link layer and it resides directly on top of the physical layer. Often the link layer and physical layer are combined into the silicon hardware. The link layer is used to move packets from the Internet layer to the physical medium and silicon chipsets. The link layer adds *framing* parameters to the packet.

At this point in the network stack, we move closer to the actual TCP/IP software-based protocols and further away from the underlying hardware. This next layer is termed the Internet layer and manages the IP portion of the TCP/IP stack. This layer's main function is routing packets to the correct address. For example, a web site like www.manning.com resolves to an IP address of 104.18.34.76. This numeric representation of the text is the

primary routing and identity of Internet based traffic. The most important aspect of the IP layer is the addresses used follow a *next-hop* principle. That is, the data being sent has a destination address, but may take multiple and different routes to marshal the data across networks or the Internet at large. This process of migrating data across the Internet using multiple routes is naturally called routing. One can see how this routing works by seeing the format of an IP *datagram* in figure 6.3.

Figure 6.3 IP Packet Header



The fields of the IP packet are important. The header consists of various bit fields that help determine the version of protocol and various flags to control the data. It also consists of source and destination IP addresses. The source being the origination point of the packet and the destination is the receiving end. The fields' purposes are described as follows:

- **Version (4 bits):** This field identifies which version of the Internet Protocol is used (e.g., IPV4 and IPV6). IPV4, for example, is indicated if the bit field is set to 0100.
- **Internet Header Length (4 bits):** This is the header length, sometimes abbreviated IHL. It indicates how many 32-bit words are in the header portion of the datagram, but it doesn't include how long the data section is.
- **Service Type (8 bits):** Also termed Differentiated Services Code Point (DSCP), this is primarily used for Voice-Over-IP (VOIP) telephony and

data streaming. It helps specify the priority of certain transmitted data over others to help with quality of service.

- **Total Length (16 bits):** This is the total length of the entire datagram, including the data portion, and is measured in bytes (not bits). TCP/IP requires devices to be able to handle at minimum 576-byte datagrams. If the payload with data is larger, the packet will fragment into smaller units. The minimum size of any datagram is twenty bytes (which is simply an IP header without any data attached).
- **Identification (16 bits):** Used to uniquely identify various fragments in data transmissions. As mentioned, if the datagram is too large for a transmitter or receiver to manage, the datagram will be split into fragments. This field helps identify the various fragments and re-arrange them in proper order when they arrive.
- **Flags (3 bits):** This small field is used control fragmentation. -Bit-0 is reserved and must be set to 0. Bit-1 informs the system to not divide and fragment packets. Bit 2 implies more fragments are to be managed.
- **Fragmentation Offset (13 bits):** This represents the number of bytes in the data stream the fragment pertains to.
- **Time to Live (8 bits):** This is used to control how much time, as measured in seconds, that this particular packet is set to remain live on the Internet fabric. This field, abbreviated TTL, is necessary to prevent packets from endlessly floating and gathering on the Internet if they fail to reach their destination. This prevents overall network congestion and overall degradation of the Internet.
- **Protocol (8 bits):** This field is used to determine the type of data traffic that is being transmitted. Typically, this will be set to TCP but other types exist, like UDP transmissions.
- **Header Checksum (16 bits):** This field is used to verify the integrity of the header and assure that no fields are corrupt. If the header and checksum do not match, the packet is destroyed.
- **Source IP Address (32 bits):** This field is used to identify the origination of the message being transmitted.
- **Destination IP Address (32 bits):** This field is used to identify the destination of the message being transmitted. Remember that the destination address is where the data transmission completes. The data may be transmitted and propagate through various independent routers along the Internet.

The next level of the network stack is the transport layer. Here TCP (which stands for Transmission Control Protocol) resides. This software component provides communication between hosts and establishes or deconstructs virtual communication channels. This layer creates a *network port*. A network port is a semantic used to depict a virtual port that applications can use for communication. There are many standardized ports used by IT systems worldwide. For example, port 80 is most commonly used for web traffic, port 110 is used for email services, and port 123 is used for the Network Time Protocol.

An important part of the transport layer is it can manage connection orientated communication and connectionless communication. Connection-orientated communication uses TCP communication. This means that the data has robustness and reliability built into the protocol. Message transmitted using TCP will arrive in-order, with minimal errors, and duplicate packets/data will be discarded. Any transmissions that are lost are then resent by the transmitter. There are also congestion control and quality of service rules for connection-orientated traffic. Most traditional LAN and Internet based data uses TCP to ensure reliable and safe communication. With this level of reliability, comes at a cost of latency and throughput. Essentially each packet is acknowledged through various handshakes between the receiver and sender.

The other type of traffic that can be managed by the transport layer is connectionless communication. This includes UDP (User Datagram Protocol) which is a *best effort* protocol (sometimes called send and forget) also used widely in the Internet for streaming services like Netflix and voice data. In connectionless systems, data transmission is not necessarily reliable. Any lost packets or errors in transmission are handled by error detection and checksums. In UDP, data throughput and latency are more important than the integrity of data. In streaming applications like video, if a packet is lost out the millions or billions in flight, the application requiring that data will not fail, but there may be a small video glitch or audio pop that are typically smoothed over through software techniques.

The final layer is the actual application layer in software. This is what most users and system are exposed to such as web browsers, messaging programs, and anything that communicates across the Internet.

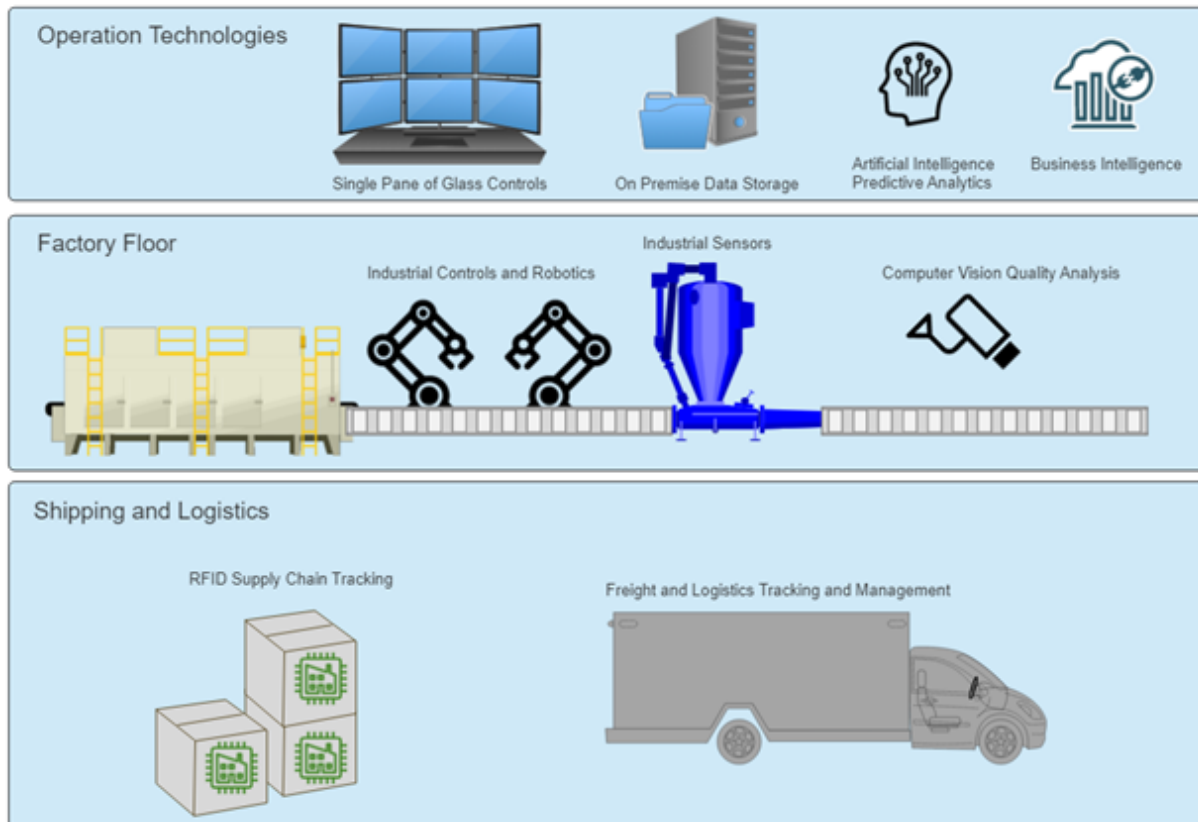
If we put all the network layers in hierarchy, we can see how the stack is built. Essentially all networking devices use this stack for communication. For TCP/IP traffic, the data migrates across the Internet through various routers, bridges and switches whose task is to inspect the data headers and marshal the message closer to the destination. This process of traversing many routers along the way is called network routing and the more hops along the way, the slower the overall transmission and larger latencies occur.

6.3 Industrial IOT Communications

Industrial automation and factory controls are one of the leading growth industries for IT. This what is driving the term *Industry 4.0*. It is projected that the digital transformation of manufacturing will have a 26% compounded annual growth rate or CAGR through 2026 and reach \$260B in market size according to Fortune Business Insights^[4].

The architecture that is used to control all the machine processes follows what is known as a SCADA architecture. It includes a breakdown of computers, networking, communication protocols, a user interface, and management functions. Typically, this equipment is used for operational technology (OT). That is different from information technology (IT) since IT is focused on the security and technical details of digital information. OT, on the other hand, is focused on the operation of physical processes and the physical machinery behind it. IT functions and personnel are focused on the office, datacenters, and related hardware like corporate laptops, network security, and identity management. OT works on the shop and factory floor. Think of operational technology as the control systems of a large nuclear plant with a myriad of controls and screens monitoring the status of critical devices.

Figure 6.4 Industry 4.0 Conceptualized Diagram



Industry 4.0 allows manufacturers to track the progress of each product being created at every stage of manufacturing including shipping and freight. This allows for just-in-time manufacturing and significantly reduces waste. Additionally, predictive analytics and business intelligence software allows the factory floor to prevent manufacturing failures before they occur. This is accomplished by monitoring the health of all machinery and inventory through sophisticated machine learning algorithms.

The format of data is important to edge use cases. They vary with application and industry. For example, in factory automation we use SCADA concepts to describe the connectivity of machines on a factory floor. Its purpose is a communication *language* used by factory machinery and automation. While these factory floor devices could use HTTP they generally don't as this legacy industry has widely adopted protocols established decades before HTTP was invented. As another example, IoT devices also communicate with the cloud, and they too need a model to send data across the internet. For these applications there needs to be a

lightweight and efficient protocol that easily transmits data from millions of nodes simultaneously.

Modern SCADA systems use edge computing and are connected to the cloud and internet. One can control an entire factory floor via a smartphone. State of the art SCADA architecture utilizes databases to store and archive all process data and have extensible and easy to manage user interfaces that build a single pane of glass of an entire manufacturing process. Yet, surprisingly, industrial automation has been slow to adopt new technologies as many shop floors rely on expensive and old machinery (sometimes called brownfield devices) that are amortized of many years or decades. We see legacy manufacturing tools being retrofitted with modern sensors and connectivity elements to allow them to be integrated with today's edge tools, analytic software, and machine learning.

6.4 The Language of Factory Machines

We covered some of the concepts of industrial protocols in chapter 2. Now we will go a bit deeper in understanding how edge computing works in industrial settings. Part of the enablement of Industry 4.0 is device connectivity and industrial protocols. In industries such as factory automation, additive manufacturing, or energy and oil production the connectivity with machinery takes the form of protocols like ModBus, BACnet, and Profibus. ModBus is a most common and widely used factory automation protocol. It was originally designed by Modicon (now part of the industrial automation giant Schneider Electric) in 1979 to support programmable logic controllers (PLC). A single medium size manufacturing site may include thousands of sensors, motor drivers, robots, and alarms. What was needed is a method to communicate with all these devices in a standard manner.

We should first briefly describe what PLCs are and what they are intended to do. A PLC is an electronic device used to control machinery (turbines, robots, pick and place machines, sirens, lights, conveyor belts, etc). PLCs deal with machinery in *hard-real-time*. That means they must gather input and make corrections to an automated device within a strict amount of time. Failure to meet the time deadline can result in catastrophic failure. Thus,

PLC's perform mission critical functions on the shop floor. A PLC is usually a self-contained device with tens to thousands of IO connections to various sensors and machines. It uses a CPU, memory, and provides a communication interface. Now with internet connectivity, these legacy PLCs have more in common with edge computers. One difference with the traditional form factor of computer and servers is that PLCs exist on the shop floor in very harsh conditions. They must withstand dust, debris, humidity, as well as shock and vibration. These devices appear as inconspicuous, tightly sealed grey sheet metal or hardened plastic boxes capable of withstanding the harshest elements and do not look like traditional IT and server electronics from the outside.

Communication with PLCs has been performed using low-speed serial interfaces such as RS232 or RS485. This allows it to connect to various sensors, actuators, and motors. Many manufacturers have migrated from custom communication protocols to standard protocols with the advent of ModBus in the 1970s as a means of standardizing on a protocol to allow for a wide range of device connectivity. There are two versions of ModBus used in the industry today:

- **ModBus RTU:** This allows up to 32 devices to be chained together using serial cables; however, only one device can be communicating with a server at one time. Addressing a device can be performed by requesting client to return (or alter) a value held at a specific memory address to the server.
- **ModBus TCP (aka ModBus EtherNet):** This newer variant of ModBus uses standard network medium such as Ethernet cables and the TCP/IP protocol for general networking. In this model, there is no physical limit to the number of devices and multiple devices can communicate simultaneously with a massive amount of bandwidth. ModBus TCP connected using Cat6 gigabit Ethernet cables could theoretically respond to over 30 million requests per second.

ModBus TCP can be packaged over internet connections while ModBus RTU would need additional hardware and software to do so. Today the ModBus protocol is managed by the ModBus organization:

<https://modbus.org>.

There are certainly other protocols and ModBus derivatives worth exploring such as Profinet and BacNet. While all can communicate over Ethernet and all are just portions of the application layer in the OSI network stack, there are differences. Profinet was developed in 2003 and is tailored for communication in real-time controls. BACNet was developed in 1987 and almost immediately adopted by the HVAC and control industry. BACNet uses predefined data types like door access, electrical load value, and lighting group number. This allows for relatively easy integration if the data types are supported by your edge project, but inflexible for new types of machinery.

	ModBus	Profinet	BACNet
Typical Usage	PLC control, factory automation, manufacturing, energy production	Safety critical situations and hard real-time control systems. Hazardous work areas.	Tank level sensing, building automation, HVAC
Network Interface	Serial, Ethernet	Ethernet	Ethernet and LAN
Communication Protocol	RTU and TCP/IP Client - Server	UDP Provider – Consumer Multiple masters	TCP/IP, Zigbee
Costs	Low	Low	

Standard Body	IEC 61158	Profibus and Profinet International (Germany), IEC 61158	ANSI/ASHRAE, ISO 16484-5
Proprietary	No	Yes, Devices require certification through Profinet International.	No
Pros	Ubiquitous industrial protocol leader. Many supporting devices and peripherals. Open and royalty free. Best for communication to existing SCADA computing systems.	Great for hard real-time control that requires <1ms accuracy. Robust to noise and communication loss. Allows for redundancy in hardware and equipment. Best diagnostics and management services.	Good balance of cost, scalability, and performance.
Cons	Limited data types and limits to data size. Security model not strong.	Limited usage. Not all Ethernet cabling can work with Profinet.	Limited adoption. Limited number of legacy devices that can be connected. Limited data types.

Suffice it to say, these protocols for industrial, energy, and factory automation have evolved differently than traditional IT systems. But given the importance and growth projections for Industry 4.0 it is important to understand their role for edge computing.

While these protocols have long been established in industrial environments, communication for consumer and commercial use cases have adopted a different set of protocols.

6.5 Message-Orientated, Stream-Orientated, and RESTful Protocols

When deciding how an edge device will communicate to the cloud, it is imperative to understand the type of data in transit. There are two types of methods commonly used to transport data over the Internet or between machines: message-oriented and stream-oriented. Message-oriented protocols can be thought of as transmitting data in discrete chunks or blobs. Stream-oriented systems will transmit continuously until all the data has been transmitted or the session ends.

Message-oriented protocols are persistent, meaning that data that is transmitted will be stored (or buffered) until the data is delivered successfully. This provides an assurance that data will make every effort to ensure reliable transmission. Think of email as an example. When you send an email message across the internet, the data will follow many different and independent paths to the destination. If a hop on the internet fails, the system will retry communication or find alternate paths to deliver the message. TCP was built for this purpose.

Additionally, message-oriented protocols can be asynchronous or synchronous in behavior. What does that mean? If a message-oriented system is synchronous, the transmitter will pause all new transmissions until it has received an acknowledgement that the last transmission arrives correctly at the receiver. Alternatively, the system can be asynchronous, and the transmitter will start a new message before the last was acknowledged.

While this sounds like a recipe for failure, the TCP protocol can identify if a packet was not received correctly, and it will attempt to resend it that packet even if it is in the wrong order.

Stream-orientated communication however is tailored for performance and low latency. A stream-orientated communication does not provide persistence like message-orientated systems, rather data may very well be lost permanently in transit. A good example of this is the real-time video that Netflix and other content delivery networks use. Netflix servers and viewers at home with an edge set-top box will stream video and music. If a packet is lost, there may be a slight aberration or lag in viewing performance, but the transmission will continue to the next frame. Stream protocols typically provide quality-of-service (QOS) parameters to adjust the stream on the fly. For example, if the transmission is noisy and many packets are lost, the quality of the viewer degrades. QOS services will then attempt to use a higher video compression rate to adjust to the noise and provide a better viewing experience.

Message-orientated protocols require a *broker* as part of the network architecture whereas streaming protocols do not. The broker assists with storing, forwarding, routing, and even transforming messages in flight.

The final model we explore is *RESTful* communication systems. RESTful systems are nuanced in such a way that they shouldn't be considered a replacement or alternative to message-orientated or stream-orientated communications nor is it a protocol entirely. It is more of an API (application programming interface). Here, servers are required to maintain the state of a shared resource. The state of the resources is never shared, only data is exchanged. A classic example of a RESTful model is the Hypertext Transfer Protocol (HTTP) which is the foundation of the World Wide Web. HTTP uses commands such as GET, PUT, POST, and DELETE to communicate between a client and the server. The service is identified with a Universal Resource Identifier (URI). A typical web site like www.manning.com one part of a URI. The web site name is called the *Authority* field of a URI, but other fields can be port numbers, schemes, pathnames, and even queries.

A fundamental difference between RESTful protocols and message-orientated protocols is that RESTful systems require the client device (e.g., a

web browser) to manage errors in transmission or even if the server fails. The other difference is that message-orientated systems require a broker to coordinated producers and consumers of information and route data correctly. RESTful systems do not need any middleman or broker, they have a strict client-server relationship.

6.6 The Most Prevalent Edge Communication Standard: MQTT

We have explored the industrial protocols and use cases; however, there are a myriad of other IoT uses that need an efficient means for communication between the cloud and edge. There may be a single device that needs to transmit temperature data to a public cloud, or there could be millions on smart thermostats in consumer homes frequently transmitting data to the cloud. A protocol needed to be used for the advent of IoT that is smaller and more efficient than HTTP and can extend to millions of nodes transmitting simultaneously. This is where protocols such as MQTT, COaP, and occasionally AMPQ fit into the picture. Each of these are application layer protocols, thus they sit in layer 7 of the OSI model and are all based on TCP/IP.

MQTT stands for Message Queue Telemetry Transport. MQTT has roots with IBM in 1993 where a technology named IBM WebSphere Message Queue was devised for distributed mainframes to communicate with each other, specifically for industries like oil and gas processing. It was proprietary to IBM use in traditional IT and mainframe designs until being released in 2010 as a royalty free protocol. At the time it was accepted by the non-profit IT standards body, the OASIS consortium which gave it it's official name and release version 3.1 to the world. Now every major cloud provider supports MQTT as the main protocol to communicate from the edge to the cloud (and back). The fact that MQTT was developed 20 years before cloud computing existed and 30 years before edge computing became pervasive is an amazing testament to the technology. Amazon, Google and Microsoft all support MQTT version 5 as the principal method to connect millions of sensors, edge computers, and IOT devices. It is the fabric the makes edge computing work. Ford Motor Company, for example, uses MQTT as the protocol to communicate with their new line up of internet

connected vehicles. MQTT allows Ford to retrieve vehicle telemetry and maintenance data. It also allows Ford to send remote commands to vehicles anywhere on the planet such as unlocking a door if the owner has locked them inside.

MQTT is a protocol intended to satisfy the following constraints:

- **Lightweight and efficient.** It is intended for small data payloads and capable of running on the smallest and cheapest embedded devices in the field. It should be bandwidth efficient to improve performance, latency and reduce network SLA charges.
- **Bi-directional.** MQTT should support edge to cloud data movement as well as cloud to edge. An example is how Ford uses MQTT to retrieve vehicular health data and to send commands to the car.
- **Scalable.** This is perhaps one of the most significant advantages of MQTT. It has been scaled to millions of IOT and edge systems across the globe. Ford, for example, plans to scale this to millions of vehicles all communicating simultaneously.
- **Reliable.** The protocol should support various levels of reliability, from least redundant to very high reliability in the event of total system failure. This is especially true for some of the networks and communications medium we studied where connectivity cannot be guaranteed.
- **Secure.** Designed with security built in. Messages will be encrypted, and devices will use widely adopted standards for authentication and identity.

MQTT is a *publish-subscribe* type of protocol sometimes called *pub-sub*. This is radically different than regular web traffic, ModBus, or other protocols. Typically, when a device like a personal computer wants to communicate to a server it reaches across a network to a well-known address (e.g. www.microsoft.com or 192.168.0.202) and to a network port (e.g. port 80 for web sites). Data is transmitted from the server to the PC and each packet transferred is acknowledged as received. MQTT does no such thing.

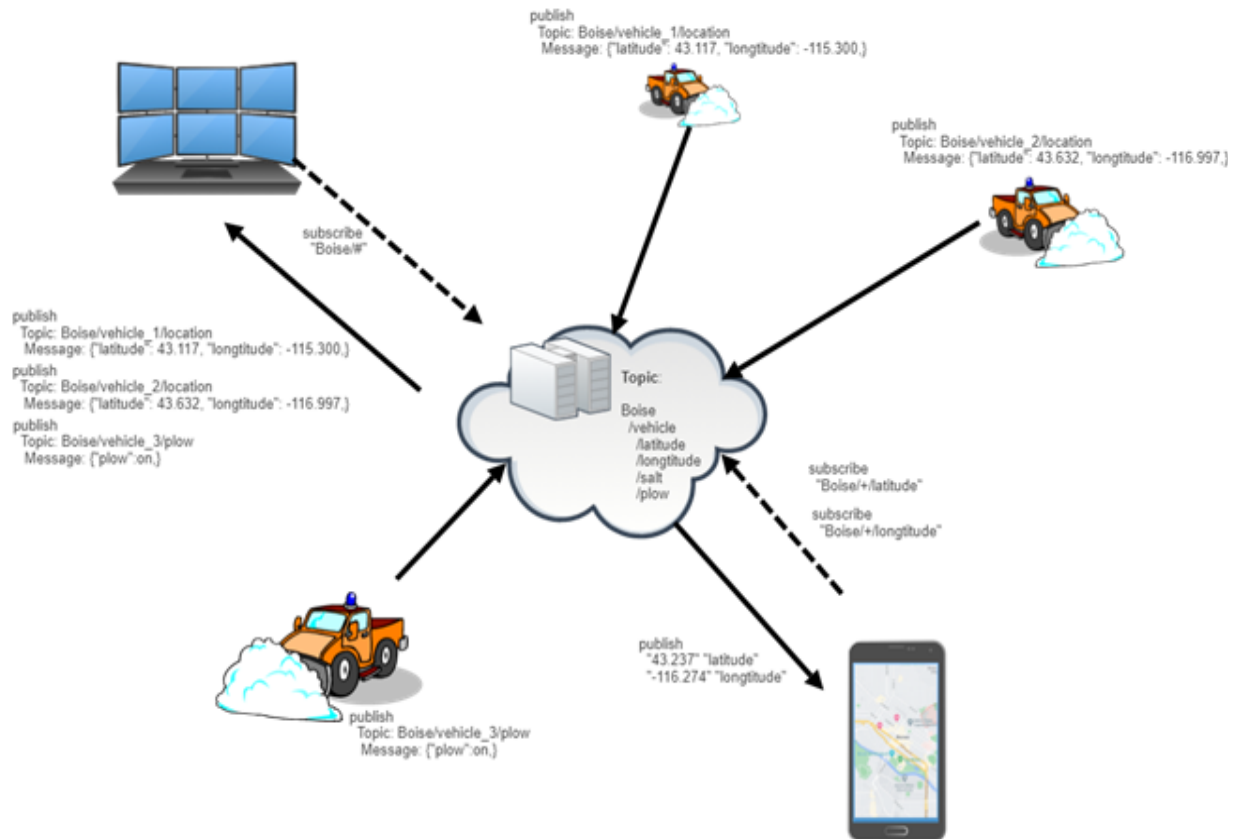
When you use MQTT, a client device transmitting data is called a publisher and the client receiving data is the subscriber. Neither a publisher nor subscriber is required to identify itself with a physical identity like an IP

address (e.g., 192.168.0.202). The physical identity may not even be known to a simple IoT device. A message can also be published and received by whomever else subscribes to that event. This allows messages to be time-invariant which is important for protocols like LoRa where a device may publish or read from a subscribed topic after long periods of time. As we have learned in other parts of this book, there is no single definition or technology to define edge computing, rather you must understand all the variants.

A MQTT broker is a software component that governs connections. A typical public cloud service can ingest millions of messages per second using their MQTT brokers. The edge devices that communicate with the broker are called MQTT clients. Clients will subscribe to what are termed *topics or topic branch*. A client may also publish to a topic branch. In this way a client can either produce data or consume data. A topic is simply a method to identify the fields or data elements of interest to subscribers.

A system I designed used an edge computer that tracked snowplows in a municipality. Each plow was fitted with an edge computer. The computer on board would monitor the GPS location of the vehicle, track salt disbursement, and if the snowplow is up or down. The edge computer would gather data and then transmit it to a receiving system via cellular or LoRa communication. A MQTT broker running in the cloud would manage all the data in various topic hierarchies and anyone could see in real-time where the vehicles are and know which road have been plowed and when they had been last plowed. Such a system could look like this (figure 6.4):

Figure 6.5 MQTT Architecture. Note the pub-sub model where different devices actively publish data while subscribers receive broadcasts.



In the example and illustration, the topic branch managed by the central cloud broker has the appearance of a file structure by intention. Topics are managed by hierarchy such as:

```
Boise/vehicle_1
    /latitude
    /longitude
    /salt-on-off
    /plow-on-off
```

Each plow would contact the broker and publish its status. Other subscribers such as a central manager overseeing the municipal plows would subscribe to the branch and receive data in real-time. In another case, a citizen may have a smartphone app that also subscribes to just the latitude and longitude information of the plows and plot them on a map to see which roads have been cleared for a commute.

Edge systems must be resilient to loss of communication. MQTT offers many techniques to ensure a message will arrive at the broker. First, if a new

client subscribes with the broker (say another smartphone user), messages are not retained at the broker, only the last message is stored all others have been assumed to have been transmitted to subscribers listening. There are methods of data assurance are called Quality of Service (QOS) for MQTT. There are three levels of QOS:

- **QOS Level 0:** This is the default level of quality. Data is not assured reliable communication by default. Data will be transmitted using *fire-and-forget*. The receiver will not acknowledge transmissions either. This is useful for data that can be lost where the system will be adversely affected.
- **QOS Level 1:** This level requires the receiver of data to acknowledge successful reception back to the transmitter at least one time.
- **QOS Level 2:** The highest level of data integrity requires that the sender and receiver of data both acknowledge transmission. The receiver of data will send an acknowledgement back to the sender, then the sender will send a subsequent acknowledgement back to the receiver. This will generate considerable traffic and slow transmissions but may be required for mission-critical use cases.

Another function of MQTT is called *Last Will and Testament*. While it sounds ominous, it simply allows a client that publishes data to inform the broker of a message to send to all subscribers if a connection is lost. The broken is then responsible for broadcasting the “last will and testament” on behalf of the client. In our example, if a client snowplow loses cellular connectivity (e.g., while traveling in a tunnel) the broker will broadcast a message such as *Lost connection* to all clients.

6.7 Alternative protocols: CoAP and AMPQ

There are many other protocols analogous to MQTT worth mentioning that are common for edge communication. CoAP is a particular common protocol in edge computing. CoAP stands for Constrained Application Protocol and was created by a working group in 2014. It is mostly intended as a communication protocol for deeply embedded devices that are processor, memory, and bandwidth constrained. CoAP is similar to the internet Web protocol HTTP in structure. Because of this, it is a RESTful

protocol and requires no broker like MQTT. Because it is RESTful it doesn't require a broker but does have a server to manage connections. The CoAP server is also termed the *origin server*. Think of it as a very lightweight HTTP replacement. While it doesn't replace HTTP and won't allow you to view web sites, it allows for easy addressing in much the same way you address a web site using a URI syntax:

```
coap://host[:port]/[path][?query]
```

CoAP defines different roles for the actors in a CoAP network:

- Server: The destination endpoint. Servers are also originators of a response.
- Client: The initiator of a request. Clients also are the destination of responses.
- Origin Server: The server where a resource resides.
- Observer: A client node can connect to a resource and receive updates when that resource change. This is like MQTT subscriptions.

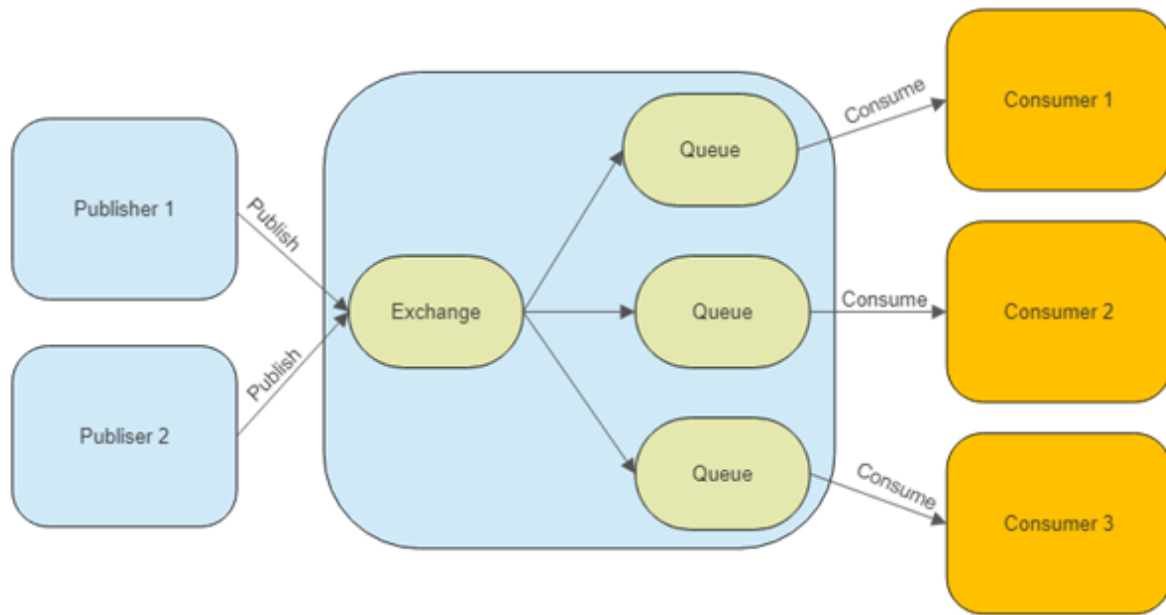
Two edge nodes may exchange data efficiently using CoAP between themselves. Likewise, a CoAP edge node may push data to a CoAP capable server in the cloud. CoAP also has facilities to ensure reliability. These are in the form of confirmable (CON) and non-confirmable (NON) requests. Confirmable requests must be acknowledged by the receiver within a time limit. If the acknowledgement isn't received, the data is resent. Non-confirmable messages are simply broadcast in a *fire-and-forget* model. There is no assurance the message will be received in a non-confirmable request. Since CoAP is based on UDP and not TCP, messages can be lost in transmission and there are no facilities to recover the data automatically like TCP enables. Therefore, if data integrity is required, using CON messages in CoAP will be necessary.

AMPQ is yet another alternative protocol used from edge to the cloud. Owing its creation to engineers at J.P.Morgan Chase in London in 2003, it was originally used for financial transactions. Today it is an open standard used by Cisco, Red Hat, and VMWare. More details on AMPQ can be found at www.ampq.org.

Being a message-orientated protocol provides AMPQ a degree of resilience the transmitted data will be delivered with some form of guarantee. Like the other protocols the data in flight can be encrypted for security. A *frame* is the basic means of communication in AMPQ. A frame can perform tasks such as opening a connection between a client and host, transfer data, close connections, and so on. AMPQ is like MQTTT with a publish-subscribe type model. A central server manages the various producers and consumers of data and allows them to secure subscribe to the service. The server is also termed a *broker* which binds various producers and consumers of data to a set of queues. These queues are buffers of data that stream into the server and are identified using keywords. For example, an AMPQ server that is monitoring traffic for various roads in a metropolitan area. Each road could be its own queue receiving data from roadway sensors in real-time. In this case, each road becomes a keyword to identify it. Someone interested in certain road condition would receive messages for only those roads by keyword. AMPQ is also more flexible than MQTT with regard to how messages are routed from publisher creating data to consumers reading and interpreting data. Each reader could be associated with their own queue, this allows for bespoke types of routing patterns.

AMPQ relies on what is called an *exchange*. The exchange is the target of the message publish operation, which then routes messages to queues. When a consumer receives data from the queue they are listening to, all data can arrive in synchronous order. Although asynchronous communication can also be enabled. For reference, MQTT is asynchronous by design. The following figure illustrates the AMPQ architecture

Figure 6.6 AMPQ Queue Based Architecture



6.8 Protocol Comparison

We have briefly explored some of the top layer protocols for communication between edge computers and the cloud. It is appropriate to contrast them, since choosing the wrong protocol can lead to inefficient designs.

	MQTT	CoAP	AMPQ
Network Model	Message-Orientated	RESTful	Message-Orientated
Computational Load	Medium	Low	High
Security	Payload authentication	None (Requires DTLS)	TLS based Encryption: Yes

	Payload encryption (Requires TLS/SSL)		Authentication: Yes
Protocol Overhead	Medium	Low	High
Quality of Service Control	Yes	Partial (using CON message)	Yes
TCP or UDP	TCP	UDP	TCP
Maximum Message Size	256MB	64KB	2GB (theoretical)

The computational load is the amount of processing and data formatting required on the device. This is important when considering some edge nodes may be very small embedded systems requiring power by battery or renewable resource like solar. In this case, AMPQ requires more resources and computational power while CoAP s one of the most efficient.

Security is managed differently between these protocols. MQTT and CoAP have no built-in security and require external packages to provide both authentication services and data encryption. MQTT being a TCP protocol requires SSL or TLS services. The security chapter will detail SSL and TLS.

Quality of Service is another important parameter in choosing the correct edge protocol. QOS. CoAP manages QOS and data integrity using a simple confirmable (CON) message. CoAP is not as robust and configurable for quality of service as MQTT or AMPQ.

Now that we have seen different protocols, the designer of an edge system should choose wisely to understand the type of data being transmitted and also to understand what level of quality and security is needed in the application. Another important factor in determining the correct protocol is whether the cloud backend supports the protocol and which version is supported.

6.9 Summary

- The OSI model of network stacks illustrates how all modern networks are built using an abstraction and layering approach.
- TCP/IP is the basis of all modern networking. It is a layered architecture from physical hardware to the application layer. TCP being connection-orientated allows for data to route through various paths in the network like the Internet with assurance of data integrity.
- UDP being connectionless does not embed data reliability within the protocol.
- Industrial devices have a long heritage of bespoke protocols for machinery communication. Most notably is ModBus, which can be TCP based to connect various machinery like PLCs to other nodes and edge computers.
- There are three types of interconnectivity models to consider for edge deployment: message-orientated, stream-orientated, and RESTful. Message orientated requires a broker to manage publishers and subscribers of data while RESTful systems are analogous to the Web with a strict client-server model.
- MQTT is by far the most used edge communication protocol. Being message-orientated, it requires a broker to handle publishers and subscribers to data or events.
- CoAP is a very light-weight RESTful edge protocol that is UDP based. It is most useful in power constrained devices but does not allow for the configurability of MQTT or AMPQ nor the same levels of quality of service.
- AMPQ is a message-orientated protocol used in many financial transactions and mission critical applications. It is a computationally complex and demanding protocol requiring more edge resources than others.

^[4] “Industry 4.0 Market Size”, Fortune Business Insights, 2020
<https://www.fortunebusinessinsights.com/industry-4-0-market-102375>

7 Making the Edge work through AI

This chapter covers

- How to effectively partition work on edge and cloud systems.
- Situational awareness applications.
- Machine Learning and AI applications on edge systems.
- Cloud training and edge inference.
- Time-series analysis.
- Federated computing – blending many edge systems into AI and decision systems.

So far, we have explored the hardware and software frameworks for edge computing. We have not detailed exactly what edge systems do, however. We should ask ourselves, “how does an edge device communicate with the cloud (or should it)?” We should think about what actions does an edge computer make. Logic and applications running on edge systems can be simple utilities and drivers for hardware and sensors simply to rely data back to the cloud. At the other extreme, we even have edge systems make decisions themselves using machine learning. This chapter details how edge systems interact with cloud systems. We will examine what happens in the cloud when edge components are added to the mix. We also will talk about cloud usage in machine learning and predictive analytics.

7.1 The Purpose for Clouds with Edge Computing

In many cases the edge is an extension of the cloud. Typically, a public cloud acts as a central control point of control. Public clouds offer massive global scale, high performance, and significant storage capabilities. This has allowed enterprises to run their applications such as enterprise resource planning (*ERP*) systems, databases, human resource management (*HRM*) platforms in the cloud rather than supporting them locally. Cloud

infrastructure can be considered centralized (since most clouds are in regional areas and housed in large datacenters). The edge is more decentralized providing a locus of control closer to where data is generated. A cloud can simply lease bare hardware to a customer. This is sometimes called Infrastructure as a Service (*IaaS*). Other customers just want to run an application in the cloud such as a global website. This service is sometimes called Software as a Service (*SaaS*). However, while these types of services are perfect for applications like hosting a global website or archiving a company's data, it is far from the source of data generation. This is where the edge comes to play.

The relationship between an edge computer and a cloud service is unique. An edge computer will deal with data locally close to where data is generated or served to customers. In our earlier examples these sources would be handling data from sewer monitors to streaming content to Netflix users. The cloud systems act as an overall orchestrator. It may manage provisioning of software and patches to edge nodes, control security and login credentials, and feed on data being filtered by the edge. A cloud system is not required to orchestrate all edge systems. However, as a system scales, a cloud becomes critical to efficiently designing a production system.

So, what should run on the cloud and what should run on the edge? How do you divide or partition the architecture appropriately? This, like most engineering problems, is about optimizing constraints. These are the typical constraints you will deal with an edge system:

- 1. How much data is generated at each edge node?**

- a. Is the edge node collecting massive amounts of unstructured data like video from several surveillance cameras? If so, that can be a significant amount of data to marshal to the cloud. The edge should preprocess the data.
- b. Does the edge have enough local storage for a worst-case load? Perhaps the data is temporal and doesn't need to be stored or for privacy reasons can't be stored. For example, an edge system monitoring a home-care patient has a litany of privacy and HIPPA laws that regulate its content and transmission. Preprocessing data may include computer vision algorithms and machine learning

classifiers to detect certain events. In this case look for a positive hit of an event of interest and then transport a historical slice of data to the cloud for further processing.

2. What are the service costs of transporting data to the cloud?

Service charges (or SLAs) imposed by a communications provider for broadband service have to be taken into consideration. Whether it's a fiber optic connection, a 5G cellular service, or LoRaWAN, data plans come in a variety of forms with monthly data caps and rates for various bandwidth. Data plans can be shared by multiple edge devices or each edge device may have its own unique plan. Verizon for example has a variety of shared plans for IOT devices that range from \$1/month for 50 KB of total data to \$80/month for 10GB of shared data. If your edge system must scale, these SLA costs can become a major source of expense. Many edge proof-of-concepts have failed because the overall charges of data were overlooked, and the system couldn't scale.

3. What are the costs of cloud use? Public clouds are not free. Expect charges based on how many devices are connected to the cloud, how many message requests are sent to cloud ingestion engines, and how much processing and storage of data are needed. Often cloud systems will provide a customer with an initial trial period where data and storage is metered to some level and are essentially free. This is useful for evaluating cloud provider's services, you should be aware of how the cloud will charge you. Some providers charge you per data transferred to their system. Some charge for data that is stored or archived. Some will charge based on what services you use in the cloud. Expect a combination of tiered charges.

4. Is the system mission critical? Perhaps the edge system is managing real-time factory automation or servicing palliative in-home health care. In these situations, the system needs to be autonomous and not reliant on the cloud or any WAN connectivity to be functional. Imagine a situation where edge systems monitored vehicular engines but required data to be processed in the cloud to be functional. While 4G and 5G cellular may provide robust connectivity, you can't provide any guarantee it will always work, such as when you drive through a tunnel.

5. **Can your system afford latency?** One of the primary benefits of the edge is it resolves the device to cloud latency issue. We have already studied how edge to cloud latency impacts a system in previous chapters. Think about autonomous driving vehicles, for example. Imagine if all decisions on driving and path-correction required data to be sent to the cloud, processed, and sent back to the vehicle. A vehicle driving 60 MPH on a freeway will travel 88 feet in one second. If the cloud takes 100ms to transmit vehicular data, process it, and send course correction back, that car will have travelled 8.8 feet before responding. The average braking distance is 180 feet for a car travelling that fast. This clearly will not work and is life threatening. Autonomous driving systems measure a litany of video, lidar, and GPS information thousands of times per second.
6. **How will you manage, secure and scale edge nodes?** If you have thousands of edge systems deployed, how will you manage their health, capture logs, update firmware, and control access? Is the system truly decentralized and independent, or do you need a central control? Often you want a central facility to manage the edge, control access, and present that “single pane of glass”. Clouds provide an excellent central management for many edge nodes.

This thought exercise can help establish what role the edge should have versus the cloud. The two often are tied together with the edge acting as a “local branch manager” and the cloud acting as a “corporate executive”.

7.2 Work on the Cloud and the Edge

Edge computing may trickle data to the cloud. The cloud system will provide service for data ingestion from edge nodes. It will manage deployment and software management to the edge. It may allow for container-based deployment of software. It will handle identity and access control. More advanced cloud systems will also be responsible for data archival of all the data streaming from the edge.

In the cloud, data may be ingested by millions of sources. For example, take the popular Smart doorbell market that is wildly popular in consumer homes. Anytime a presence is detected within view of a doorbell, video data

is recorded and streamed via consumer Wi-Fi to awaiting ingestion engines in the cloud. There data is archived and visible to users with the correct credentials. Ring doorbells, for example generate enormous amount of data when we consider over 10 million Ring doorbells have been sold. Only hyperscale cloud systems that can auto-scale and have tremendous storage can manage such a task.

Another service a cloud routinely performs is management and visibility of all the edge nodes, what they are controlling, and providing alerts to issues that arise. Operation Technologists (OT) traditionally like to see a “single pane of glass” or a single user interface that conveys as much information as possible. For example, an edge system that is tracking snowplows and attempting to efficiently route them during a snowstorm will ingest GPS coordinates, plow fuel capacity, salt capacity, and upcoming weather radar all overlayed on a municipal map of a city that shows which roads have been plowed and when. A single pane of glass dashboard is a common requirement for nearly all OT and edge use cases.

7.3 Edge Workloads

After establishing what role the cloud provides versus the edge, we will now talk about a few relevant services and functions for edge computers. An edge device may perform functions like serving content to video streams or it may gather data from sensors (a process known as *sensor fusion*). For example, in a sensor fusion application, a system tries to analyze road and traffic conditions in a smart city application. Here sensors involving roadside cameras would be the most natural way to determine if there are traffic congestion situations. However, video combined with roadside noise sensors, electromagnetic sensors built into roadways, and information collected by smart cards relayed to central servers can all be combined and their disparate data aligned to make better judgments of the traffic situation. All this data is managed and collected by various edge systems all collecting and transmitting different data in different time domains to a central service.

As we have seen, edge systems can be quite simple, e.g., proxying data from IOT sensors to the cloud for storage and processing. Here the edge

system is a communication bridge or gateway. To understand this better we will explore edge patterns.

7.3.1 Edge Patterns

Used as a gateway, an edge device can perform specific functions and have different roles based on the use case. Gateways can have the role of *transparent gateway* or *translation gateways*.

In a transparent gateway, the edge device bridges IoT sensors and systems to the cloud as if there is no edge service between the two. The edge system operates in a manner that simply performs a connection service. This places some burden on the end IoT device. It must have the logic, networking stacks, and processing power to communicate directly to the cloud as if no edge device were intermediary. The IoT device must be able to provision itself correctly, add whatever security measures (like certificate management) are necessary and have all the software necessary to communicate with the cloud (e.g. MQTT or AMPQ).

In a translation gateway, much less demand is placed on the IoT device. This is typical when the IoT device is physically small, low powered, and use non-IP based networking standards like Bluetooth. Essentially the IoT device alone cannot communicate directly with the cloud. This is where edge systems operate in a translation manner. The edge system will manage, secure, and provision all IoT devices that have a relationship with that particular edge node but operates in an opaque manner obscuring the identities of the underlying IoT devices from the cloud. To communicate between the IoT device and the cloud data will be transformed by the edge node. Transformations include:

- **Protocol translation:** In this model, the edge will communicate to IoT devices using one protocol (e.g. Bluetooth), gather and manage data flowing in and present the data to the cloud using standard protocols like MQTT across the Internet. A user cannot interact directly with the IoT device attached in those model.
- **Identity translation:** In this model, the edge device will create and assign a unique identity to each attached IoT device. The IoT device

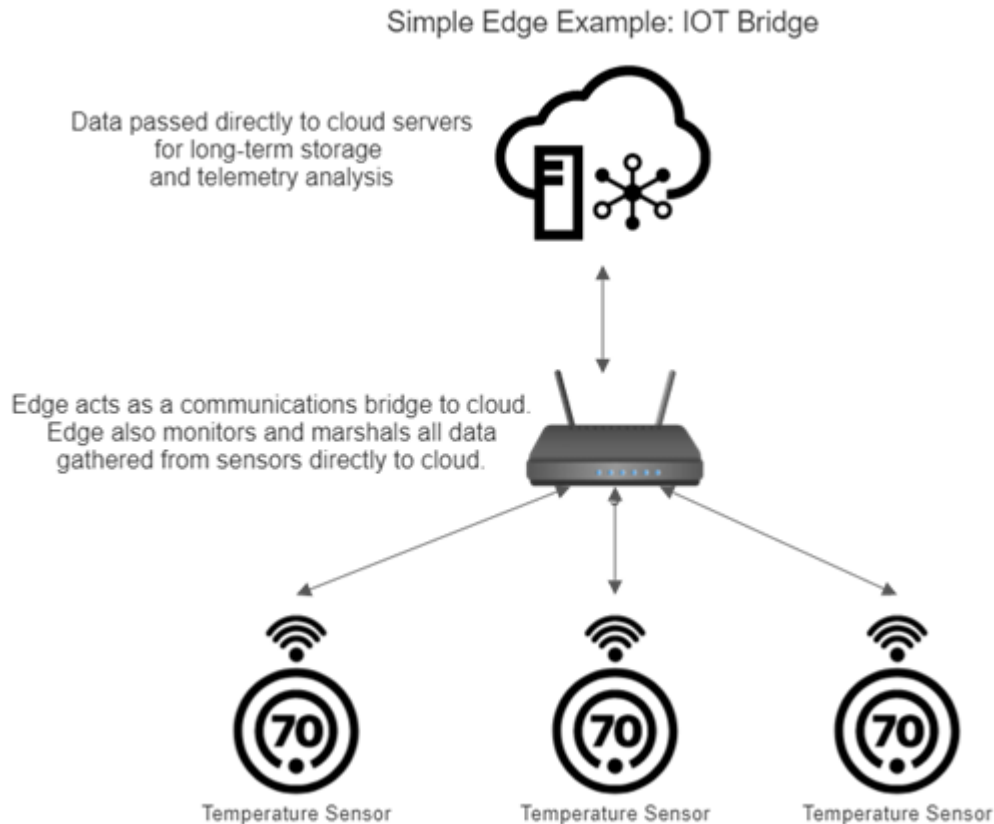
still requires protocol translation (e.g. standard IP to Bluetooth) but allows a user to have direct control and interaction with the IoT device through the edge gateway.

7.4 Example Workload Organization

In more complex scenarios, the edge computer performs mission-critical and real-time tasks where the added latency of moving data to the cloud and providing actionable response may not work. In these scenarios, the edge must perform the tasks locally to be useful.

The first example is a simple use case where the edge performs communications bridging for remote sensors and then delivers data to the cloud. In this case, latency is not an issue. Most of the heavy lifting will be performed on the cloud. Since the cloud will have gathered a significant amount of data it can also perform advanced analysis and predictive analytics on the collection of data.

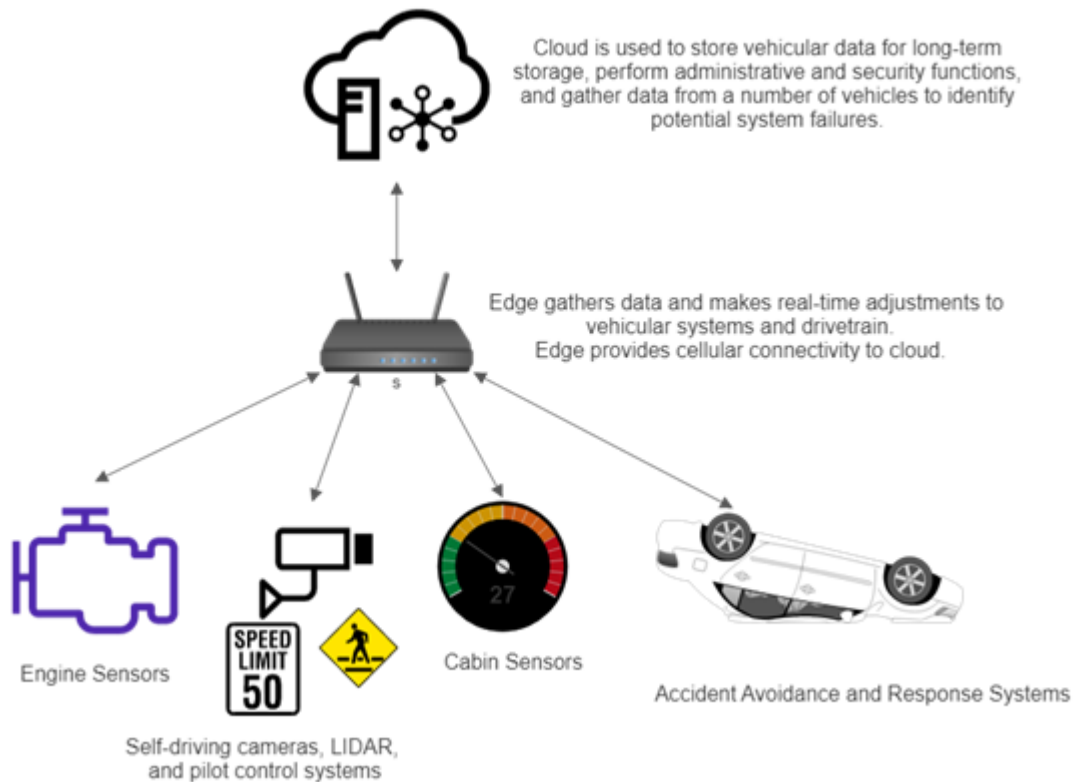
Figure 7.1 A simple mock-up use case for edge applications. Here the edge node acts as a communication bridge whose sole purpose is to marshal data from far edge sensors to cloud for archive and analysis.



The second use case provides more computational resources to the edge node. While still connected to the cloud, the partitioning of software is quite different than the first scenario. Here we are using edge computing in a vehicular system. Since the vehicle may have limited or no connectivity at times but must perform life-critical and real-time functions for safety, the system must operate autonomously with or without cloud connectivity. Most self-driving automobiles today rely on extremely high performance and costly edge systems with significant amounts of computational power, storage capacity, and fault tolerance. Most self-driving systems also must perform rigorously in harsh environments (e.g. heat, cold, vibrations, etc.). The partitioning of services favors the edge in this case, where the cloud is used for long term storage, management, firmware updates, and so on.

Figure 7.2 A complex edge use case. Here software and services are partitioned with most of the computational load on the edge system. This is necessary for latency intolerant, real-time, and life-safety situations. Vehicular systems typically will utilize exceptionally powerful edge computers.

Complex Edge Example: Self-Driving and Real-time Vehicular Analysis



The following subsections will illustrate some of the common functionality and services that are used in commercial edge applications.

7.4.1 Situational Awareness Applications

Situational awareness systems are used in places to gather data of a system and respond to events. Some examples of situational awareness are combat systems that gather data on troop movement, weather, terrain, and enemy intelligence. Here is a situational edge system in action:

A fighter-jet on reconnaissance scanning a valley to gather information and sees a vehicle typically used by friendly forces. The edge system on the jet runs a machine learning inference algorithm on the video feed and receives a notification on the heads-up display of an augmented-reality (AR) headset that this vehicle belongs to friendly forces.

However, the pilot knows that local forces had been overrun and adversaries are behind the wheel of this vehicle. At this point, the pilot adds new information to the situational awareness system and the edge machine corrects the artificial intelligence's inference recommendation. This new information is then propagated securely to other squad mates, autonomous drones, and to central command to adjust for the new situation on the ground. When the warfighter above corrects the AI output, for example, all warfighters and devices receive that correction and maintain cohesion.

This is one example of situational awareness algorithms operating on the edge. There are other examples outside of military usage such as edge system monitoring subways with real-time video analytics to determine potential terrorist threats. Another example are systems that will track all employees in a public facility like a hotel or airport and respond to emergencies or disaster situations by communicating to all employees to respond based on their physical location and role. Situational awareness has been used in monitoring environmental health in large municipalities and responding to events like hurricanes and earthquakes.

7.4.2 Machine Learning for the Edge

Machine learning (*ML*) is broad and complicated topic and has demonstrated huge advances for organizations and people using ML. It is estimate that the overall market for ML in consumer devices, cloud, enterprises, IOT, medicine, agriculture, and industry will grow 44.06% between 2018 and 2024^[5]. The research firm McKinsey and Company had projected ML will contribute \$13T in global GDP by 2030^[6]. ML is pervasive: 75% of users' Netflix videos are chosen by ML algorithms. Autonomous driving from Tesla simply wouldn't work without ML. ML is used in drug discovery, silicon chip design, business logic, stock trading, energy distribution, and of course edge computing.

Machine learning isn't a buzzword to throw at a problem. It's vastly more complicated than that and many good books and online materials exist to go as deep as you want on the subject. Machine learning can provide insights through classification or regression. Classification is simply identifying a state, object, or image by a label. A machine learning system built as a

classifier will predict a discrete value (e.g., this image is a dog versus a cat). Regression analysis is another type of learning system used to predict a continuous value, not a discrete value. For example, regression analysis would be used to predict a stock price or value of a home based on multiple criteria. Regression is a statistical technique that uses numerical processes that we learned in algebra like linear regression, curve fitting, and least squares method. These essentially draw a line and predict the next value.

7.4.3 Rules and Decision Systems

Predictive analytics use statistical and machine learning processes to make future decisions based on real-time data as well as historical data. The complexity of predictive analysis can take many forms. In the simplest use case, a rules engine may be designed on the edge computer. A rules engine contains two parts: conditions and actions. Normally these are expressed as logical if-then-else clauses:

```
if (number_vehicles_parked in a smart garage equals the  
max_parking_spots)  
then display "lot full" message on entrance  
else (display max_parking_spots - number_vehicles_parked)
```

This simple example uses smart parking detection systems embedded at each parking spot on some garage. That data is fed to an edge computer that execute a simple rules engine to display how many spots are available and if the lot is full. This example shows the condition:

“Are there available spots in the garage; with the action being to display different messages to vehicles entering the lot.”

We can extend this rules engine to be more sophisticated (and complex) by attempting to emulate human reasoning or intuition into the design. This is also called an expert system and have roots to the 1980s where they grew in popularity as some of the first commercially viable artificial intelligence applications.

The problem with rules engines is that the world consists of analog events. Many systems are not black and white, on or off, true or false, and they can

change dynamically. Another issue is called the knowledge acquisition problem. Expressing all the nuances of a system by a human expert is time consuming and adds significant complexity to the overall rules engine.

Rules engine (and other machine learning models) can make use of a random forest. Not a real forest, but an algorithm that uses multiple models (rules or otherwise) to make a decision of a system. Why multiple models? Essentially one model can be biased. This is especially true when we start looking at deep learning models that can be trained or overfit to certain outcomes. Random forests use multiple models or decision trees to arrive at an outcome. Think of this as having multiple experts program a different set of rules for the rule engine. This creates a “forest” of models and decision trees that can be used to overcome inherent bias.

7.4.4 Time-Series Analysis

The most widely adopted workload for edge machines is time-series analysis for signals and data streams. When we talk about signals, we are actually talking about a list of data values paired with timestamps. That is, data values that change over time and may have dependencies on other signals in a system. Audio data is a form of signal. Temperature measurements are another form of signals. Signals may be binary (0 or 1) or analog in nature.

Good examples of time-series analysis include medical monitoring devices such as EKGs, fluid monitors, breathing tubes, and so on that constantly monitor patients. Consumer and household edge systems also are constantly transmitting time-series data to cloud servers. For example, household thermostats are constantly uploading home temperatures and HVAC status to public cloud databases allowing the consumer to see how efficiently their home heating and air conditioning are working.

7.4.5 Proportional Integral Derivative Controllers

PID algorithms have been extensively used in industrial and factory automation for decades and used in appliances like ovens, coffee makers, and breweries. The first manifestations of PID systems were mechanical in

nature and have roots with James Watt's steam engine invention in late 1700's. There he developed a mechanical governor to control the speed of steam engines. In the 20th century PID system first found their way in Naval navigation and steering controls and later in industrial machinery.

A PID is a mathematical tool to create a "control loop". In other words, this tool allows a machine to find a desired set point (like the temperature of a spinning motor) against the current measured temperature (called process variables: PV). The algorithm will try to correct the system to achieve the desired set point (SP). This is accomplished by using *proportional* control. Here the system will attempt to adjust a variable linearly and expect a linear increase or decrease in the set point. The *integral* portion of PID reacts to the error and the amount of time the error in the set point was observed. If the variable being controlled is not adjusting the set point correctly, the system will make corrections to the system to limit overshoot and undershoot. The derivative portion of the algorithm attempts to dampen the fluctuations of the corrections as they occur.

As an example, the temperature of a motor can be adjusted by slowing down the motor, turning on cooling fans, or maybe changing the ambient temperature of the room to compensate. Without detailing the complexities of the theory and mathematics, a PID will try to fit a curve of where a value current is and where it should be by adjusting other factors (like the speed of the motor). Additionally, a PID looks at the rate of correction over time and adjusts to ensure a timely change to the desired set point. Using an edge system allows for analysis of many more process variables and complex interactions such as dynamically adjusting the set point.

7.4.6 Probabilistic Analysis Systems

Probabilistic models include mathematical constructs such as Bayesian models (aka Bayesian statistics). Bayesian, made famous by Rev. Thomas Bayes's Theorem developed in the eighteenth century, is a statistical method that doesn't use least squares or curve fitting to predict a value or what a system is about to do. These systems determine the probability that an event will occur in the future is based on prior knowledge of the system (that's basically all Bayes models do). Put in real-world terms: "What is the

probability that a machine will fail based on the current temperature of the device?” The result of Bayesian model isn’t a discrete answer but a probability that an event will occur or is occurring.

Bayesian networks are programmed using expert knowledge of the system to determine probability weightings. These types of machine learning algorithms are best suited for applications where the entire system can’t be observed. They are also good for systems that must respond to noisy, error-prone, and unreliable data sources. For example, they are used in industrial IOT and in monitoring time-series data. A probabilistic edge computer may inspect the rate that a temperature is changing in a machine and predict when the machine will fail. Missing data for example will not affect the operation of a Bayesian network since bad or missing data adjust that probability distribution momentarily. If the system collects a proportionally large amount of valid data, poor data will be washed out in the probability distribution.

7.4.7 Deep Learning Models

Deep learning is based on mimicking human brain function by examining patterns represented in data in small subsections to larger abstract areas. This type of machine learning uses mathematical functions to mimic neural paths. These functions need to be trained over and over again before they become useful. This form of ML takes its name from the fact that the network built by these functions can be very deep with many independent layer processing more abstract forms of the data.

The most common form of deep learning is the convolutional neural network (CNN) built upon progressive layers. A CNN will assist with classification and identification of objects, especially in image recognition. It performs this function by reducing an image into a set of feature identifiers such as very small curves, gradients, lines, and color splotches. The first layer of a CNN may have a unique set of very small image features and progressive layers built up upon those smaller features into larger combinations. The process of finding distinguishable marks in a large bitmap image is a matrix math operation called a convolution, hence the name. Determining if a small area of a larger is represented by the feature is

called an activation function. If the function meets a threshold the activation is true, and the next layer of the CNN uses that value for further computation.

7.5 Federated Machine Learning

An interesting new use of edge systems and machine learning is what is called “federated machine learning.” The algorithms we have talked about are used in many edge systems but don’t make use of the distributed nature of edge machines. The models are all programmed or trained centrally (in a cloud). This implies that the training data used in the cloud has been acquired from edge systems, trained, and the models programmed back into the edge machines.

Federated machine learning utilizes the fact that the data doesn’t necessarily need to be transmitted back the cloud for training. Training could be performed locally on each edge machine. There may be potentially many edge nodes that have gathered their own data. We have already said that edge machines are typically smaller and less powerful than a cloud server, why would you perform a complicated task like machine learning training on an edge node? The answer is privacy. Additionally, the data collected by an edge node is a subset of all the data its peers have collected. So, training on a small set of local data is much more tenable than training on a massive data set from thousands of edge nodes.

Many customers value the data that is collected and analyzed from their machines. A manufacturer of pharmaceuticals may have many proprietary and secret processes in their factory processes. They certainly would not want to share that data with their competitors. However, they may all use similar edge computers to monitor their machinery (e.g., mixing tanks) and would like to use the most accurate machine learning models to predict things like viscosity of certain chemicals. They use their edge system to train locally but pass model parameters (such as model weights) to a central node to collect and average. In this way, multiple manufacturers may use the same edge computers and same machine learning algorithms, but the data is kept locally with each customer.

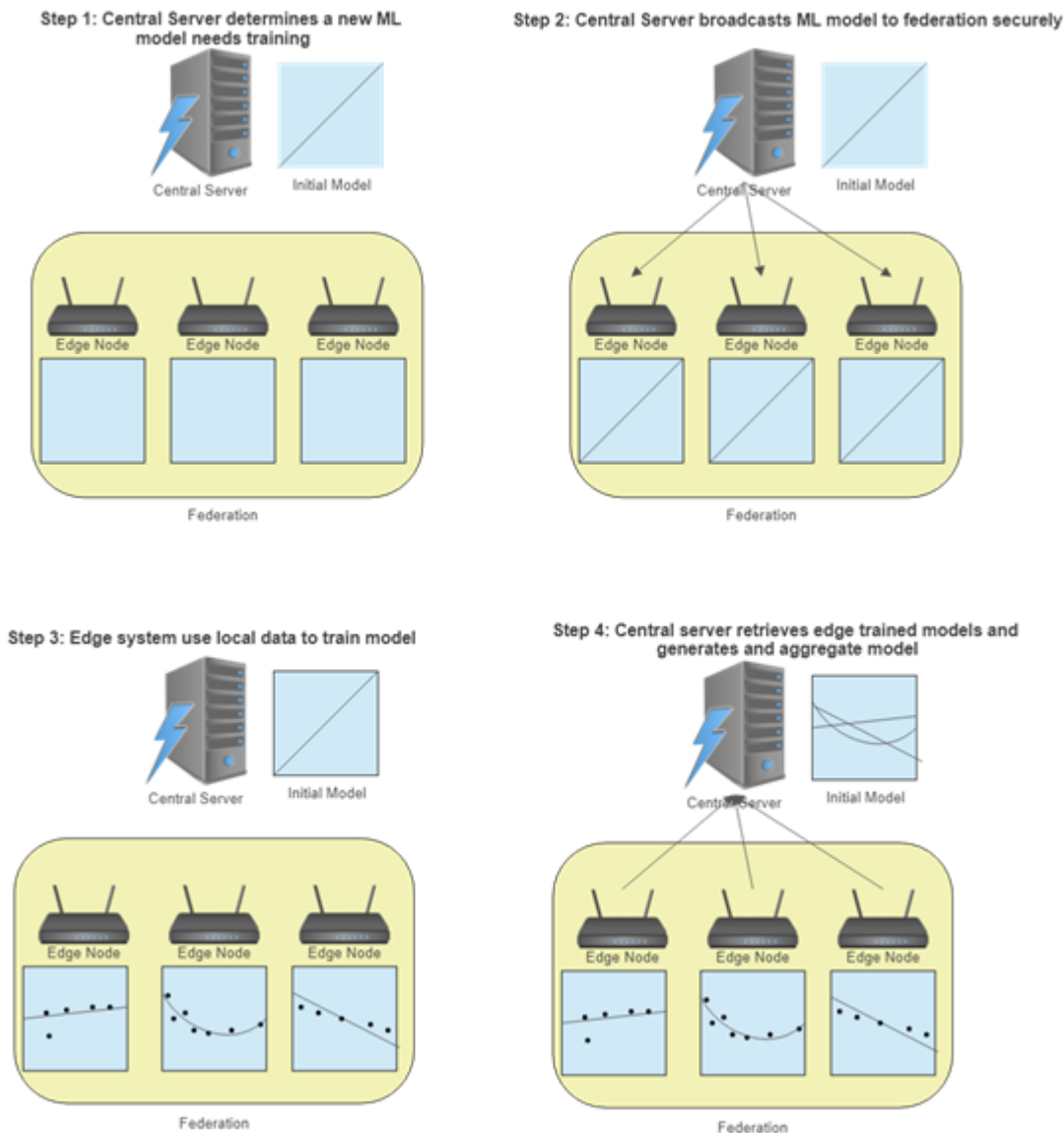
A typical federated machine learning system utilizes a central server (in the cloud) which acts as an orchestrator and parameter collector. The server orchestrates when an edge node should run a training session. For example, our manufacturing customer may have down times when the factory floor is down (e.g., weekends). The server also collects model parameters from various edge nodes and averages the weights and parameters. There may be many edge customers, and each may have many edge nodes. Each node will have collected a rich database of samples to run through the training algorithm. When the model is sufficiently trained to a degree of accuracy, the weights and parameters are then sent to the central server and blended. The server may simply average model weight or use other approaches like regional biasing where model parameters are grouped together by geographical region, customer type, or origin. The new parameters are then transmitted back to each edge node. This will have refined the models for all customers without revealing any secret data.

The steps involved in federated machine learning are as follows:

1. Data is collected locally on an edge node.
2. Central server determines the model and machine learning network to be trained. Securely transmits model to federation.
3. Edge performs local machine learning training on that particular dataset. The user may also trim data, correct errors and provide supervision to training model.
4. The ML parameters of each edge node are exchanged using a central server
5. A shared global model is built on the central server and distributed to each node in the federation.
6. Each edge node reincorporates the global shared model locally.
7. Each edge node now has a broader set of parameters trained securely and privately of from each edge node taking part in the federated network.

This local training does require more computational resources on the edge, but allows for the security and privatization of datasets. Figure 7.3 illustrates the process of federated learning.

Figure 7.3 Four step process of federated learning. Note how local data is used for training on each edge node, but only the trained model is shared while the customer data is kept private.



Federated machine learning is a perfect use of edge systems since the machines are distributed and the data collected stays local and private. Besides manufacturing, we will see future autonomous vehicles collecting data and refining self-driving machine learning models as a federated collective. We may also see federated systems in healthcare, where ML models are diagnosing diseases, but patient data must be protected and private by law. The algorithms are improved during each training run for each customer without sacrificing data privacy. While this technology is

still in its infancy and in the realm of academic research, it is a promising use of edge systems and will become more prevalent.

7.6 Training in the Cloud, Inference at the Edge

All these models represent the execution of machine learning at the source. In other words, they provide inference to classify an event, object or state or predict a future value through statistical, probability, or recurrent methods. We have not spoken to how these models come to existence in the first place, that is the role of training. A rules engine may be trained based on a set of conditions and outcome of an “expert”. Other deep learning models like CNNs need automated and computationally heavy training methods.

There are two fundamental techniques for training:

1. **Supervised Learning:** Training uses a dataset where each value or image is assigned a label. Typically, that label is assigned by a human or group of humans to complete as the training sets may be very large with thousands or millions of images.
2. **Unsupervised Learning:** This method uses no labels for the training set. Rather it utilizes mathematical rules to reduce redundancy. For example, it may use regression to attempt to find clusters of like objects, images, or values.

Training a CNN involves repeated motions through the network. A training set is used to see if the result of an image classifier is correct. If the result is incorrect, a process known as back propagation is used to run the network backwards from classified label back to the first layer and adjust the weights of each node in each layer along the way. This process is repeated many times over until the system stabilizes. In the end, the goal is to have a machine learning system that has a better outcome than a random coin toss of 50% correct or 50% incorrect. We see machine learning models that attempt to achieve 95% accuracy or higher especially in mission critical settings.

One issue with training is overfitting models and bias. Training uses a known dataset of images. This representative set should approximate the

randomness of the real world. What we find is that it doesn't always represent the real analog world well in all cases. Retraining models should be part of an edge solution when new data is received. In this case, if an edge device experiences incorrect prediction, retraining the model based on the newly acquired data should be performed.

For fully automated driving scenarios that use CNNs to identify road and vehicular object, training may take a month of compute time on the most powerful servers in a large hyperscale cloud. The cost can be upwards of \$100M to provide an accurate model for these life critical models.^[7] The result of the training is a network and features that are used to self-driving vehicles. Training is especially problematic as the training sets are massive in size and the amount of processing to handle back propagation is huge. Typically, an edge node may not be fast enough or have enough memory to sufficiently train a model (although we will see how federated machine learning may resolve that). Public and private clouds using large servers with massive amounts of RAM and storage and perhaps graphics processing units or even custom logic like Tensor TPU processors from Google can assist in crunching these massive and time consume datasets and matrix math operations. A modern trend for training is that the process may be distributed across multiple servers and processors throughout the datacenter to further speed the process. Edge systems (even high performance MEC and CoLo servers) typically don't have these resources.

The resulting network after training can and should be offloaded to the edge. Inference systems (whether a decision tree or a complex RNN) are intended to be fast and in many cases real-time. The networks are manageable in size and have one function.

Edge devices offer a unique opportunity for machine learning. Since the edge device usually runs an inference version of machine learning, it is where the ML model is fully tested on real world applications. Sometimes the ML model is wrong (just as human judgement can be incorrect at times). In the event of failure, the edge device can communicate back to the cloud that an error in the model has been detect. In that canse the cloud can absorb new know inputs from the edge node that led to the incorrect result and retrain the model with new data. This process of continually retraining

and improving the model is important and should be part of any edge ML system.

7.7 Proper Use of Machine Learning

Not all machine learning or predictive analysis algorithms apply for every application. There is wide variance in what a machine learning model can do and choosing an inappropriate model will result in system failure. Here is a representative table of how the areas different ML algorithms can be used and typical solutions that already have used them on the edge.

Table 7.1 Machine Learning applications and use cases.

Model	Applicable Areas	Typical Use Cases
Statistical Models: Rules Engine	Simple logic sequences. Small edge systems that result in binary decisions.	Edge gateways, smart thermostats. Factory control systems. Agriculture and livestock management.
Statistical Models: Complex Event Processors	Time-series analysis of sensors. Correlating multiple disparate events.	Logistics and shipping. Municipal fleet management for smart cities.
Statistical Models: PID	Real-time analysis of signals and other time-series events. Control systems.	Industrial automation. Machinery control. Navigation and piloting.

Probabilistic Models: Bayesian Network	Forecasting and predicting future values based on noisy data but structured sources.	Energy monitoring. Manufacturing.
Deep Learning: CNN	Image recognition and classification. Image analytics and security systems.	Smart city surveillance. Factory automation and quality control.
Deep Learning: RNN	Predicting events on time-correlated series of data. Speech and sound recognition.	Apple Siri. Healthcare systems. Factory automation.
Federated Machine Learning	Applicable for large volume edge systems (100's to 1000's of nodes).	Allows customers to use globally trained ML models, but keep their datasets private.

7.8 Summary

- The workloads that run on edge systems can be as simple as a network bridge to the cloud or as complex as self-driving systems in vehicles.
- Successful edge system architects should have answers for the following questions before proceeding with a design:
 - **How much data is generated at each edge node?**
 - **What are the service costs of transporting data to the cloud?**
 - **What are the costs of cloud use?**
 - **Is the system mission critical?**
 - **Can your system afford latency?**

- **How will you manage, secure and scale edge nodes?**
- **Several types of edge analysis and AI are available depending on the correct application and use-case.**
 - Situational awareness applications use edge nodes to monitor and respond to events in real-time close to where the events occur. Examples include military combat applications and municipal emergency systems (e.g., smart cities).
 - Machine learning applications are commonly used on edge systems to supply predictive insights in real-time. Edge systems connect the physical world to the digital world. This offers keen insight into significant amounts of data. ML algorithms are adept in finding patterns within data.
 - Time-series analysis is a prevalent use of edge systems. This type of application is suited to store, analyze, and apply prediction models of data associated with time.
 - Probabilistic models are used on edge devices where data may be noisy or loosely correlated. Bayesian networks are an example of probabilistic models.
 - Deep learning models are a form of artificial intelligence used to find patterns, classify objects or events, or predict a future event. Deep learning models like convolutional neural networks are the basis of functionality like computer vision and automated driving and assistance for vehicles.
- In edge applications most often, the edge is used for inference while the cloud is used for data storage, model training, and model correction.
- Federated machine learning pulls together all these processes of edge AI in combination with many edge nodes contributing to the overall integrity and accuracy of models. Federated machine learning takes advantage of thousands to millions of edge nodes gathering data that is used to build better trained models.

[5] Columbus, L. “Roundup of Machine Learning Forecasts and Market Estimates, 2020”, Forbes, 2020
<https://www.forbes.com/sites/louiscolumbus/2020/01/19/roundup-of-machine-learning-forecasts-and-market-estimates-2020/>

[6] Bughin, J. “Notes from the AI frontier: Modeling the impact of AI on the world economy”, McKinsey & Co. 2018,
<https://www.mckinsey.com/featured-insights/artificial-intelligence/notes-from-the-ai-frontier-modeling-the-impact-of-ai-on-the-world-economy>.

[7] Khan,S & Mann A. “AI Chips: What They Are and Why They Matter”, Center for Security and Emerging Technology, April 2020.

8 Security at the Edge

This chapter covers

- Understanding types of information attacks and exploits.
- Reviewing use cases where edge security vulnerabilities have been compromised resulting in substantial financial damages.
- Defining hardware-based security for edge systems.
- Identifying software and systems-based security best practices.
- Analyzing communication and networking security.

Security must not be overlooked or considered an appendage to a system—any system. We live in a world where there is a greater propensity that a device or system will be compromised maliciously than fail on its own. Security, as they say, is only as good as the weakest link in a chain. With edge system the “chain” just got longer and is far more distant and remote. The edge is the most attractive medium for recent systemic attacks on infrastructure. The reason the edge is so powerful is that the number of edge computers and IOT devices has grown exponentially, while the number of data center servers and personal computers has not. Edge systems are considered a weak point. Much of the lessons learned in IT security and datacenters has not propagated to edge systems. We also have to consider new elements of IT security that weren’t paramount in corporate facilities or buttoned-up data centers. This chapter will provide a treatment of all the considerations around edge security as well as draw analogy from compromised systems.

8.1 Types of Security Vulnerabilities

Typically, there are three forms of malicious cyber-attacks:

1. **Penetration of a system for theft or sabotage.** When we think of the lone hacker, using a backdoor or stolen credentials to infiltrate a system, we think of a penetration attack.

2. **Monetary and ransomware attacks.** These include the prevalence of ransom-ware attacks that are essentially viruses or malicious code intended to encrypt or hijack personal systems, records, data, and photos from a user. These forms of attacks infect one user or a corporate system with the intention that the data is so valuable the user will be forced to pay the attacker (using cryptocurrency for anonymity).
3. **Distributed Denial-of-Service Attacks (DDOS).** This form of attack is intended to overwhelm a system and bring a target's infrastructure to a crawl or prevent users from accessing a service altogether. Typically, this involves saturating a target system with innocuous requests, but doing so with millions of agents all accessing a system simultaneously.

It should be understood that no system is foolproof no matter how hardened it may seem. A designer should use every standard form of security hardware and software available to minimize the *attack surface*. The attack surface can be thought of as what vulnerabilities are exposed on a system that an attacker could exploit. A closed system with no form of network communication to the outside world in a locked box with no method to physically manipulate it is still vulnerable. If the system still uses power in some form either from a public utility or regenerative with solar, an attacker could still exploit the system by cutting its source of power. In the next section we will dive into real-world examples of edge level security failures and their causes.

8.2 The Most Pervasive Internet Hack: Mirai

The IT world has been plagued with various sophisticated attacks in the last 30 years. Hacking and computer crime is nothing new. Various attacks have been around in forms for 50 years. It was not until the Internet became pervasive that cyber-attacks escalated. That event occurred in 1998 with the Morris worm which took advantage in a defect in common Unix mail systems at the time across an early version of the Internet called Arpanet. The worm affected mainframes running Unix and caused roughly \$100,000 to \$4 million dollars in damage. Since then, the number and level of attacks has grown tremendously. Here we will detail some of the most profound cyber-attacks that affected edge systems.

8.2.1 Mirai

If you ever thought the internet in aggregate couldn't be taken down, you are wrong. As matter of fact, the internet's major content and eCommerce providers were all abruptly cut from service on October 21, 2016. This included shutting down access to Amazon, Netflix, Paypal, HBO, Github, AirBnB, and Twitter. Over the months from August 2016 through November 2016, millions of users were shut out of one system or another across the globe. This was the result of the largest global attack of the internet ever recorded and to the date is the most prevalent and malicious attack globally. The attack was from an internet worm called Mirai.

The Mirai worm was a denial-of-service attack. The remarkable trait that made Mirai unlike any other attack prior was that the agents used to create a massive denial-of-serve were not commandeered PC's and servers. Rather the attackers used edge systems as the vehicle to create massive outages over several months. Mirai worked as a self-replicating and self-propagating worm that actively searched for and found vulnerable IoT and edge devices to host the attack. Once a device had become infected, the worm would use that device to randomly scan the internet accessible by the device for other potential hosts. Penetration of a device was not overly complicated. Mirai used a small database of sixty-four often used logins and passwords to attempt to gain access to a system. These included such login/password combinations as:

Login: admin	Password:1111
Login: admin	Paswword:1234
Login: guest	Password: guest
Login: root	Password: admin

Once penetrated, the user of the edge device had no knowledge there was any malicious access. The devices behaved normally and without any notion that it was a compromised agent. At this point, the device is considered a *zombie*. That is, it has every appearance of being functional and usable, but has been compromised with nefarious code. If the zombie device found a compatible system and successfully broke in, it would search the system to ensure the payload of malware was compatible. The infection actually searched the system for other competing worms and malware and removed

them! It obfuscated its footprints by deleting the downloaded malware as well.

If the device found a compatible host and successfully installed itself, it would communicate with a central command and control system (C&C). A C&C is typical of a botnet since there is a central controller instructing infected nodes who and when to attack next. After the hacker identified a target, he or she would instruct the C&C to inform the army of zombie devices to launch a mutually coordinated denial-of-service attack on a specific company called “Krebs on Security” (<https://krebsonsecurity.com>) that hackers purposefully intended to disable. This included flooding the web site with HTTP data, UDP data, or TCP data. This volumetric attack would force the website to be unable to respond to normal events and in many cases, completely shut down.

On its first day, it had found and successfully infected 65,000 systems. Within three months it had successfully found and created 600,000 attack vectors. These devices ranged from consumer Wi-Fi routers, internet-connected web-cameras, air-quality monitors, set-top entertainment boxes, and other IoT devices running Linux. The attack on Netflix and Paypal were the result of Mirai being instructed by the C&C to direct its denial-of-service attack focus on Dyn. Dyn is a popular Domain Name Service (DNS) provider. Essentially DNS providers (like Google) translate a name like www.microsoft.com to a set of IP addresses like 104.87.6.4. Anytime you type a web site name in a browser a DNS lookup must occur at least once. Mirai was instructed to target its zombie nodes to Dyn in October of 2016. This unfortunately affected the name resolution to anyone requiring service from Amazon, Netflix, and Paypal on that day. As matter of fact, not all 600,000 nodes were used; measurements suggest only 107,000 zombie edge systems were involved in the compromise.

Denial-of-service attacks are measured by the amount of overwhelming data being generated to saturate a target’s infrastructure. To date, the Mirai attack was the most profound and strongest ever recorded at 623 gigabits of malevolent data generated per second.

When we deconstruct this attack, some things became clear and give us concern of how prevalent and vulnerable edge computing is. First, the attack

was not sophisticated. The penetration parameters used 64 typical login/passwords combinations. Most of these were set by the manufacturers as default credentials for devices. Second the ability to inject code given administrative control is fairly easy in an edge system with capable resources. Many of these systems allowed for root access (administrative control), which allowed the attacker to change the software unbeknownst to the user. Even access to these systems were gained with ease as ports and access to the devices were left exposed. We will see in the following sections how to “harden” edge systems and things to consider when building or purchasing such devices.

8.2.2 Grand Theft Auto

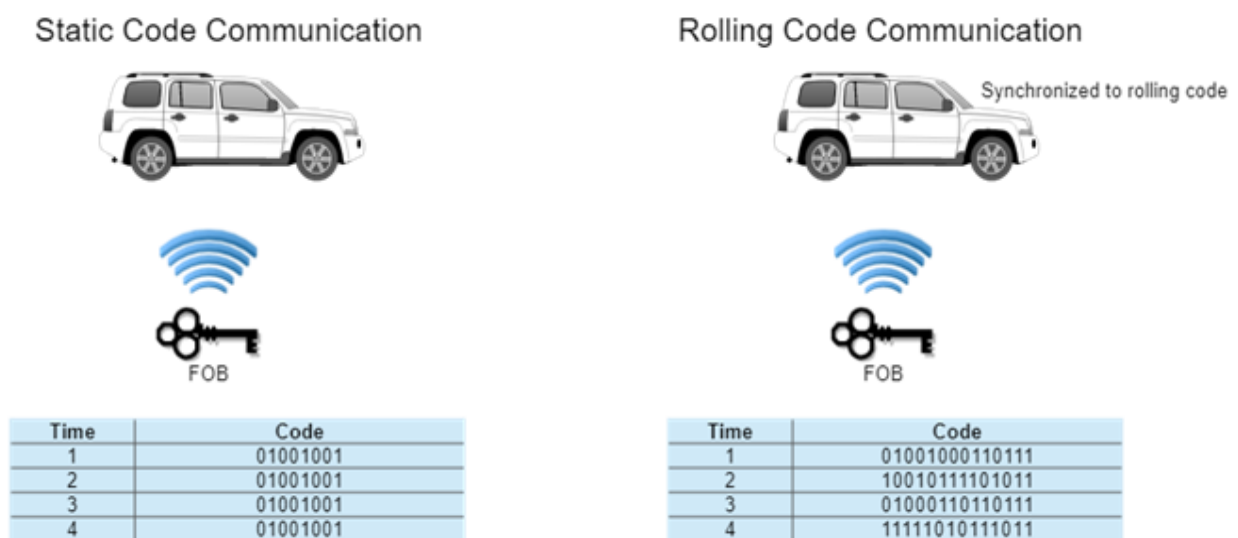
In 2022, Europol arrested 31 individuals and seized \$1.1 million dollars of stolen assets in a number of European countries. The perpetrators had stolen a number of vehicles, predominately Honda manufactured cars from 2018 to 2020. In this situation, the perpetrators used what is called a *replay attack*.

The attack surface here was the communications link between keyless FOBs and the vehicle. The keyless FOBs operated in the 433.215 MHz frequency range and used unencrypted transmissions to open and lock the doors remotely. The wireless protocol was not a standard protocol and did not randomize the frequency, encrypt communication, nor create a random code each time the FOB was used. This left a wide attack surface for vehicular theft, all without breaking a window or doing any cosmetic damage to the vehicle.

To perform the attack, the attacker used a simple appliance that allows for digital communication in various frequency ranges. The tool (called a *Software Defined Radio or SDR*) can simply listen to a specific frequency range and record all data transmitted. It can also record all data received and allow the attacker to modify it. In this example, the attacker used the tool to sniff for information in the keyless FOB specific frequency range. The attacker had to be in proximity to the vehicle, but that usually isn't a problem for someone intent on grand theft. The attacker simply listens and records the FOB digital signal. If the code retrieved isn't encrypted or randomized it can be reused indefinitely to open the car. That is exactly how this exploit occurred.

Of course, the best way to prevent such an attack is to use encrypted, randomized codes and rolling codes in wireless communication. A rolling code is simply a pseudo-random code that is transmitted with the transmitter and receiver both synchronized to the change in pattern. In this case, a vehicle and FOB are in lockstep with the changing pattern of codes. The code sequence rolls over and reuses the sequence after some number of iterations. You may wonder, how does this work if I am not in proximity of my car and I hit the button several times as once might do in real-life. In this case, the FOB and the vehicle are not in sync anymore. To remedy this, the next time the user tries to unlock their car from the FOB, the vehicle will receive the incorrect sequence and determine that the FOB is now using a future sequence of codes and not the next immediate code it was expecting. It will then adjust its sync point and start using the new sequence.

Figure 8.1 Static and Rolling Code Communication Example



Early we said, no system is foolproof, even a rolling code could be manipulated and exploited using more sophisticated attacks. The attacker will first start by jamming the radio signals to prevent an authentic FOB from communicating normally with the vehicle. Jamming simply involves using the RF generator described early to generate a high amplitude signal in the same frequency range of the FOB. The vehicle owner attempts to use their FOB, but the jammer prevents the car from receiving a quality signal, and it ignores the user. The attack will also record the rolling code being sent

by the FOB. This doesn't immediately grant the attacker access to the rolling code sequence but requires a second step.

The frustrated owner will then hit their FOB again to lock their car. The attacker simply jams the frequency a second time. In this case, the attacker immediately replays the first sequence and the door locks. However, the attacker has two binary sequences, the first and the second transmission). After the victim leaves their vehicle area, the attacker can now replay the second sequence and have access to the vehicle.

8.2.3 Credit Card Fraud: Through the HVAC Supplier

Another successful attack using edge-based computing was the circuitous path taken by hackers in 2014 to obtain credit card identifications from Home Depot customers. The effect was that over 56 million credit cards had been illegally obtained and had cost the company \$179 million and required Home Depot to offer credit monitoring services to all potential victims (<https://www.infosecurity-magazine.com/news/home-depot-to-pay-2725m/>). This was not the first case of credit card fraud that year. Earlier Target stores had exposed a month's long security breach executed in almost the same manner. In the case of Target, 40 million credit card numbers and credentials were illegally obtained.

How did one of the biggest credit card scams come about? Was there a sophisticated legion of hackers that broke into the corporate databases and central computers? No, they got into Target's IT security through the third-party suppliers' credentials for the edge computers controlling HVAC maintenance in each of their department stores. A series of cascading security failures then allowed the attackers to effortlessly install dangerous malware that would sit idle on customer point-of-sale (POS) terminals and scrape memory for signatures of credit cards which were then sent to a Ukrainian and Russia source. The malware would inspect DRAM on the edge POS computers anytime a card was swiped. Since the contents of the card information was held in plaintext form in memory it didn't need to be decrypted.

These cascading failures of this attack included:

- The fact that third-party vendor credentials allowed unfettered access to Home Depot IT infrastructure, and those credentials worked outside of the corporate LAN.
- Upon logging in with stolen credentials, the thousands of POS terminals and cash registers also existed openly and visibly on the corporate LAN. Typically, you want to separate or layer security with monetary devices on a separate virtual LAN (VLAN) segment. Hardware switches with strict access control lists are the only bridges between a corporate LAN for email and POS terminals for financial transactions.
- The attackers understood and could infiltrate the POS terminals easily. The terminals were based on very old and insecure embedded operating systems without routine patches for security exploits.
- There was little logging of data and traffic to see the attack unfolding. Using networking monitoring and logging tools would have allowed the patterns of data traffic to not go unnoticed for months compromising even more customer's credit information.
- Finally, confidential data like credit card numbers were stored in plaintext on the POS computer. The designers probably did not consider that malware software could be installed and search through memory for confidential information. A better solution would have been to ensure all data from IO to destination is encrypted end-to-end, even in local memory.

8.3 Security Architecture

So how do we circumvent these security issues? What can we expect and what should your customers demand? Security needs to be considered for every component in an edge machine. That is, we have to consider security in hardware, middleware and software, communications, and even the packaging of the device. Remember, edge systems are an extension of the cloud and are Internet connected to be called an edge machine. They are also often physically located in remote or unsecured areas. Because of these abilities, edge machines have a greater surface area than many traditional IT devices.

We break up the security problem into constituent parts in the following subsections.

8.3.1 Hardware Security

Edge computing's power is placing high performance and pervasive computing close to data sources. These sources can be on a factory floor, in a self-driving car, or in a consumer's home like a webcam or doorbell.

Because of the widespread adoption of these appliances and the fact that they don't have the convenience of physical corporate or office security (a secured door to a data center), these machines have an element of physical security that traditional IT devices didn't focus on.

There are four elements to hardware security I recommend to customers:

- The device should be physically hardened from not only malicious attackers who want to compromise a system, but also from the natural elements that surround and edge box. A device should also test and verify that its physical security has not been damaged and report findings quickly. Logging of physical events like opening the external enclosure should also be logged.
- The device should be trusted from the first line of code that is executed to the last. Utilizing modern hardware paradigms help maintain this confidence. Additionally, modern silicon and CPU architectures include a variety of features to assist in security and limit exposure to malicious events, especially in software.
- All data on the device should be encrypted and protected with the strongest levels of security available. This is especially true for things like user credentials, SIM identification, and fiduciary data. These types of credentials should reside in their own protected environment.
- Unused interfaces and hardware should be turned off or, if possible, permanently removed from the device. This includes no-loading parts and removing interfaces that provide an attacker with another vector.

Examining these paradigms one at a time, we start with physical security. This includes the chassis and box that the edge device resides within. Many edge computers like Schneider Electric residential power meters and 5G small cell antenna arrays must weather the elements as they will be deployed

in harsh environments with wind, rain, and varying degrees of temperature. Within the industry, “Ingress Protection” or IP rating is a standard measure of how a device will perform with varying degrees of environmental effects. These standards are captured in the MIL-STD-810 (military), IEC 60529 (International Electrotechnical Commission), and RTCA/DO-160 (Radio Technical Commission of Aeronautics). IP ratings are a two-digit numeral index. The first digit provides a rating on how the device will protect against foreign bodies like dust or human appendages. The second digit describes how the device will protect against moisture such as indirect rain, being blasted with a high-pressure hose, or complete submersion. For example, an IP rating of IP67 will provide complete protection from all dust using a vacuum seal and provide 30 minutes of full submersion in a fluid at a depth of 1 meter.

Like modern IT equipment, edge systems should provide a root-of-trust (ROT). A ROT is simply a mechanism that provides attestation that the software it is about to run is genuine and authenticated. It also provides security that any additional software it loads will also follow the same attestation paradigm. When a computer is first powered on, a process known as “bootloading” or “boot-strapping” occurs to bring up initial low-level software, then an operating system, then user applications. There are multiple phases to booting a computing, and a hardware root-of-trust provides confidence in this boot chain. Because roots-of-trust are inherently trusted, they must be secure by design. As such, many roots of trust are implemented in hardware so that malware cannot tamper with the functions they provide. The functions within this secure hardware module include cryptographic hash hardware, encryption engines, random number generators, and tamper proof microprocessors. The device can also securely store keys used in running the computer. The root will boot itself before other components or the main microprocessor boots.

A typical module that should be part of the hardware security to provide a root-of-trust is a TPM module. TPM stands for “trusted platform module”. The functions and security validation of TPM modules is part of the ISO /IEC 11889 International Specification. Modern IT equipment and operating systems like Microsoft Windows rely on TPM to form a root-of-trust, and certainly secure edge computers should consider its adoption.

Often hardware is left “as-is” meaning there is an assumption that unused ports and interfaces are naturally secure, and it is better to have the ports usable than artificially disabled. That is not true. By ports, we are referring to all hardware interfaces like unused USB ports, serial ports, SDIO ports, internal debug ports, general purpose IO ports, and even display ports. The reason ports pose a risk is they open the opportunity for an attack vector to be exposed. For example, the most opportune vector would be a debug port in hardware. This could be a serial port or a special interface called a JTAG (Joint Test Action Group) port. Both could allow an attacker with access to the hardware to connect a simple terminal or JTAG emulator to the device and collect information on the device, underlying operating system, and debug logs. Even worse, a JTAG device kept live and unsecured could allow the attack complete control of the hardware with the ability to step into running software, alter it, and view contents of memory. It is best to perform a security audit and disable all unused ports and high-risk interfaces before a production rollout.

8.3.2 Software Security

The software layers running on an edge machine should also be designed with a “security-first” model. Any exploit that has been witnessed in traditional IT infrastructure can be exploited on edge hardware. We have seen 30 years of exploits of various internet worms, zero-day exploits, computer viruses, and ransomware in personal computers. In the early 2000’s, many PCs and networks were compromised due to security flaws in software, operating systems, and middleware. Viruses like the CodeRed in 2001 and the Blaster worm in 2003 were examples of early rapidly spreading viruses that penetrated systems by compromising commercial software. They used techniques called buffer-overflows to inject more data than allowable between software interfaces (APIs). Malicious code was piggybacked to the end of a normal message and that extra payload would find its way into executable portions of system memory. In the end, hundreds of thousands of PCs and workstations had been infected. Today software exploits still exist and can come in the form of ransomware (WannaCry in 2017) where user data will be encrypted and only released back to the user upon payment of ransom.

Edge systems have played “catch up” in terms of security. Since edge systems comprised everything from small, embedded computing devices to large servers using various operating systems and hardware, they don’t immediately share from the history of hacks of traditional IT systems over the last 20 years. A malicious agent is always looking for the weakest link in a system to exploit it, as we learned from the attack on Target stores. Edge systems provide many less secure pathways that have been patched or remedied in modern PC and IT systems. Therefore, historically less sophisticated exploits have an opportunity to attack weaker and less secure edge systems. It is imperative to ensure modern edge computing devices use the latest methods to secure themselves.

Such methods include:

- Using CPUs and operating systems that make use of non-execution protection or executable space protection. This is a hardware and software concept to ensure that areas of software that actually execute code are allowed to and areas that may not execute software (stack and data space) may not. It sounds trivial, but this is a method to prevent buffer-overflow code injection. Modern processors like ARM and x86 cores provide this level of protection, but older and smaller microcontrollers may not.
- Address space layout randomization (ASLR) is another hardware and software mechanism to randomize where software resides in memory. Often a hacker will exploit or reuse privileged portions of commercial code to perform nefarious actions. For example, they may inject code that calls a normally safe routine to open a network port. If the address space of memory and where code resides is randomized this makes it prohibitively difficult for the attacker to call a routine.

One extremely important facility that must be considered for all edge systems is patching and update services. A system must be designed with the a priori knowledge that software will need to be updated in the field. Updating software is necessary for a number of reasons such as adding new functionality or fixing defects. It also is required to provide security updates and patches. All software running on a machine should be upgradable including boot code, operating systems, and runtime applications. When a number of edge machines are deployed in the field, a process and system

must be developed to update, validate, and control software deployment. We talked earlier about central control being cloud based and this makes perfect sense a correct partitioning of cloud and edge services. Most cloud providers have services that allow for secure and managed software updates to field devices. The cloud agent will have knowledge of all running machines and can stage software rollouts after they have been provisioned. A manifest of each edge device is maintained, which is essentially a database of patches and software versions that had been rolled out to each appliance. The cloud agent also will roll out updates based on edge downtime. For example, in a content delivery network, the cloud central agent will roll an update during times of low demand (e.g., during early morning hours). When an edge device is being managed for a software update, the machine must have a recovery mechanism. Essentially, if the update fails, it should securely roll back software to a manifest of previously working software versions. This ensures constant uptime and fail-safe recovery.

Another important aspect in building secure software is to securely log events within a system. Logging is important, as we saw in the attack on Target stores. Had the software changes and machine behaviors been logged and relayed to an administrator, the amount of reciprocal damage could have been mitigated. By logging, as much information that can be tracked should be without affecting the underlying resources of the system like storage space and performance. Telemetry data should be relayed to a central cloud agent to look for anomalies and determine if there are changes in the pattern of usage. Normally, edge systems follow a certain pattern of CPU use, memory usage, and network traffic. Anomalies can be uncovered using statistical and machine learning techniques to detect if and when a change in behavior occurred. This would allow the administrator time to shut down certain edge nodes to prevent collateral damage. Logs should be collected on criteria such as:

- Errors and system events: memory exhaustion, storage errors, illegal code execution, etc.
- Attempted and failed login attempts
- New ports that have been opened or closed
- Network ingress and egress data rates
- CPU usage
- Inbound or outbound network accesses

- Thermal events (overheating)
- Privileged code execution
- Amount of storage data written and read
- Access to privileged accounts and services like administrator or root logins and secure shell (SSH) access.

Software must also utilize encryption wherever and whenever possible in data transit and data storage. There are three basic forms of encryption:

- **Symmetric key encryption:** This is the most basic form of encryption where a single key is used to encrypt as well as decrypt the same data. AES and 3DES are algorithms that use symmetric keys.
- **Public key encryption:** This form of encryption relies on separate keys. A private key is used to decrypt data while a public key that is generally available can be used to encrypt a message. TLS (Transport Layer Security) used in networking is an example of public key encryption.
- **Cryptographic hashes:** This is a “one-way” method of encryption. Hashes are used to validate software authenticity, device attestation, and securing data at rest. Typical hash functions are SHA2 and SHA3.

The designer of an edge system should choose the latest cryptographic standard available. Chose SHA3 versus SHA2 when considering hashing functions. Choose AES256 (256-bit blocks) versus legacy AES-128 or even older technologies like 3DES that utilize 56-bit keys. Preferably use public key encryption. This is a proven and modern standard that is difficult to break, but not impenetrable. The most sensitive aspect of encryption is the protection of the private keys which must be stored on the edge device. Sophisticated attackers will attempt to gain access to these keys and then have access to any similar edge machine.

Another important paradigm in the architecture of a robust and secure system is to *remove* software. Just like hardware interfaces that are unused or only used for development and debugging; software too should only deploy what is absolutely necessary for field operation. Adding a myriad of software libraries, tools, and interfaces widens the attack vector field, giving attackers more opportunities to exploit a system. The designer should audit and scrub a production release of debugging information and tools, extra libraries, and

dead or unused code. For example, during development of an edge machine, the convenience of having a web server on a host may be useful, but a web service on an edge machine along with all the peripheral software to make it run is a huge risk and opens up several network vulnerabilities.

8.3.3 Communications and Network Security

Networking and communication services must also be secure. A general principle is to encrypt and secure all data from source to destination. Plaintext data should rarely be used in modern systems as most hardware now includes extremely fast AES encryption mechanisms that can secure all data. Data in motion can be intercepted and read or altered. It should be assumed that all data leaving an edge machine is not secure unless the designer explicitly secures it.

Both PAN and WAN communication should be encrypted. For the most part, many of the modern communication protocols presented in this book have inherent security that can be used. Often security will be enforced. For example, most cloud providers will not accept data from edge nodes that are not using standard encryption services. On the WAN side, a common encryption protocol is Transaction Layer Security or TLS. TLS is used in protocols like MQTT, CoAP, and various other network stacks. TLS replaces the older Secure Sockets Layer (SSL) protocol popular in the 1990's with a much more secure form of encryption using the SHA-256 algorithm. In short, TLS operates with a host server (e.g., cloud service) sending a digital certificate to the edge device (client) with a public key. The edge confirms the authenticity of the key and uses it to create cyphertext-encrypted data to be sent to the server.

A philosophy of security is called Zero-Trust. This is especially relevant to edge and IOT devices as they are communicating with each other or to a public cloud remotely. Zero trust assumes there is no network edge. Anything that can be communicated with is considered local and directly connected. Zero trust requires all devices capable of communicating on a given network to be authenticated, authorized, and continuously validated. All devices will be verified to meet the configuration and postures allowed before access is granted to join the network. This sounds obvious, but in traditional networks, the rule of thumb had been trust but verify. That is, if a

user was within a company firewall and perimeter, they were allowed to join a network. They could join a company network with an infected or compromised device only to have their device verified sometime later. This gave time for the malicious software to do its damage. Zero Trust validates user identity, device credentials and authenticity, geographic location, firmware versions, operating system and patch levels, even requiring certain applications be installed or removed. All this must be performed before the Zero Trust management allow the user and device to access their network. The National Institute of Standards and Technology maintains the Zero-Trust standard called NIST 800-207.

Some rules of network and communication security are:

1. Always use End-to-end encryption. Treat all IO whether it is wireless, wired, local, or wide area as being compromised. Anytime IO leaves the edge computer or travels to it, it is not in one's control. At all times, use standard encryption methodologies such as TLS 1.2.
2. Monitor data in transit. Log data and respond to events that are anomalous. As stated, most edge devices will behave in a predictable manner and variances to data flow may be the result of malicious attacks.
3. Close unused ports and services that are not used. TCP/IP allows for 65,536 ports to exist behind an IP address. Ports are used to access email services (port 25), web servers (port 80), and so on. Having open ports on the internet is one of the most common methods of malicious code attacks. Hackers routinely scour the internet looking for machines with exposed ports (e.g., HTTP, SSH, email etc.). Another recommendation is to change well known ports for services to other random ports. This obfuscates the attack surface (but does not remove it completely).
4. Consider a firewall for data traffic. Firewalls are hardware or software-based systems that prevent unauthorized access to a computer network. They are used to ensure users without access cannot interact with a particular system. It controls Internet traffic inbound and outbound from a device. Without a firewall, virtually any data can exit your computer or network. In an edge system, both the edge device and the cloud should have firewalls. Generally, any network facing device can

employ a firewall.

There are various types of firewalls:

- a. Packet Layer Firewalls: analyzes all network data inbound and outbound and inspects all data for malicious data that can infect your network.
 - b. Circuit Level Firewalls: This is built into the network stack (refer to the OSI model in this book). It monitors the handshake process when using TCP/IP between two machines. Specifically, it's ensuring the handshake process is genuine and authentic.
 - c. Application Layer Firewalls: These ensure that data being generated by an edge application is authentic and has not been compromised. Essentially, it ensures the data in transit has not been tampered with.
5. Create whitelists and access control lists (ACL) to only allow connection to clients that have been authenticated.
 6. If there are attempts to gain electronic access through brute-force attacks. The device should register and log those events, but also block IP addresses that are generating repeated attempts to gain access.
 7. Another level of security is segmentation and micro-segmentation. Segmentation essentially divides a network into specific slices. These slices can be isolated to specific users and devices. For example, one might place IOT devices on a different segment than mission critical servers. Access control lists (ACL) and Virtual Local Area Networks (VLAN) can be set up to logically place a device in its own quarantined network. While all the physical infrastructure (e.g. Ethernet cabling) is the same, the logical networks are partitioned. Micro-segmentation takes this a step further and allows different applications to be safely routed to different network segments on the same machine. This limits the *blast radius* if by chance a device launches an attack on the IOT VLAN but the corporate LAN remains intact.

8.3.4 Physical Security

We also must consider the security of the device itself. In some edge use cases that we have examined the device may be in a remote area and cannot rely on security found in datacenters or enterprise locations. A modern data center has significant physical and location-based security. Only select

badged and authorized employees may enter a hyper-scale data center. Datacenters also provide monitoring, camera surveillance, and guards. An edge system providing real-time analysis in an agriculture environment will not have this level of security.

Additive security includes tamper detection. For example, if the device is opened either purposely or maliciously, sensors should detect those events. Events should be logged and broadcast to the cloud (or other edge systems) to indicate a physical event is transpiring. Sophisticated edge systems also include key access or biometric sensors to open a chassis. This allows for individual control and authorization to the device.

Physical security starts with the machine itself. The edge device must supply reactive intrusion and tamper resistance. What that implies is that the device should be aware if its chassis has been opened or dislodged by an unauthorized person. Authorization may imply it needs card access or a unique key to open the device. However, that does not prevent an attacker from damaging the enclosure to gain access or simply stealing the entire device. The edge machine should also have tamper resistance circuitry. If the device has been forcibly opened, it should register the information, and signal the upstream cloud that a threat has been detected. While the machine may be lost, it aids in litigation and for insurance to report the event.

8.4 Final Thoughts on Security

In conclusion, security never ends. You must be prepared to update and remediate security issues on the fly and in the field. Security must be designed from the start and applied to each layer (hardware, software, and networking) equally to avoid a weak link in the chain. Above all, assume all data in transit can be intercepted and encryption of data is a “must” requirement from edge to the cloud and back.

Security issues can be devastating and embarrassing situations, it is best to think about security first and early in design as edge systems have become the new target of widespread attacks.

8.5 Summary

- Security should not be an afterthought. Security should be the foundation for a successful edge machine. You are more likely to experience a cyber event than have modern machinery fail.
- Security is based on the surface area of attacks. Surface area is the exposed elements of a system that an attacker can interface with (e.g., networks, installable malware). Edge devices are unique as their surface area is significantly larger than other IT machinery.
- The typical attacks include denial of service, monetary attacks, and penetration/theft.
- We explored three large scale recent attacks that have affected edge computing:
 - Mirai was the single largest denial-of-service attack in the world. It used backdoors to install malware on retails set-top boxes and surveillance cameras. IT resulted in millions of dollars lost and degradation of the entire Internet.
 - Edge systems are also vulnerable to radio and communication attacks. We examined a spoofing and replay attacks that compromised millions of dollars in stolen vehicles using attacks on keyless FOBs.
 - Edge systems have also been used for credit card theft using backdoors of HVAC edge computers. This attack surface then opened up other surfaces on point-of-sale cash registers that allowed the attackers to install malware.
- Hardware security considers elements in silicon and at the electronics level that should be included in a system. Unlike software that can be updated, hardware must deliver security from day one. Baseline hardware security measures include:
 - Ensure the CPU and chipset provide memory protection against software malware.
 - Ensure all non-volatile data is encrypted on edge storage devices.
 - Removal of unused interfaces and electronic ports that are not used in a product but may be necessary for development.
 - Ensure the system includes boot security and countermeasures like Trust Platform Modules. These provides a root-of-trust to ensure the system boots with known good software and operating systems.
- Software security includes:

- Ensure you have a fail-safe remote software upgrade process. Keep the system updated with all security patches and fixes.
- Utilize CPU features to aid in security like address space randomization.
- Edge systems are known for sensing in IOT environments. The edge system should also sense itself. This implies monitoring and logging events and finding potential threats. This is analogous to many of the analytic software running on edge systems except it will monitor the health and well-being of its own system.
- Utilize software encryption, especially for all data in flight inbound and outbound to the cloud.
- Communications security:
 - Use end-to-end encryption.
 - Monitor and log data and network activity. These are critical to analyzing and identify anomalous activity.
 - Close unused networking ports and services that are unused. Anything that reduces the attack surface areas should be considered.
 - Integrate a firewall on the edge to monitor for nefarious data traffic before it causes a problem.
 - Create whitelists of allowed networks that can access the edge system. Also, consider reactive measures to thwart brute-force attacks by closing and disallowing certain networks or IP addresses.
- Physical security needs to be considered as well since edge systems are often in remote and unprotected areas which make it attractive to physical tampering.

9 Edge computing futures and predictions

This chapter covers

- Understanding the numerous factors and constraints of edge computing economics.
- Various regulation and standards that should be considered for edge systems.
- Future trends in edge computing including pervasive IOT systems, energy harvesting, new battery technologies, virtual reality and augmented reality, synthetic sensing, and new industrial edge applications.

Throughout this book we have studied the technical nuances of edge systems. We have learned that edge systems come in many shapes and sizes to tackle problems to where data is consumed or produced. We have also learned that edge systems combine hardware, software, communications, and cloud components. We have to ask how can edge systems be monetized and effectively deployed commercially. What are the futures of edge machines? Early in this book we mentioned the pendulum the industry has taken moving from centralized mainframes to PCs and then to cloud computing. Will edge computing be just another IT fad? This chapter will make the bold predictions of edge computing. To be successful, edge machines must add value and make economic sense for a company to invest in that technology.

9.1 Capitalizing on Edge Computing

A successful edge or IOT project must consider the economic value. Let's use an example of a simple edge use case. This could be a simple hobbyist temperature monitor of their hot-tub. Even in this simple case there are two factors that impact the cost:

- **(CAPEX)** The initial capital expense for the thermal sensor, edge computer, and wide area network hardware. That may be ethernet, Wi-Fi, cellular, and so on.
- **(OPEX)** The cost of energy to power this computing device and network hardware. Additionally, the cost of network communication through an internet service provider.

Such a small device can be built with hobbyist hardware like a Raspberry Pi (\$35), cables and miscellaneous hardware (\$15), and a cellular modem (\$50). This brings the initial capital expense (CAPEX) to roughly \$100. Power can be considered an operational expense (OPEX), although very little will be consumed by a small embedded system. A Raspberry Pi in a low power state will consume roughly 2 watts of power with an additional 1 watt for the cellular radio. Based on typical North American power rates of roughly 13.31 cents per kWh, the cost of a month's power would be \$0.29. Now when we examine the rates for communication service, that varies considerably based on data rates, data caps, and service type (satellite, cellular, cable broadband, DSL). For contrast, typical data rates for various services at the time of this writing is captured in the following table:

Table 9.1 Contrast of service level agreement plans for various carries.

Service	Average North American Cost	Average Speed	Data Cap
Wired Broadband ^[8]	\$60/month per location	290Mbps	1TB
Cellular LTE ^[9]	\$56/month per device	5 to 12 Mbps (LTE)	10GB
Cellular NB-IoT ^[10]	\$2.50/month per device	80 to 127 Kbps	500KB

LoRa	\$51/month 250 nodes	27 to 50 Kbps	Unlimited
Starlink Satellite	\$99/month per location	100 Mbps	Not Determined

These data rates and caps must be considered for the use case. Edge machines transmitting unstructured data like video data will consume significantly more bandwidth and require larger data caps than something transmitting intermittent temperature data from a hot tub.

For this to be successful, it must provide more value than the \$100 of upfront hardware cost and more value than the \$2.79 monthly energy and data rate plans. Often these charges must be passed on to your customer. Most if not all IoT and edge computing devices at scale have a recurring monthly service fee.

We haven't broached the cost of integrating with a cloud provider. They are a contributing factor in the overall OPEX and must not be ignored. Hyperscale cloud providers have similar plans that charge for various services such as data ingestion, data storage, access control and identity management, and cloud computing. Different bundling of packages and level of service can be obtained (or negotiated).

While a single edge device is interesting, commercial deployments must consider mass deployments and scale. As an example, a development I worked on provided connectivity to 1000 semitrucks in a large fleet to monitor hundreds of different parameters of the health of the engine and vehicle. These were connected via cellular WAN to the cloud. In aggregate, the costs per month were several thousand dollars. Should that operational expense be managed by the edge provider or the customer? That is a recurring question on how edge devices are managed and how they are sold. In the case of the fleet, the customer managed the operational expenses.

9.2 Regulations and Standards for IOT and Edge Computing

It is important to recognize standards in information technology and how they affect edge computing. Edge devices may find themselves in financial transactions, medical devices, and military use cases. Because of this, there are established standards and compliance criteria that must be demonstrated to sell into certain markets. We present an overview of standards that should be considered when selling into such markets.

Financial Transactions: The Payment Card Industry Data Security Standard (PCI DSS) is a security standard for organizations that process credit cards. There are four levels of compliance dependent on the volume of transactions that are processed annually (20,000 to over 6 million). Elements of the standard that will affect edge devices include provisioning of firewalls on network traffic, removal of vendor defined passwords, encrypting and hashing stored cardholder data, and protections against malware. PCI certification requires testing and various compliance audits.

Medical: Edge devices used in some medical use cases require special handling and regulation. Some requirements include the UL 2900 specification (Underwriters Laboratory) and the Health Insurance Portability and Accountability Act HIPAA certification. Both target medical healthcare providers, patients, and insurers from fraud and theft. Details of HIPAA regulations are voluminous and beyond the scope of this chapter or book. One can expect strict guidance on methods of encryption for data at rest and data in transit, access control, requirements, unique identification, usage monitors and logging, and automatic log-off of idle computers.

Self-Driving Vehicles: Automated driving and assistance systems (ADAS) have significant regulations to protect the health and safety of drivers and pedestrians. There are significant and established regulations for vehicles in general managed by the US National Highway and Safety Administration. These include standards for nearly every component of a vehicle (https://www.nhtsa.gov/sites/nhtsa.dot.gov/files/documents/motor_vehicle_safety_unrelated_uncodified_provisions_may2013.pdf). The ISO 26262

specification should also be considered for electronics involved in self-driving vehicles. The 26262 standard serves to define the functional safety of automotive electronics and electrical systems, including edge computing devices. The standard defines rules for the production, testing, and servicing of ADAS equipment. Additionally, the 2017 Self-Driving Act intended to federally manage the design and testing of automated vehicles and to override disparate state regulations of such vehicles (<https://www.congress.gov/bill/115th-congress/house-bill/3388>). That bill has yet to become law.

Communication Certification: Edge systems are defined and valued by their internet connectivity. It cannot be assumed that using one form of communication in one region will work in another region across the globe. Take for example, cellular WAN technology. Cellular systems need device certification to work in a specific country. This can be a lengthy and costly process to ensure that a cellular system meets regional rules regarding frequency distribution, power, and so on.

In lieu of the pervasiveness of edge computing and IOT, regulatory bodies have proposed legislation and standards to ensure security of systems. Security issues have come to the attention of congress after such IOT attacks like the Mirai botnet (studied earlier in this book) and nation-state cyberwarfare such as the Stuxnet virus (<https://spectrum.ieee.org/telecom/security/the-real-story-of-stuxnet>). One such bill in the United States was the internet of Things (IoT) Cybersecurity Improvement Act of 2019 (<https://www.congress.gov/bill/116th-congress/house-bill/1668>). The original bill was sponsored in 2017 and became law on 12/4/2020. The bill requires the National Institute of Standards and Technologies (NIST) as well as the Office of Management and Budget (OMB) to increase cybersecurity standards for IOT devices. The bill requires NIST to develop and publish standards and guidelines for the federal government on the appropriate use and management by agencies of IoT devices owned or used within the federal government. We would expect NIST to provide requirements such as software authentication and remote upgrade security, removal of deprecated communication protocols and encryption standards, methods for edge computers to isolate and remediate threats, methods of multi-factor authentication of edge devices,

and written certification from government contractors that machines meet such standards.

We have explored US federal regulatory specifically, but it should be known that international regulatory bodies have differing standards. Europe makes use of the European Union Agency for Cyber-Security Issues and Security of internet of Things. Australia utilizes the IoT Alliance of Australia to set security standards. It is wise to understand the regulatory rules and biases for each region you intend to scale to.

9.3 Future of Edge Computing

We conclude this work with forward looking view of the future of edge systems. Some technologies exist today but haven't been fully capitalized upon. The analyst firm Gartner has said "by 2022 more than 50% of enterprise-generated data will be created and processed outside of the data center or cloud" (B. Gill, D. Smith, "The Edge Completes the Cloud: A Gartner Trend Insight Report", Gartner, September 2018). The processor manufacturer, ARM, has stated that by 2025 there will be 200 billion edge and IOT systems and 1.45 trillion-megabytes of data generated per day. 40% of that data will be generated by machines on the edge. We aren't there yet, but we have the technologies and capabilities to achieve that goal. The edge will not replace the cloud but complement it to serve certain functions. Early in this book, we defined edge computing as serving the following goals:

- Reducing system latency.
- Reducing the cost of cloud.
- Bridging networks.
- Providing resiliency and robustness.
- To provide content security and filtering.
- Building an extension of the cloud.

Technologies that generate edge data or reach billions of IoT devices are in emerging areas of computing that should be monitored. Here we capture some of the interesting technologies that have the propensity to further define the edge.

We continue to enjoy the benefits of hardware advances that can bring more computing power to the edge and provide high quality communication to devices. We now can entertain very small and power efficient embedded devices that allow for the connectivity of things that had never been “connected”. Imagine if the costs of embedded computing became much lower that the operational costs exceed the cost of hardware? Earlier we stated that a \$100 cost for a small Raspberry Pi hobbyist machine. We can foresee a future where that hardware may broach \$10 and still have the computing capabilities of a larger more expensive machine. Now the cost of a cellular data plan would outweigh the costs of the actual hardware in a month of use. This changes the dynamics of edge computing.

9.3.1 Pervasive Edge Computing, Smart Concrete

If the cost of computing and connectivity became this low, we can start envisioning ARM’s predictions of 200 billion devices or more. We then anticipate the idea of things like “smart concrete”. Small, embedded nodes that are on the order of \$1 or less could be injected into setting concrete for roadways, bridges, and even building structures. These devices would constantly monitor stress and sheering loads in infrastructure and provide first warning of imminent failure. Currently the monitoring of infrastructure is a slow and manual process that addresses surface issues. Here a device would be set into concrete and monitor the structure for the life of the building.

9.3.2 AR/VR

We should also talk about larger edge machines and some upcoming technologies being thought and planned for by the largest technology companies. While augmented reality and virtual reality systems and headsets have existed since the 1990’s and have been commercialized by Microsoft with the HoloLens and Facebook with their Oculus Rift, these systems are still not pervasive in everyday life. One needs to consider the amount of computing necessary to manage an AR/VR headset. Consider these devices must render and display video graphics up to 8K in resolution on two lenses simultaneously at 60 frames per second. They must also include sensors monitoring head and body position, audio processors, and

hand movement, all in real-time. These systems require extremely low latency to avoid a sickening experience for the user. AR/VR systems also need connectivity to be useful. Additionally, these devices must be light enough to sit conformably on one's head and power efficient enough to run for hours. These challenges are being conquered through the iterative engineering process. It should be anticipated that various AR/VR appliances will come to fruition in the next ten years allowing the connectivity to millions of users. These systems will require advances in edge computing to manage these constraints, but the market does exist and should be prepared for.

9.3.3 Immersive Interactions and Synthetic Sensing

Immersive interaction includes new use segments such as smarter homes. Today we enjoy consumer “smart home” devices like Amazon Alexa that provide voice interactive smart assistance. Future devices will expand the capability of stationary home assistants and mobile assistants. These devices will become much more aware of the environment and situation in which something occurs. For example, a future home assistant will respond to more than just queries like “Alexa, give me a recipe for apple pie”. Rather a device will understand situational context like who is in the home and what group of people are in range of the device and listening. It will understand the owner's behavior from the time they wake, to when they eat, to when they sleep. It will understand emergencies and watch over the home and the users in case of emergency. We will even see edge systems assisting in elder care and memory care of individuals, providing them safety as well as assistance when needed. All these features will require more processing on the edge and interconnections to several different sensors and systems.

Two elements on the emerging technology horizon that contribute to immersive systems include synthetic sensing and situational awareness systems (<https://doi.org/10.1145/3025453.3025773>). Synthetic sensing is the combination of different physical sensors (temperature, heat, electromagnetic radiation, sounds, etc.) that are combined and normalized in time and passed to a machine learning inference engine that determines what the combined sensors reveal. A single temperature sensor may simply

report changes in ambient heat. Whereas the combination of various sensors can reveal that the oven was left on overnight.

9.3.4 Devices that Understand You

Imagine a machinery that is actively making predictions and assisting your life and wellbeing. Beyond just the convenience of a smartphone map, think about a smartphone that interfaces with traffic, weather, your calendar, your kids' whereabouts, your home, parking lot at work, your to-do list, and your personal health metrics. This is a system that brings together a vast array of disparate information to make sense of the situation at hand or a situation that may occur. Situational awareness is like synthetic sensing in that it is a system to derive context of what is happening around us.

Situational awareness systems will extend beyond just connected sensors to extracting information from various local and remote sources. For example, a situational awareness system could be used in agriculture systems to monitor both the irrigation and soil moisture content, but also extract information from weather forecasting sources like NOAA to avoid over and underwatering situations. Another example of edge based situational awareness is in smart cities. Edge systems can monitor freeway traffic in real-time, weather conditions, and past models of traffic flow to adjust speed limits and lane usage for optimal traffic flow during rush hours.

9.3.5 Industrial and Systems Control

We have already talked about SCADA systems and industrial controllers as ripe for IoT and edge computing. There is significant room to improve edge systems in factory machinery. Many systems on the shop floor are so called "brownfield" devices meaning they have been used in operation for years or decades. Not all factories are Tesla "Gigafactories" with significant robotic controls and automation, rather many industries amortize their investment in machinery over decades. Small businesses and industries represent the 99% of all business in the US.^[11] Amongst these are older factories that are looking for methods to improve manufacturing yield, worker safety, and product quality. Edge systems can provide these opportunities by passively monitoring machinery for failure, assisting with product quality pre and postproduction through rules engines and machine

learning, and monitoring the health and safety of workers through vision systems. These techniques allow a legacy manufacturer to improve through processes without incurring significant capital expense.

9.3.6 Innovation in Sensors and Electronics

The edge is where the physical world meets the digital world. New innovations in sensors and electronics will enable devices to reach more things than ever. One example of electronics is printable circuits or flexible electronics. Throughout this book we have talked about hardware and naturally we think of electronics as being a relatively stiff and inflexible set of ceramic and copper material on a fiberglass board. However, new innovations are enabling electronics to be malleable and flexible. Electronics are “printed” on circuit boards made of sheer plastic or similar membranes. This allows them to bend and conform to irregular surfaces. This innovation will enable computer to be woven into wearable materials to monitor health and fitness. Edge systems may be enabled and even powered by your own body heat.

9.3.7 Batteries and Energy Harvesting

The obvious issue with Smart Concrete is providing power to the millions of these edge sensors deeply embedded into hardened concrete mix. The costs become much larger if you must provide cable power to each node along an interstate. This is where advances in energy harvesting come to play. The easiest way to describe energy harvesting is a device that sources its own power without reliance on connecting to the power grid or rely on a battery. Although a battery device may augment an energy harvester, most of the time they are self-sufficient. Solar is a classic example of an energy harvesting technology that has been around for decades. However, energy may also be harvested using induction of converting radio signals that surround us into electricity. Temperature differences also can be used to manufacture energy. Even motion and stress of cars passing along a highway can generate electricity through the uses of micro electromechanical machines. These are miniature energy plants that can be designed into silicon like a microprocessor. Energy harvesting will become a pervasive technology in the coming decade. Already, there are sensor

systems to detect open doors and windows for security that harvest energy from the motion of someone opening the window or releasing a latch on the sensor. This is not an extraordinary amount of energy produced, but enough to power a sensor and a small computer for a brief period of time (e.g., 100 ms) to register an event. Other harvesting systems are being developed to feed on radio signals like Wi-Fi in a room as an energy source.

Another exciting area of rampant innovation is in battery technology and energy sources. All the edge computers here rely on electricity to be present. Many remote edge systems already run on small rechargeable lithium batteries or renewable systems like solar. New types of IOT devices will gain their power from other sources. Battery technology is considering using solid state batteries that will replace lithium ion. Solid state batteries are based on solid electrolytes and carbon free components. They have significantly higher energy density than current lithium-ion batteries of about 10x. Additionally, they are not prone to fire and smoke damage associated with compromised lithium devices.

9.3.8 MECs

MECs refer to multi-access edge or mobile-access edge and is an integral part of 5G cellular. MECs allows for low-latency, localized data, and an expansive IOT network. 5G carriers can now process data locally close to mobile subscriber's device rather than shuttle data through the Internet or to a cloud provider. For example, a cell tower (also called a Radio Access Network RAN) could host MEC edge hardware and services.

MECs hold promise that new types of end-user application and services will be developed. For example, content delivery networks could rely on MEC systems to store locally cached copies of movies and videos close to the end user. Data and services can also be localized to what is important to a specific coverage area. For example, MEC systems in an agricultural area could be used to provide edge services for smart-agriculture systems in automated farming, milk production, and cattle location tracking.

9.3.9 Democratized Communication Systems

We have spent a good part of this book understanding communication systems and networking as that defines the edge. This section calls out the operational expenses associated with WAN networks like cellular. A bold prediction of the future is pervasive and extremely low-cost communication for edge devices. Normally a cellular or WAN provider builds the cellular infrastructure around the region or country they service. ATT, Verizon, and T-Mobile all have a vast cellular network in the US with thousands of towers and Radio Access Networks (RANs). These commercial carriers establish proprietary cellular service.

An emerging trend is WAN infrastructure provided by individuals, private homeowners, and private citizens. Rather than relying on a large telco to provide WAN connectivity, an adhoc network is built by private citizens to span large areas of coverage. Most notably is a project called Helium, which is a decentralized network. Helium provides LoRaWAN network connectivity for IOT devices. In many LoRaWAN systems the infrastructure needs to be built by the edge or IOT user themselves. This includes the central gateways, antennas, and power supply. Helium relies on a large number of individuals to provide gateway and wireless coverage in their own homes and property. This allows IOT devices, sensors, and edge system to have a large nationwide coverage without relying on a commercial 5G or cellular provider.

The incentive for individuals to provide these services is they are rewarded in a form of cryptocurrency for their effort. Each Helium gateway generates rewards in the form of Helium crypto for data that is transferred through their hotspot (this is the term for the Helium gateway). They also are rewarded for “proof of coverage”. Hotspots are occasionally given tests to cryptographically prove that they are supplying radio coverage in a particular place and a particular time. These tests are assigned randomly and automatically, and passing them earns a hotspot operator Helium reward (called HNTs). This is much different than traditional cryptocurrency miners that rely on extremely powerful and power-hungry hardware and graphical processing units (GPU) to “mine” cryptocurrency like Bitcoin or Ethereum. These types of crypto miners use thousands to millions of crypto servers in one location to provide validation of currency transactions through the process of mining. Helium provides proof of coverage based on

location. Each location (which may be a private residence) can only have one hotspot/gateway. As matter of fact the algorithm requires hotspots to be 300meters apart or the algorithm will degrade rewards given to an individual. This encourages wide network coverage. It only takes 50 to 100 Helium hotspots to provide full coverage for a metropolitan area.

9.3.10 Video Gaming and Entertainment

The final area we watch is in video gaming and entertainment services. We have already explored how Netflix pioneered edge computing for content delivery networks in their streaming catalog. To address their users' needs for low latency and high-quality video streaming services, Netflix has used CDN systems in different geographical areas. Online video gaming has now become an industry of its own. Whereas in the past video gaming required a PC or console in the home, today content can be generated by powerful servers in the cloud and streamed to any player regardless of their location. Users can play on a mobile device over a 5G or LTE network or play on a couch in their home. Video gaming expands the constraints and issues of CDN systems profoundly. Whereas a movie streaming from Netflix will have the data pre-compressed and ready to be streamed from anywhere in the movie, video games must render and create a new image for every frame at 30 or 60 frames per second. Additionally, they must respond to user events from a controller or joystick in milliseconds of time to be playable. This bidirectionality and strict latency requirements will necessitate further improvements in edge systems and CDN networks. CDN systems for entertainment and gaming can significantly lower the latency of bidirectional data which is critical to gaming.

9.4 Summary

What have we learned here:

- To be commercially practical, edge systems must consider the full cost impact in terms of capital expenses (CAPEX) and operational costs with service level agreements (OPEX). As systems scale it is imperative to consider the costs between communication carriers and cloud providers.

- Dependent on the use cases, it is necessary to consider how the device and edge service will be constrained by regulations and commercial standards. For example:
 - Financial transactions requiring PCI compliance.
 - Self-driving vehicles managed by ISO 26262 ADAS regulations.
 - Medical IOT systems requiring HIPAA compliance.
 - Additionally, new legislation is being drafted by the federal government in the United States to govern IOT and edge security.
- Future edge computing use cases:
 - Pervasive compute: as edge systems continue to cost reduce and scale through Moore's Law, we will witness compute everywhere. A good example is low-cost sensors embedded into concrete and roadways.
 - New battery technologies: new solid-state batteries will eventually replace lithium-ion to allow for greater energy storage in a smaller area as well as improve the safety.
 - Energy harvesting: new electro-mechanical devices will allow for sensors to be self-powered through mechanical motion.
 - AR/VR: A great example of the power of the edge where wearable devices like AR/VR headsets will connect to the cloud but provide low latency rendering on the person.
 - MECs: Integral to 5G, MECs allow for edge computing on 5G carrier hardware allowing for edge computing power and 5G service to be aggregated.
 - Democratized communication: new adhoc community driven networks will provide pervasive connectivity as an alternative to commercial networks.
 - Video Gaming and Content Delivery Networks: Shifting from video streaming systems, the future interactive systems demanding ultra-low latency will rely on edge computing to provide services.

[8] <https://www.highspeedinternet.com/resources/how-much-should-i-be-paying-for-high-speed-internet-resource>

[9] <https://www.allconnect.com/blog/the-average-cost-of-a-cellphone-plan-in-the-us-is-113>

^[10] <https://marketplace.att.com/products/att-nbiot-one-rate-plan-296251>

^[11] <https://www.sba.gov/sites/default/files/advocacy/2018-Small-Business-Profiles-US.pdf>

A friendly introduction

Edge Computing

Perry Lea

MEAP

 MANNING

